Proyecto 1: Dashboard de Análisis de Datos de Ventas y Financiero

Dificultad: 🟡 **Media-Baja** (Enfoque en Lógica, Arrays, Objetos y DOM básico)

Un panel donde el usuario introduce o carga una serie de transacciones/ventas para calcular métricas avanzadas, filtrar resultados en tiempo real y persistir los reportes localmente.

- **¿Qué hace el proyecto?**
  - Procesa un listado de datos complejos (objetos dentro de arrays).
  - Calcula el total de ingresos, promedio, producto más vendido y ventas por categoría.
  - Permite filtrar por rangos o categorías, ordenar de mayor a menor y guardar preferencias.
- **Temas cubiertos del temario:**
  - **Objetos y Arrays:** Manipulación con `Object.keys()`, `Object.entries()`, literales.
  - **Métodos avanzados de Arrays:** Combinación intensa de `map()`, `filter()` y `reduce()`.
  - **Funciones:** Funciones flecha, anónimas (callbacks), buenas prácticas de modularización.
  - **DOM:** Generación dinámica de tablas/cards con `createElement()`, `appendChild()`.
  - **Persistencia:** Guardado de informes y configuraciones en `localStorage`.
- **Puntos de dificultad para los estudiantes:**
  - El razonamiento lógico para encadenar `filter()` y `reduce()` sin ensuciar el código.
  - El cálculo del elemento con más frecuencia utilizando un acumulador de tipo Objeto en `reduce()`.

https://github.com/aristides1000/Dashboard-de-An-lisis-de-Datos-de-Ventas-y-Financiero---Uneweb---javascript-nivel-2

https://aristides1000.github.io/Dashboard-de-An-lisis-de-Datos-de-Ventas-y-Financiero---Uneweb---javascript-nivel-2/

```
<!DOCTYPE html>
<html lang="es" class="dark">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Dashboard Financiero y de Ventas - JavaScript Nivel 2</title>
```

```
    <!-- Tailwind CSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>
    <script>
        tailwind.config = {
            darkMode: 'class',
            theme: {
                extend: {
                    colors: {
                        slate: {
                            850: '#111827',
                            900: '#0f172a',
                            950: '#080d1a',
                        },
                        brand: {
                            emerald: '#10b981',
                            cyan: '#06b6d4',
                            violet: '#8b5cf6',
                            rose: '#f43f5e',
                            amber: '#f59e0b'
                        }
                    },
                    fontFamily: {
                        sans: ['Inter', 'sans-serif'],
                    }
                }
            }
        }
    </script>
    <!-- Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;60
0;700&display=swap" rel="stylesheet">
    <!-- FontAwesome Icons -->
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.
min.css">
    <style>
        body {
            font-family: 'Inter', sans-serif;
            background-color: #080d1a;
            color: #f3f4f6;
        }
        /* Custom Scrollbar */
        ::-webkit-scrollbar {
```

```
            width: 8px;
            height: 8px;
        }
        ::-webkit-scrollbar-track {
            background: #0f172a;
        }
        ::-webkit-scrollbar-thumb {
            background: #334155;
            border-radius: 4px;
        }
        ::-webkit-scrollbar-thumb:hover {
            background: #475569;
        }
        .glass-card {
            background: rgba(15, 23, 42, 0.75);
            backdrop-filter: blur(12px);
            border: 1px solid rgba(255, 255, 255, 0.08);
            box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
        }
        .glass-input {
            background: rgba(30, 41, 59, 0.6);
            border: 1px solid rgba(255, 255, 255, 0.1);
            color: #f8fafc;
        }
        .glass-input:focus {
            border-color: #06b6d4;
            outline: none;
            box-shadow: 0 0 0 2px rgba(6, 182, 212, 0.25);
        }
    </style>
</head>
<body class="min-h-screen pb-12 antialiased">

    <!-- Header Principal -->
    <header class="border-b border-slate-800 bg-slate-900/80 sticky
top-0 z-50 backdrop-blur-md">
        <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4">
            <div class="flex flex-col md:flex-row md:items-center
md:justify-between gap-4">
                <div class="flex items-center space-x-3">
                    <div class="p-2.5 bg-gradient-to-tr from-cyan-500
to-blue-600 rounded-xl shadow-lg shadow-cyan-500/20">
                        <i class="fa-solid fa-chart-line text-xl
text-white"></i>
                    </div>
                    <div>
```

```
                        <div class="flex items-center space-x-2">
                            <h1 class="text-xl font-bold tracking-tight
text-white">Sales & Financial Analytics</h1>
                            <span class="px-2.5 py-0.5 text-xs
font-semibold bg-cyan-500/10 text-cyan-400 border border-cyan-500/20
rounded-full">JS Level 2</span>
                        </div>
                        <p class="text-xs text-slate-400">Demostración
práctica: Objetos, Arrays, Métodos Funcionales y DOM</p>
                    </div>
                </div>

                <!-- Botones de Acción Global -->
                <div class="flex items-center space-x-3">
                    <button id="btn-seed" class="px-3.5 py-2 text-xs
font-medium text-slate-300 bg-slate-800 hover:bg-slate-700
hover:text-white rounded-lg border border-slate-700 transition flex
items-center gap-2">
                        <i class="fa-solid fa-rotate-left
text-cyan-400"></i> Restaurar Datos
                    </button>
                    <button id="btn-toggle-code" class="px-3.5 py-2
text-xs font-medium text-cyan-300 bg-cyan-950/60 hover:bg-cyan-900/80
rounded-lg border border-cyan-800/60 transition flex items-center
gap-2">
                        <i class="fa-solid fa-code text-cyan-400"></i>
<span id="code-toggle-text">Ver Guía Pedagógica</span>
                    </button>
                </div>
            </div>
        </div>
    </header>

    <main class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 pt-8 space-y-8">

        <!-- Banner Educativo Desplegable -->
        <section id="pedagogical-guide" class="hidden glass-card
rounded-2xl p-6 border-l-4 border-l-cyan-500 transition-all
duration-300">
            <div class="flex justify-between items-start mb-4">
                <div class="flex items-center gap-2 text-cyan-400
font-semibold">
                    <i class="fa-solid fa-graduation-cap text-lg"></i>
                    <h2>Conceptos Clave de JavaScript Nivel 2
Aplicados</h2>
                </div>
```

```
                <button id="btn-close-guide" class="text-slate-400
hover:text-white text-sm"><i class="fa-solid fa-xmark"></i></button>
            </div>
            <div class="grid grid-cols-1 md:grid-cols-3 gap-4 text-xs
text-slate-300">
                <div class="bg-slate-900/60 p-3.5 rounded-xl border
border-slate-800">
                    <span class="font-bold text-emerald-400 block
mb-1">1. Métodos de Arrays Avanzados</span>
                    <p class="text-slate-400">Uso de <code
class="text-cyan-300">reduce()</code> para calcular acumulados de
ingresos y mapas de frecuencia, <code
class="text-cyan-300">filter()</code> para búsquedas de texto y
categorías, y <code class="text-cyan-300">map()</code> para renderizado
fluido.</p>
                </div>
                <div class="bg-slate-900/60 p-3.5 rounded-xl border
border-slate-800">
                    <span class="font-bold text-violet-400 block
mb-1">2. Manipulación de Objetos</span>
                    <p class="text-slate-400">Uso de <code
class="text-violet-300">Object.entries()</code> y <code
class="text-violet-300">Object.keys()</code> para transformar
acumuladores por categoría en estructuras navegables en tarjetas de
rendimiento.</p>
                </div>
                <div class="bg-slate-900/60 p-3.5 rounded-xl border
border-slate-800">
                    <span class="font-bold text-cyan-400 block mb-1">3.
DOM Dinámico & LocalStorage</span>
                    <p class="text-slate-400">Creación iterativa de
elementos con <code
class="text-cyan-300">document.createElement()</code>, delegación de
eventos y sincronización en tiempo real con <code
class="text-cyan-300">localStorage</code>.</p>
                </div>
            </div>
        </section>

        <!-- Secciones de Tarjetas KPI Dinámicas -->
        <section class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4
gap-5" id="kpi-cards-container">
            <!-- Total Ingresos Card -->
            <div class="glass-card rounded-2xl p-5 relative
overflow-hidden group hover:border-emerald-500/40 transition
duration-300">
```

```
                <div class="absolute -right-4 -bottom-4 w-24 h-24
bg-emerald-500/10 rounded-full blur-xl group-hover:bg-emerald-500/20
transition"></div>
                <div class="flex items-center justify-between">
                    <span class="text-xs font-medium text-slate-400
uppercase tracking-wider">Total Ingresos</span>
                    <div class="p-2 rounded-lg bg-emerald-500/10
text-emerald-400 border border-emerald-500/20">
                        <i class="fa-solid fa-dollar-sign"></i>
                    </div>
                </div>
                <div class="mt-4">
                    <h3 id="kpi-total-ingresos" class="text-2xl
font-extrabold text-white">$0.00</h3>
                    <p class="text-xs text-slate-400 mt-1 flex
items-center gap-1">
                        <span id="kpi-count-transacciones"
class="text-emerald-400 font-semibold">0</span> transacciones
registradas
                    </p>
                </div>
            </div>

            <!-- Promedio de Venta Card -->
            <div class="glass-card rounded-2xl p-5 relative
overflow-hidden group hover:border-cyan-500/40 transition duration-300">
                <div class="absolute -right-4 -bottom-4 w-24 h-24
bg-cyan-500/10 rounded-full blur-xl group-hover:bg-cyan-500/20
transition"></div>
                <div class="flex items-center justify-between">
                    <span class="text-xs font-medium text-slate-400
uppercase tracking-wider">Promedio de Venta</span>
                    <div class="p-2 rounded-lg bg-cyan-500/10
text-cyan-400 border border-cyan-500/20">
                        <i class="fa-solid fa-calculator"></i>
                    </div>
                </div>
                <div class="mt-4">
                    <h3 id="kpi-promedio-venta" class="text-2xl
font-extrabold text-white">$0.00</h3>
                    <p class="text-xs text-slate-400 mt-1">Por ticket de
venta</p>
                </div>
            </div>

            <!-- Producto Más Vendido Card -->
```

```
            <div class="glass-card rounded-2xl p-5 relative
overflow-hidden group hover:border-violet-500/40 transition
duration-300">
                <div class="absolute -right-4 -bottom-4 w-24 h-24
bg-violet-500/10 rounded-full blur-xl group-hover:bg-violet-500/20
transition"></div>
                <div class="flex items-center justify-between">
                    <span class="text-xs font-medium text-slate-400
uppercase tracking-wider">Producto Top (Frecuencia)</span>
                    <div class="p-2 rounded-lg bg-violet-500/10
text-violet-400 border border-violet-500/20">
                        <i class="fa-solid fa-award"></i>
                    </div>
                </div>
                <div class="mt-4">
                    <h3 id="kpi-producto-top" class="text-xl font-bold
text-white truncate">N/A</h3>
                    <p class="text-xs text-slate-400 mt-1">
                        <span id="kpi-producto-top-count"
class="text-violet-400 font-semibold">0</span> ventas realizadas
                    </p>
                </div>
            </div>

            <!-- Categoría Líder Card -->
            <div class="glass-card rounded-2xl p-5 relative
overflow-hidden group hover:border-amber-500/40 transition
duration-300">
                <div class="absolute -right-4 -bottom-4 w-24 h-24
bg-amber-500/10 rounded-full blur-xl group-hover:bg-amber-500/20
transition"></div>
                <div class="flex items-center justify-between">
                    <span class="text-xs font-medium text-slate-400
uppercase tracking-wider">Categoría Top ($)</span>
                    <div class="p-2 rounded-lg bg-amber-500/10
text-amber-400 border border-amber-500/20">
                        <i class="fa-solid fa-layer-group"></i>
                    </div>
                </div>
                <div class="mt-4">
                    <h3 id="kpi-categoria-top" class="text-xl font-bold
text-white truncate">N/A</h3>
                    <p class="text-xs text-slate-400 mt-1">
                        Ingreso: <span id="kpi-categoria-top-monto"
class="text-amber-400 font-semibold">$0.00</span>
                    </p>
```

```
                </div>
            </div>
        </section>

        <!-- Fila Principal: Formulario + Filtros & Controles -->
        <div class="grid grid-cols-1 lg:grid-cols-12 gap-8">

            <!-- Columna Izquierda: Formulario para Agregar Venta (4
cols) -->
            <section class="lg:col-span-4">
                <div class="glass-card rounded-2xl p-6 sticky top-24">
                    <div class="flex items-center gap-2 mb-5">
                        <i class="fa-solid fa-plus-circle text-cyan-400
text-lg"></i>
                        <h2 class="text-base font-bold
text-white">Registrar Nueva Venta</h2>
                    </div>

                    <form id="form-nueva-venta" class="space-y-4">
                        <div>
                            <label for="input-producto" class="block
text-xs font-semibold text-slate-300 mb-1">Nombre del Producto</label>
                            <input type="text" id="input-producto"
required placeholder="Ej: Monitor Gamer 27''"
                                   class="w-full rounded-xl px-3.5
py-2.5 text-sm glass-input">
                        </div>

                        <div class="grid grid-cols-2 gap-3">
                            <div>
                                <label for="select-categoria"
class="block text-xs font-semibold text-slate-300
mb-1">Categoría</label>
                                <select id="select-categoria" required
class="w-full rounded-xl px-3 py-2.5 text-sm glass-input bg-slate-900">
                                    <option
value="Electrónica">Electrónica</option>
                                    <option value="Ropa">Ropa</option>
                                    <option value="Hogar">Hogar</option>
                                    <option
value="Alimentos">Alimentos</option>
                                    <option
value="Servicios">Servicios</option>
                                </select>
                            </div>
                            <div>
```

```
                                <label for="input-monto" class="block
text-xs font-semibold text-slate-300 mb-1">Monto ($)</label>
                                <input type="number" id="input-monto"
step="0.01" min="0.01" required placeholder="0.00"
                                       class="w-full rounded-xl px-3
py-2.5 text-sm glass-input">
                            </div>
                        </div>

                        <div class="grid grid-cols-2 gap-3">
                            <div>
                                <label for="input-fecha" class="block
text-xs font-semibold text-slate-300 mb-1">Fecha</label>
                                <input type="date" id="input-fecha"
required class="w-full rounded-xl px-3 py-2.5 text-sm glass-input">
                            </div>
                            <div>
                                <label for="input-cliente" class="block
text-xs font-semibold text-slate-300 mb-1">Cliente</label>
                                <input type="text" id="input-cliente"
required placeholder="Nombre cliente"
                                       class="w-full rounded-xl px-3
py-2.5 text-sm glass-input">
                            </div>
                        </div>

                        <button type="submit" class="w-full py-3 px-4
bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400
hover:to-blue-500 text-white font-semibold text-sm rounded-xl shadow-lg
shadow-cyan-500/25 transition duration-200 flex items-center
justify-center gap-2 mt-2">
                            <i class="fa-solid fa-check"></i> Guardar
Transacción
                        </button>
                    </form>
                </div>
            </section>

            <!-- Columna Derecha: Filtros, Tabla y Análisis de
Categorías (8 cols) -->
            <section class="lg:col-span-8 space-y-6">

                <!-- Barra de Filtros y Búsqueda -->
                <div class="glass-card rounded-2xl p-5 space-y-4">
                    <div class="flex flex-col sm:flex-row
sm:items-center justify-between gap-3 border-b border-slate-800 pb-4">
```

```
                        <h2 class="text-base font-bold text-white flex
items-center gap-2">
                            <i class="fa-solid fa-filter
text-cyan-400"></i> Filtros de Datos
                        </h2>
                        <span id="resultados-count" class="text-xs
text-slate-400 bg-slate-800 px-3 py-1 rounded-full border
border-slate-700">
                            Mostrando 0 registros
                        </span>
                    </div>

                    <div class="grid grid-cols-1 sm:grid-cols-3 gap-3">
                        <!-- Búsqueda por Nombre -->
                        <div class="relative">
                            <i class="fa-solid fa-magnifying-glass
absolute left-3.5 top-3 text-slate-500 text-xs"></i>
                            <input type="text" id="filtro-busqueda"
placeholder="Buscar producto o cliente..."
                                   class="w-full pl-9 pr-3 py-2 text-xs
rounded-xl glass-input">
                        </div>

                        <!-- Filtro Categoría -->
                        <div>
                            <select id="filtro-categoria" class="w-full
py-2 px-3 text-xs rounded-xl glass-input bg-slate-900">
                                <option value="TODAS">Todas las
Categorías</option>
                                <option
value="Electrónica">Electrónica</option>
                                <option value="Ropa">Ropa</option>
                                <option value="Hogar">Hogar</option>
                                <option
value="Alimentos">Alimentos</option>
                                <option
value="Servicios">Servicios</option>
                            </select>
                        </div>

                        <!-- Ordenar Por -->
                        <div>
                            <select id="filtro-orden" class="w-full py-2
px-3 text-xs rounded-xl glass-input bg-slate-900">
                                <option value="reciente">Fecha: Más
Reciente</option>
```

```
                                <option value="antiguo">Fecha: Más
Antiguo</option>
                                <option value="monto-desc">Monto: Mayor
a Menor</option>
                                <option value="monto-asc">Monto: Menor a
Mayor</option>
                            </select>
                        </div>
                    </div>
                </div>

                <!-- Tabla Dinámica de Transacciones -->
                <div class="glass-card rounded-2xl overflow-hidden
border border-slate-800">
                    <div class="px-6 py-4 border-b border-slate-800 flex
justify-between items-center bg-slate-900/40">
                        <h3 class="text-sm font-bold text-white flex
items-center gap-2">
                            <i class="fa-solid fa-table-list
text-cyan-400"></i> Registro de Ventas
                        </h3>
                    </div>

                    <div class="overflow-x-auto">
                        <table class="w-full text-left text-xs">
                            <thead class="bg-slate-900/80 uppercase
tracking-wider text-slate-400 border-b border-slate-800">
                                <tr>
                                    <th class="py-3 px-4">Fecha</th>
                                    <th class="py-3 px-4">Producto</th>
                                    <th class="py-3 px-4">Categoría</th>
                                    <th class="py-3 px-4">Cliente</th>
                                    <th class="py-3 px-4
text-right">Monto</th>
                                    <th class="py-3 px-4
text-center">Acción</th>
                                </tr>
                            </thead>
                            <tbody id="tabla-ventas-body"
class="divide-y divide-slate-800/60 font-medium">
                                <!-- Filas generadas dinámicamente con
JavaScript -->
                            </tbody>
                        </table>
                    </div>
```

```
                    <!-- Estado Vacío -->
                    <div id="empty-state" class="hidden text-center
py-12 px-4">
                        <div class="inline-flex p-4 rounded-full
bg-slate-800/80 text-slate-500 mb-3">
                            <i class="fa-solid fa-inbox text-2xl"></i>
                        </div>
                        <p class="text-sm font-semibold
text-slate-300">No se encontraron transacciones</p>
                        <p class="text-xs text-slate-500 mt-1">Prueba
cambiando los filtros o agregando un nuevo registro.</p>
                    </div>
                </div>

                <!-- Desglose Por Categorías (Demostración de
Object.entries y reduce) -->
                <div class="glass-card rounded-2xl p-6">
                    <div class="flex items-center justify-between mb-4">
                        <h3 class="text-sm font-bold text-white flex
items-center gap-2">
                            <i class="fa-solid fa-chart-pie
text-violet-400"></i> Distribución de Ventas por Categoría
                        </h3>
                        <span class="text-xs text-slate-400">Generado
dinámicamente con Object.entries()</span>
                    </div>
                    <div id="categoria-breakdown-container" class="grid
grid-cols-1 sm:grid-cols-2 gap-4">
                        <!-- Tarjetas de Categorías generadas vía JS -->
                    </div>
                </div>

            </section>
        </div>
    </main>

    <!-- Modal de Notificaciones Toast -->
    <div id="toast-container" class="fixed bottom-5 right-5 z-50 flex
flex-col gap-2 pointer-events-none"></div>

    <!-- CÓDIGO JAVASCRIPT PRINCIPAL -->
    <script>
        /**
         *
================================================================
         * JAVASCRIPT NIVEL 2 - DASHBOARD DE VENTAS & FINANZAS
```

```
         *
================================================================
         * Este script implementa todos los conceptos del syllabus:
         * 1. Objetos Literales, Constructores y Manipulación.
         * 2. Métodos Avanzados de Arrays: map(), filter(), reduce(),
sort().
         * 3. Uso de Funciones Flecha y Callbacks.
         * 4. Manipulación Avanzada del DOM (createElement, appendChild,
removeChild).
         * 5. Almacenamiento Persistente mediante LocalStorage.
         */

        // ========================================
        // 1. ESTADO DE LA APLICACIÓN & DATOS DE MUESTRA
        // ========================================

        /**
         * Datos de muestra iniciales (Estructura: Array de Objetos)
         */
        const DATOS_EJEMPLO = [
            { id: "tx-101", producto: "Laptop Pro 15''", categoria:
"Electrónica", monto: 1250.00, fecha: "2026-03-10", cliente: "Carlos
Mendoza" },
            { id: "tx-102", producto: "Camisa Formal Slim", categoria:
"Ropa", monto: 45.50, fecha: "2026-03-11", cliente: "Ana Gómez" },
            { id: "tx-103", producto: "Cafetera Expresso", categoria:
"Hogar", monto: 180.00, fecha: "2026-03-12", cliente: "Roberto Silva" },
            { id: "tx-104", producto: "Monitor Gamer 27''", categoria:
"Electrónica", monto: 320.00, fecha: "2026-03-14", cliente: "Lucía
Fernández" },
            { id: "tx-105", producto: "Consultoría IT (1h)", categoria:
"Servicios", monto: 90.00, fecha: "2026-03-15", cliente: "Tech
Solutions" },
            { id: "tx-106", producto: "Set de Cuchillos Cocina",
categoria: "Hogar", monto: 65.00, fecha: "2026-03-16", cliente: "Mariana
López" },
            { id: "tx-107", producto: "Laptop Pro 15''", categoria:
"Electrónica", monto: 1250.00, fecha: "2026-03-18", cliente: "Empresa
Global" },
            { id: "tx-108", producto: "Paquete Snacks Gourmet",
categoria: "Alimentos", monto: 28.00, fecha: "2026-03-19", cliente:
"David Ruiz" }
        ];

        // Estado Global con persitencia en localStorage
        let transacciones = cargarTransacciones();
```

```
        // ==========================================
        // 2. FUNCIONES DE PERSISTENCIA (LOCALSTORAGE)
        // ==========================================

        /**
         * Obtiene las transacciones guardadas en localStorage o retorna
los datos de ejemplo
         */
        function cargarTransacciones() {
            const guardado = localStorage.getItem('js_nivel2_ventas');
            if (guardado) {
                try {
                    return JSON.parse(guardado);
                } catch (e) {
                    console.error("Error al parsear LocalStorage:", e);
                    return [...DATOS_EJEMPLO];
                }
            }
            return [...DATOS_EJEMPLO];
        }

        /**
         * Guarda el array actual de transacciones en localStorage
         */
        const guardarEnLocalStorage = () => {
            localStorage.setItem('js_nivel2_ventas',
JSON.stringify(transacciones));
        };

        // ==========================================
        // 3. MÉTODOS DE ARRAYS & LÓGICA FINANCIERA
        // ==========================================

        /**
         * Calcula las métricas generales utilizando exclusivamente
métodos avanzadas de Array
         * @param {Array} lista - Lista de transacciones filtradas o
totales
         * @returns {Object} Objeto con los indicadores clave
         */
        const calcularMetricas = (lista) => {
            if (lista.length === 0) {
                return { total: 0, promedio: 0, productoTop: "N/A",
productoTopCount: 0, catTop: "N/A", catTopMonto: 0 };
            }
```

```
            // A) TOTAL INGRESOS: Uso de reduce() para sumar los montos
            const total = lista.reduce((acumulador, tx) => acumulador +
parseFloat(tx.monto), 0);

            // B) PROMEDIO DE VENTA: División entre total y la longitud
del array
            const promedio = total / lista.length;

            // C) PRODUCTO MÁS VENDIDO (Frecuencia): Uso de reduce() con
un objeto acumulador
            const conteoProductos = lista.reduce((acc, tx) => {
                acc[tx.producto] = (acc[tx.producto] || 0) + 1;
                return acc;
            }, {});

            // Determinar cuál clave (producto) tiene la mayor
frecuencia mediante Object.entries()
            let productoTop = "N/A";
            let productoTopCount = 0;

            Object.entries(conteoProductos).forEach(([prod, count]) => {
                if (count > productoTopCount) {
                    productoTopCount = count;
                    productoTop = prod;
                }
            });

            // D) CATEGORÍA LÍDER EN INGRESOS: Uso de reduce() agregando
montos por categoría
            const ingresosPorCategoria = lista.reduce((acc, tx) => {
                acc[tx.categoria] = (acc[tx.categoria] || 0) +
parseFloat(tx.monto);
                return acc;
            }, {});

            let catTop = "N/A";
            let catTopMonto = 0;

            Object.entries(ingresosPorCategoria).forEach(([cat, monto])
=> {
                if (monto > catTopMonto) {
                    catTopMonto = monto;
                    catTop = cat;
                }
            });
```

```
            return { total, promedio, productoTop, productoTopCount,
catTop, catTopMonto, ingresosPorCategoria };
        };

        /**
         * Filtra y ordena la lista de transacciones según la
configuración del usuario
         */
        const obtenerTransaccionesFiltradas = () => {
            const busqueda =
document.getElementById('filtro-busqueda').value.toLowerCase().trim();
            const categoria =
document.getElementById('filtro-categoria').value;
            const orden = document.getElementById('filtro-orden').value;

            // 1. Filtrado combinado (filter)
            let resultado = transacciones.filter(tx => {
                const coincideTexto =
tx.producto.toLowerCase().includes(busqueda) ||

tx.cliente.toLowerCase().includes(busqueda);
                const coincideCat = (categoria === 'TODAS') ||
(tx.categoria === categoria);
                return coincideTexto && coincideCat;
            });

            // 2. Ordenamiento (sort)
            resultado.sort((a, b) => {
                if (orden === 'reciente') return new Date(b.fecha) - new
Date(a.fecha);
                if (orden === 'antiguo') return new Date(a.fecha) - new
Date(b.fecha);
                if (orden === 'monto-desc') return b.monto - a.monto;
                if (orden === 'monto-asc') return a.monto - b.monto;
                return 0;
            });

            return resultado;
        };

        // ========================================
        // 4. MANIPULACIÓN Y RENDERIZADO DEL DOM
        // ========================================

        /**
```

```
         * Formatea números a moneda USD
         */
        const formatearMoneda = (valor) => {
            return new Intl.NumberFormat('en-US', { style: 'currency',
currency: 'USD' }).format(valor);
        };

        /**
         * Renderiza las Tarjetas KPI principales
         */
        const renderizarKPIs = (metricas, countTotal) => {
            document.getElementById('kpi-total-ingresos').textContent =
formatearMoneda(metricas.total);

document.getElementById('kpi-count-transacciones').textContent =
countTotal;
            document.getElementById('kpi-promedio-venta').textContent =
formatearMoneda(metricas.promedio);

            document.getElementById('kpi-producto-top').textContent =
metricas.productoTop;

document.getElementById('kpi-producto-top-count').textContent =
metricas.productoTopCount;

            document.getElementById('kpi-categoria-top').textContent =
metricas.catTop;

document.getElementById('kpi-categoria-top-monto').textContent =
formatearMoneda(metricas.catTopMonto);
        };

        /**
         * Genera dinámicamente las filas de la tabla mediante
document.createElement()
         */
        const renderizarTabla = (lista) => {
            const tbody = document.getElementById('tabla-ventas-body');
            const emptyState = document.getElementById('empty-state');

            // Limpiar contenido previo del tbody
            tbody.innerHTML = '';

            if (lista.length === 0) {
                emptyState.classList.remove('hidden');
            } else {
```

```
                emptyState.classList.add('hidden');
            }

            // Badge de colores para las categorías
            const BadgesCategoria = {
                'Electrónica': 'bg-cyan-500/10 text-cyan-400
border-cyan-500/20',
                'Ropa': 'bg-violet-500/10 text-violet-400
border-violet-500/20',
                'Hogar': 'bg-emerald-500/10 text-emerald-400
border-emerald-500/20',
                'Alimentos': 'bg-amber-500/10 text-amber-400
border-amber-500/20',
                'Servicios': 'bg-rose-500/10 text-rose-400
border-rose-500/20'
            };

            // Iteración con map / forEach para la creación de elementos
DOM
            lista.forEach((tx) => {
                const tr = document.createElement('tr');
                tr.className = 'hover:bg-slate-800/40 transition
duration-150 group';

                const badgeClass = BadgesCategoria[tx.categoria] ||
'bg-slate-700 text-slate-300';

                // Inserción modular del HTML interno
                tr.innerHTML = `
                    <td class="py-3 px-4 text-slate-400
whitespace-nowrap">${tx.fecha}</td>
                    <td class="py-3 px-4 font-semibold
text-white">${tx.producto}</td>
                    <td class="py-3 px-4">
                        <span class="px-2.5 py-1 text-[10px]
font-semibold rounded-md border ${badgeClass}">
                            ${tx.categoria}
                        </span>
                    </td>
                    <td class="py-3 px-4
text-slate-300">${tx.cliente}</td>
                    <td class="py-3 px-4 font-bold text-right
text-emerald-400">${formatearMoneda(tx.monto)}</td>
                    <td class="py-3 px-4 text-center">
                        <button class="btn-eliminar p-1.5 text-slate-500
hover:text-rose-400 transition"
```

```
                                data-id="${tx.id}" title="Eliminar
transacción">
                            <i class="fa-solid fa-trash-can
pointer-events-none"></i>
                        </button>
                    </td>
                `;

                tbody.appendChild(tr);
            });

            // Actualizar contador
            document.getElementById('resultados-count').textContent =
`Mostrando ${lista.length} de ${transacciones.length} registros`;
        };

        /**
         * Genera las tarjetas de resumen por categoría utilizando
Object.entries() y reduce()
         */
        const renderizarDesgloseCategorias = (metricasGlobales) => {
            const container =
document.getElementById('categoria-breakdown-container');
            container.innerHTML = '';

            const ingresosCat = metricasGlobales.ingresosPorCategoria ||
{};
            const totalGlobal = metricasGlobales.total || 1; // Prevenir
división por cero

            const listaCategorias = Object.entries(ingresosCat);

            if (listaCategorias.length === 0) {
                container.innerHTML = `<p class="text-xs text-slate-500
col-span-2">Sin datos de categoría.</p>`;
                return;
            }

            listaCategorias.forEach(([cat, monto]) => {
                const porcentaje = ((monto / totalGlobal) *
100).toFixed(1);

                const card = document.createElement('div');
                card.className = 'bg-slate-900/60 p-4 rounded-xl border
border-slate-800 flex flex-col justify-between';
```

```
                card.innerHTML = `
                    <div class="flex justify-between items-center mb-2">
                        <span class="text-xs font-semibold
text-slate-200">${cat}</span>
                        <span class="text-xs font-bold
text-cyan-400">${formatearMoneda(monto)}</span>
                    </div>
                    <div class="w-full bg-slate-800 rounded-full h-2
overflow-hidden mb-1">
                        <div class="bg-gradient-to-r from-cyan-500
to-violet-500 h-2 rounded-full transition-all duration-500"
                             style="width: ${porcentaje}%"></div>
                    </div>
                    <span class="text-[10px] text-slate-500
text-right">${porcentaje}% del total</span>
                `;

                container.appendChild(card);
            });
        };

        /**
         * Función orquestadora que actualiza la interfaz por completo
         */
        const actualizarDashboard = () => {
            const filtradas = obtenerTransaccionesFiltradas();
            const metricasFiltradas = calcularMetricas(filtradas);
            const metricasTotales = calcularMetricas(transacciones);

            renderizarKPIs(metricasFiltradas, filtradas.length);
            renderizarTabla(filtradas);
            renderizarDesgloseCategorias(metricasTotales);
        };

        // ==========================================
        // 5. GESTIÓN DE EVENTOS & INTERACTIVIDAD
        // ==========================================

        // Evento Formulario: Agregar Nueva Transacción

document.getElementById('form-nueva-venta').addEventListener('submit',
(e) => {
            e.preventDefault();

            // Captura de valores de los inputs
            const producto =
```

```
document.getElementById('input-producto').value.trim();
            const categoria =
document.getElementById('select-categoria').value;
            const monto =
parseFloat(document.getElementById('input-monto').value);
            const fecha = document.getElementById('input-fecha').value;
            const cliente =
document.getElementById('input-cliente').value.trim();

            if (!producto || isNaN(monto) || !fecha || !cliente) {
                mostrarToast('Por favor completa todos los campos
correctamente.', 'error');
                return;
            }

            // Creación del nuevo objeto transacción
            const nuevaVenta = {
                id: 'tx-' + Date.now(),
                producto,
                categoria,
                monto,
                fecha,
                cliente
            };

            // Agregar al inicio del array
            transacciones.unshift(nuevaVenta);

            // Persistencia
            guardarEnLocalStorage();

            // Actualizar vista
            actualizarDashboard();

            // Resetear formulario
            e.target.reset();
            // Restaurar fecha por defecto
            document.getElementById('input-fecha').valueAsDate = new
Date();

            mostrarToast('Transacción agregada exitosamente', 'exito');
        });

        // Evento Delegado: Eliminar Transacción de la Tabla

document.getElementById('tabla-ventas-body').addEventListener('click',
```

```
(e) => {
            if (e.target.classList.contains('btn-eliminar')) {
                const id = e.target.getAttribute('data-id');

                // Confirmación simple mediante array filter
                transacciones = transacciones.filter(tx => tx.id !==
id);
                guardarEnLocalStorage();
                actualizarDashboard();

                mostrarToast('Registro eliminado correctamente',
'info');
            }
        });

        // Listeners para Filtros

document.getElementById('filtro-busqueda').addEventListener('input',
actualizarDashboard);

document.getElementById('filtro-categoria').addEventListener('change',
actualizarDashboard);

document.getElementById('filtro-orden').addEventListener('change',
actualizarDashboard);

        // Botón para Restaurar Datos de Muestra
        document.getElementById('btn-seed').addEventListener('click', ()
=> {
            transacciones = [...DATOS_EJEMPLO];
            guardarEnLocalStorage();
            actualizarDashboard();
            mostrarToast('Datos de muestra restaurados', 'info');
        });

        // Mostrar / Ocultar Guía Pedagógica
        const guideEl = document.getElementById('pedagogical-guide');
        const codeToggleText =
document.getElementById('code-toggle-text');


document.getElementById('btn-toggle-code').addEventListener('click', ()
=> {
            guideEl.classList.toggle('hidden');
            const isHidden = guideEl.classList.contains('hidden');
            codeToggleText.textContent = isHidden ? 'Ver Guía
```

```
Pedagógica' : 'Ocultar Guía';
        });


document.getElementById('btn-close-guide').addEventListener('click', ()
=> {
            guideEl.classList.add('hidden');
            codeToggleText.textContent = 'Ver Guía Pedagógica';
        });

        // Notificación Toast
        function mostrarToast(mensaje, tipo = 'exito') {
            const container =
document.getElementById('toast-container');
            const toast = document.createElement('div');

            const colorBg = tipo === 'exito' ? 'bg-emerald-600' : tipo
=== 'error' ? 'bg-rose-600' : 'bg-cyan-600';

            toast.className = `${colorBg} text-white text-xs
font-semibold px-4 py-3 rounded-xl shadow-xl flex items-center gap-2
transform transition duration-300 pointer-events-auto`;
            toast.innerHTML = `<i class="fa-solid fa-circle-info"></i>
<span>${mensaje}</span>`;

            container.appendChild(toast);

            setTimeout(() => {
                toast.classList.add('opacity-0', 'translate-y-2');
                setTimeout(() => container.removeChild(toast), 300);
            }, 3000);
        }

        // ========================================
        // 6. INICIALIZACIÓN
        // ========================================
        window.onload = () => {
            // Establecer fecha de hoy en el formulario por defecto
            document.getElementById('input-fecha').valueAsDate = new
Date();
            // Cargar datos y renderizar interfaz
            actualizarDashboard();
        };
    </script>
</body>
</html>
```

Explicación del proyecto línea por línea:

Este primer bloque establece la **estructura inicial**, la **configuración del diseño (Tailwind CSS)** y las **dependencias de fuentes e iconos**.

Explicación línea por línea

1. Estructura HTML Base y Tema

```
HTML
<!DOCTYPE html>
```

- **Línea 1:** Declaración del doctype. Le indica al navegador que el documento utiliza la especificación **HTML5** estándar.

```
HTML
<html lang="es" class="dark">
```

- **Línea 2:** Etiqueta raíz del documento HTML.
  - lang="es": Especifica que el contenido principal de la página está en **español** (útil para motores de búsqueda y lectores de pantalla).
  - class="dark": Activa el **Modo Oscuro** de Tailwind CSS desde el inicio a nivel global.

```
HTML
<head>
```

- **Línea 3:** Apertura de la cabecera del documento (<head>). Contiene la metainformación, títulos, scripts de configuración y enlaces a recursos externos que no se muestran directamente en el cuerpo visible de la página.

2. Metadatos del Documento

```
HTML
   <meta charset="UTF-8">
```

- **Línea 4:** Define la codificación de caracteres como **UTF-8**, permitiendo la correcta visualización de acentos, eñes (ñ) y caracteres especiales sin errores de renderizado.

```
HTML
   <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

- **Línea 5:** Configura la vista previa en dispositivos móviles (**Responsive Design**). Indica que el ancho de la página coincida con el ancho de la pantalla (width=device-width) y que la escala inicial sea del 100% (initial-scale=1.0).

HTML
```
  <title>Dashboard Financiero y de Ventas - JavaScript Nivel 2</title>
```

- **Línea 6:** Define el **título de la pestaña** en el navegador web.

3. Carga y Configuración Dinámica de Tailwind CSS

HTML
```
  <!-- Tailwind CSS CDN -->
   <script src="https://cdn.tailwindcss.com"></script>
```

- **Líneas 7-8:** Comentario y carga de la librería **Tailwind CSS** desde su CDN oficial. Permite aplicar estilos utilizando clases de utilidad directamente en el HTML sin escribir archivos .css manuales.

HTML
```
  <script>
      tailwind.config = {
```

- **Líneas 9-10:** Apertura de un script JavaScript para extender la configuración por defecto de Tailwind mediante el objeto global tailwind.config.

HTML
```
          darkMode: 'class',
```

- **Línea 11:** Configura el modo oscuro para que responda a la presencia de la clase CSS dark en el tag <html> (definida en la línea 2).

HTML
```
          theme: {
              extend: {
```

- **Líneas 12-13:** Entra al objeto de personalización del tema para **añadir** colores o fuentes personalizadas sin sobrescribir la paleta por defecto de Tailwind.

HTML
```
                  colors: {
                      slate: {
                          850: '#111827',
                          900: '#0f172a',
                          950: '#080d1a',
                      },
```

- **Líneas 14-19:** Define tonalidades personalizadas para la gama de grises azulados (slate). Crea tonos más oscuros (850, 900, 950) para dar profundidad a las tarjetas y al fondo del dashboard en modo oscuro.

HTML

```
                brand: {
                    emerald: '#10b981',
                    cyan: '#06b6d4',
                    violet: '#8b5cf6',
                    rose: '#f43f5e',
                    amber: '#f59e0b'
                }
            },
```

- **Líneas 20-27:** Crea la paleta de colores personalizada de la marca (brand), permitiendo usar clases como bg-brand-emerald o text-brand-cyan para acentos visuales y tarjetas de métricas.

HTML

```
            fontFamily: {
                sans: ['Inter', 'sans-serif'],
            }
        }
    }
}
</script>
```

- **Líneas 28-34:** Asigna la tipografía tipográfica **Inter** como la fuente principal sin serifas (sans) para todo el proyecto y cierra el objeto de configuración de Tailwind.

4. Tipografías e Iconos Externos

HTML

```
<!-- Google Fonts -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

- **Líneas 35-37:** Establece conexiones previas (**Preconnect**) con los servidores de Google Fonts para acelerar la descarga de las fuentes tipográficas.

HTML

```
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">
```

- **Línea 38:** Importa la fuente **Inter** con grosores que van desde *Light* (300) hasta *Bold* (700).

HTML

```
<!-- FontAwesome Icons -->
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
```

- **Líneas 39-40:** Importa la librería **FontAwesome 6** para renderizar los iconos de tarjetas, botones y acciones (ej.).

Este bloque CSS define los **estilos globales**, la **personalización de la barra de desplazamiento** (scrollbar) y las clases principales para el **efecto de cristal esmerilado (Glassmorphism)** del dashboard.

Explicación línea por línea

1. Apertura y Estilos Base del Cuerpo (`<body>`)

HTML
<style>

- **Línea 1:** Abre el bloque de estilos CSS embebidos dentro de la cabecera `<head>`.

CSS
```
body {
    font-family: 'Inter', sans-serif;
    background-color: #080d1a;
    color: #f3f4f6;
}
```

- **Línea 2:** Selecciona la etiqueta `<body>` para aplicar reglas globales.
- **Línea 3:** Asigna la tipografía **Inter** (cargada previamente desde Google Fonts) con caída a `sans-serif` si la fuente no carga.
- **Línea 4:** Establece un color de fondo azul muy oscuro (`#080d1a`), equivalente al tono personalizado `slate-950` definido en la configuración de Tailwind.
- **Línea 5:** Define un color de texto gris muy claro (`#f3f4f6`) para garantizar un alto contraste sobre el fondo oscuro.
- **Línea 6:** Cierra la regla CSS para `body`.

2. Personalización de la Barra de Desplazamiento (Scrollbar Custom)

CSS
```
/* Custom Scrollbar */
::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}
```

- **Línea 7:** Comentario explicativo sobre la barra de navegación personalizada.

- **Líneas 8-11:** Pseudoelemento `::-webkit-scrollbar`. Define el grosor general de las barras de desplazamiento (tanto vertical como horizontal) a **8px** para motores basados en WebKit (Chrome, Edge, Safari, Brave).

CSS
```
::-webkit-scrollbar-track {
    background: #0f172a;
}
```

- **Líneas 12-14:** Pseudoelemento `::-webkit-scrollbar-track`. Define el fondo del riel o canal por donde se desliza la barra con un tono azul slate oscuro (`#0f172a`).

CSS
```
::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
}
```

- **Líneas 15-18:** Pseudoelemento `::-webkit-scrollbar-thumb`. Estiliza el "pulgar" o barra móvil que arrastra el usuario. Le da un color gris slate intermedio (`#334155`) y bordes redondeados de **4px**.

CSS
```
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}
```

- **Líneas 19-21:** Estado `:hover` del "pulgar". Al pasar el cursor sobre la barra, se aclara ligeramente (`#475569`) para ofrecer retroalimentación visual al usuario.

3. Estilos Glassmorphism: Tarjetas y Campos de Entrada

CSS
```
.glass-card {
    background: rgba(15, 23, 42, 0.75);
    backdrop-filter: blur(12px);
    border: 1px solid rgba(255, 255, 255, 0.08);
    box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
}
```

- **Línea 22:** Define la clase reutilizable `.glass-card` para darle el aspecto de cristal translúcido a los contenedores del dashboard.

- **Línea 23:** Fondo en formato `rgba` (Red, Green, Blue, Alpha) con un 75% de opacidad (`0.75`). Deja pasar parcialmente lo que esté detrás de la tarjeta.
- **Línea 24:** `backdrop-filter: blur(12px);`. Clave del efecto glassmorphism: **desenfoca 12px el contenido que queda debajo de la tarjeta**, simulando un vidrio esmerilado.
- **Línea 25:** Añade un borde blanco sutil ultrafino con solo un 8% de opacidad (`0.08`) para dar la sensación de reflejo en el borde del cristal.
- **Línea 26:** Aplica una sombra suave y profunda (`rgba(0, 0, 0, 0.37)`) para elevar las tarjetas tridimensionalmente sobre el fondo.
- **Línea 27:** Cierra la clase `.glass-card`.

CSS

```
.glass-input {
    background: rgba(30, 41, 59, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    color: #f8fafc;
}
```

- **Línea 28:** Define la clase `.glass-input` para estilizar los campos de texto (`<input>`) y listas desplegables (`<select>`).
- **Línea 29:** Fondo oscuro semitransparente (60% opacidad) para integrarse con la estética del dashboard.
- **Línea 30:** Borde blanco translúcido (10% opacidad).
- **Línea 31:** Color de texto blanco casi puro (`#f8fafc`) para que lo escrito sea claramente legible.
- **Línea 32:** Cierra la clase `.glass-input`.

CSS

```
.glass-input:focus {
    border-color: #06b6d4;
    outline: none;
    box-shadow: 0 0 0 2px rgba(6, 182, 212, 0.25);
}
```

- **Línea 33:** Pseudoclase `:focus` que se activa cuando el usuario hace clic dentro de un campo de entrada.
- **Línea 34:** Cambia el color del borde al cian de la marca (`#06b6d4`).
- **Línea 35:** Elimina el anillo de enfoque nativo e predeterminado del navegador (`outline: none`).
- **Línea 36:** Crea un resplandor o halo de enfoque cian sutil mediante un `box-shadow` externo de **2px**.
- **Líneas 37-38:** Cierre de la regla `:focus` y del bloque de estilos `<style>`.

HTML

</head>

- **Línea 39:** Cierra definitivamente la sección de cabecera `<head>` del documento HTML.

Este bloque HTML define el inicio del cuerpo del documento y la **Barra de Navegación Superior (Header) fija**, utilizando utilidades de Tailwind CSS para lograr una interfaz moderna y adaptativa (responsive).

Explicación línea por línea

1. Apertura del Body

HTML
<body class="min-h-screen pb-12 antialiased">

- **Línea 1:** Abre el elemento `<body>` de la página.
  - `min-h-screen`: Asegura que el cuerpo ocupe como mínimo el 100% de la altura de la pantalla (`100vh`).
  - `pb-12`: Añade un relleno inferior (*padding-bottom*) de 3rem (48px) para despejar el contenido respecto al borde final al hacer scroll.
  - `antialiased`: Suaviza el renderizado de la tipografía en navegadores basados en WebKit/Firefox.

2. Contenedor del Header y Sticky Navigation

HTML
<!-- Header Principal -->
<header class="border-b border-slate-800 bg-slate-900/80 sticky top-0 z-50 backdrop-blur-md">

- **Línea 2:** Comentario HTML.
- **Línea 3:** Elemento `<header>` semántico para la cabecera del dashboard.
  - `border-b border-slate-800`: Agrega un borde inferior sutil de color azul oscuro.
  - `bg-slate-900/80`: Fondo azul slate con 80% de opacidad para efecto translúcido.
  - `sticky top-0`: **Navegación pegajosa**; al hacer scroll, el header se fija en el borde superior de la pantalla.
  - `z-50`: Establece un orden de superposición elevado (*z-index: 50*) para flotar sobre el resto del contenido.
  - `backdrop-blur-md`: Aplica un desenfoque medio al fondo que pasa por detrás del header al desplazar la página.

3. Estructura y Grid del Header (Centrado y Responsive)

HTML

```
<div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4">
```

- **Línea 4:** Contenedor de ancho máximo limitado.
  - `max-w-7xl`: Limita el ancho del contenido a 80rem (1280px).
  - `mx-auto`: Centra horizontalmente el contenedor en la pantalla (*margin-left/right: auto*).
  - `px-4 sm:px-6 lg:px-8`: Padding horizontal adaptativo (16px en móviles, 24px en tablets, 32px en pantallas grandes).
  - `py-4`: Padding vertical interno de 1rem (16px).

HTML

```
<div class="flex flex-col md:flex-row md:items-center md:justify-between gap-4">
```

- **Línea 5:** Layout flexible para organizar el título y los botones de acción.
  - `flex flex-col`: En pantallas móviles apila los elementos en columna.
  - `md:flex-row`: En pantallas medianas en adelante (>=768px), los ubica en fila.
  - `md:items-center`: Alinea verticalmente los elementos al centro.
  - `md:justify-between`: Distribuye el título a la izquierda y los botones a la derecha.
  - `gap-4`: Separación de 1rem (16px) entre los elementos del contenedor.

4. Identidad Visual (Icono, Título y Badges)

HTML

```
<div class="flex items-center space-x-3">
```

- **Línea 6:** Agrupa el icono representativo y los textos del título principal alineándolos verticalmente (`items-center`) con espacio horizontal (`space-x-3`).

HTML

```
<div class="p-2.5 bg-gradient-to-tr from-cyan-500 to-blue-600 rounded-xl shadow-lg shadow-cyan-500/20">
    <i class="fa-solid fa-chart-line text-xl text-white"></i>
</div>
```

- **Líneas 7-9:** Contenedor para el icono con un fondo degradado (*gradient*).
  - `p-2.5`: Padding interno.
  - `bg-gradient-to-tr from-cyan-500 to-blue-600`: Degradado diagonal (de abajo-izquierda a arriba-derecha) que va desde cian a azul.
  - `rounded-xl`: Bordes redondeados pronunciados.

- `shadow-lg shadow-cyan-500/20`: Sombra proyectada con un 20% de opacidad cian.
- `<i class="...">`: Icono de FontAwesome representando una gráfica en crecimiento (`fa-chart-line`).

HTML

```
<div>
    <div class="flex items-center space-x-2">
        <h1 class="text-xl font-bold tracking-tight text-white">Sales & Financial Analytics</h1>
        <span class="px-2.5 py-0.5 text-xs font-semibold bg-cyan-500/10 text-cyan-400 border border-cyan-500/20 rounded-full">JS Level 2</span>
    </div>
    <p class="text-xs text-slate-400">Demostración práctica: Objetos, Arrays, Métodos Funcionales y DOM</p>
</div>
</div>
```

- **Líneas 10-17:** Bloque textual de la marca.
  - `<h1 class="...">`: Título del dashboard con texto blanco y fuente en negrita (`font-bold`).
  - `<span class="...">`: Badge distintivo tipo píldora (`rounded-full`) que etiqueta el nivel técnico del proyecto (*JS Level 2*).
  - `<p class="...">`: Subtítulo descriptivo en tono gris (`text-slate-400`) para contextualizar el propósito pedagógico del módulo.

5. Botones de Acción Global

HTML

```
<!-- Botones de Acción Global -->
<div class="flex items-center space-x-3">
```

- **Líneas 18-19:** Contenedor para los botones interactivos principales del encabezado.

HTML

```
<button id="btn-seed" class="px-3.5 py-2 text-xs font-medium text-slate-300 bg-slate-800 hover:bg-slate-700 hover:text-white rounded-lg border border-slate-700 transition flex items-center gap-2">
    <i class="fa-solid fa-rotate-left text-cyan-400"></i> Restaurar Datos
</button>
```

- **Líneas 20-22:** Botón de reinicio de datos de prueba.
  - `id="btn-seed"`: Identificador único que se capturará en JavaScript con `document.getElementById('btn-seed')` para agregar el evento `click` que restablece el `localStorage`.

    - `hover:bg-slate-700 hover:text-white`: Cambia de color al pasar el cursor para dar retroalimentación visual (*hover state*).
    - `transition`: Aplica animaciones suaves en los cambios de color.
    - `<i class="fa-rotate-left">`: Icono de reinicio/deshacer.

HTML

```
                <button id="btn-toggle-code" class="px-3.5 py-2 text-xs font-medium
text-cyan-300 bg-cyan-950/60 hover:bg-cyan-900/80 rounded-lg border border-cyan-800/60
transition flex items-center gap-2">
                    <i class="fa-solid fa-code text-cyan-400"></i> <span
id="code-toggle-text">Ver Guía Pedagógica</span>
                </button>
            </div>
        </div>
    </div>
</header>
```

- **Líneas 23-28:** Botón para conmutar/mostrar la guía explicativa del código.
    - `id="btn-toggle-code"`: Identificador capturado por JavaScript para alternar la visibilidad de la sección pedagógica.
    - `id="code-toggle-text"`: Span dinámico cuyo texto cambiará entre *"Ver Guía Pedagógica"* y *"Ocultar Guía Pedagógica"* mediante modificación del DOM.
- **Líneas 29-31:** Cierre jerárquico de los contenedores div y de la etiqueta `<header>`.

Este bloque contiene la apertura de la etiqueta principal `<main>` y la sección del **Banner Educativo Desplegable (Guía Pedagógica)**, que funciona como un panel interactivo para resumir los pilares de JavaScript Nivel 2 trabajados en el proyecto.

Explicación línea por línea

1. Apertura del Contenedor Principal (`<main>`)

HTML

```
<main class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 pt-8 space-y-8">
```

- **Línea 1:** Abre el contenedor semántico principal del dashboard.
    - `max-w-7xl`: Limita el ancho máximo a 1280px para mantener la coherencia visual con el header.
    - `mx-auto`: Centra la sección en la pantalla horizontalmente.
    - `px-4 sm:px-6 lg:px-8`: Márgenes internos laterales (*padding*) adaptables a pantallas móviles, tablets y monitores.
    - `pt-8`: Añade un relleno superior de 2rem (32px) para separar el contenido del header pegajoso.

- `space-y-8`: Aplica un espaciado vertical automático de 2rem (32px) entre cada una de sus secciones hijas inmediatas.

2. Encabezado de la Guía Pedagógica

HTML

```
<!-- Banner Educativo Desplegable -->
<section id="pedagogical-guide" class="hidden glass-card rounded-2xl p-6 border-l-4 border-l-cyan-500 transition-all duration-300">
```

- **Línea 2:** Comentario HTML.
- **Línea 3:** Crea la sección con el identificador `id="pedagogical-guide"`.
  - `hidden`: **Oculta la sección por defecto** en la carga inicial de la página. El botón *"Ver Guía Pedagógica"* del header eliminará esta clase mediante JavaScript para mostrarla.
  - `glass-card`: Aplica la clase de CSS personalizada definida con el efecto de cristal traslúcido (*glassmorphism*).
  - `rounded-2xl`: Bordes redondeados pronunciados de 1rem (16px).
  - `p-6`: Padding interno de 1.5rem (24px).
  - `border-l-4 border-l-cyan-500`: Agrega un borde izquierdo de 4px de grosor de color cian para dar un acento visual distintivo.
  - `transition-all duration-300`: Garantiza una transición fluida al mostrar u ocultar la tarjeta.

HTML

```
<div class="flex justify-between items-start mb-4">
    <div class="flex items-center gap-2 text-cyan-400 font-semibold">
        <i class="fa-solid fa-graduation-cap text-lg"></i>
        <h2>Conceptos Clave de JavaScript Nivel 2 Aplicados</h2>
    </div>
    <button id="btn-close-guide" class="text-slate-400 hover:text-white text-sm"><i class="fa-solid fa-xmark"></i></button>
</div>
```

- **Líneas 4-10:** Fila superior del banner educativo.
  - `flex justify-between items-start`: Utiliza Flexbox para colocar el título a la izquierda y el botón de cierre a la derecha.
  - `<i class="fa-graduation-cap">`: Icono de gorro académico de FontAwesome.
  - `<h2>`: Título secundario de la sección en color cian.
  - `<button id="btn-close-guide">`: Botón con el icono de equis (`fa-xmark`) que permitirá cerrar la guía directamente haciendo clic sobre él.

3. Tarjetas de Conceptos Teóricos (Grid de 3 Columnas)

HTML

- **Línea 11:** Rejilla CSS adaptativa de 3 columnas (`grid-cols-1 md:grid-cols-3`) con una separación interna de 1rem (`gap-4`) entre las 3 tarjetas de conceptos.

Concepto 1: Métodos Avanzados de Arrays

HTML

1. Métodos de Arrays Avanzados
Uso de <code class="text-cyan-300">reduce()</code> para calcular acumulados de ingresos y mapas de frecuencia, <code class="text-cyan-300">filter()</code> para búsquedas de texto y categorías, y <code class="text-cyan-300">map()</code> para renderizado fluido.

- **Líneas 12-15:** Primera tarjeta informativa del banner.
  - Explicación resumida del papel de `reduce()`, `filter()` y `map()`.
  - La etiqueta `<code>` resalta la sintaxis de las funciones en color cian claro sobre fondo oscuro.

Concepto 2: Manipulación de Objetos

HTML

2. Manipulación de Objetos
Uso de <code class="text-violet-300">Object.entries()</code> y <code class="text-violet-300">Object.keys()</code> para transformar acumuladores por categoría en estructuras navegables en tarjetas de rendimiento.

- **Líneas 16-19:** Segunda tarjeta del banner.
  - Resume el uso de los métodos de la clase global `Object` para convertir objetos clave-valor en arrays iterables.

Concepto 3: DOM Dinámico y Persistencia

HTML

3. DOM Dinámico & LocalStorage
Creación iterativa de elementos con <code class="text-cyan-300">document.createElement()</code>, delegación de eventos y sincronización en tiempo real con <code class="text-cyan-300">localStorage</code>.

```
    </div>
  </div>
</section>
```

- **Líneas 20-25:** Tercera tarjeta del banner y cierre del contenedor del banner educativo.
    - Detalla cómo se conecta la lógica del script con el árbol de elementos visuales (DOM) y la memoria del navegador.

Esta sección define el bloque de **Tarjetas de Métricas Clave (KPIs)** del dashboard. Utiliza un sistema de rejilla (*grid*) de Tailwind CSS y efectos de resplandor sutiles (*glow*) para presentar cuatro indicadores financieros principales que JavaScript calculará de forma dinámica.

Explicación línea por línea

1. Contenedor Grid Principal de KPIs

HTML

```
<!-- Secciones de Tarjetas KPI Dinámicas -->
<section class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-5" id="kpi-cards-container">
```

- **Línea 1:** Comentario en HTML indicando el inicio del bloque de métricas.
- **Línea 2:** Apertura de la sección contenedora con el identificador id="kpi-cards-container".
    - grid: Establece un diseño basado en rejilla CSS (Grid Layout).
    - grid-cols-1: Muestra **1 columna** en pantallas móviles muy pequeñas.
    - sm:grid-cols-2: Adapta la rejilla a **2 columnas** en pantallas pequeñas/medianas ($\geq 640px$).
    - lg:grid-cols-4: Muestra las **4 tarjetas alineadas horizontalmente** en pantallas grandes ($\geq 1024px$).
    - gap-5: Define un espacio uniforme de $1.25rem$ ($20px$) entre cada tarjeta.

2. Tarjeta 1: Total Ingresos

HTML

```
  <!-- Total Ingresos Card -->
   <div class="glass-card rounded-2xl p-5 relative overflow-hidden group
hover:border-emerald-500/40 transition duration-300">
```

- **Líneas 3-4:** Estructura exterior de la primera tarjeta de métricas (Ingresos).
    - glass-card: Aplica la clase con efecto de cristal traslúcido y desenfoque (*glassmorphism*).
    - rounded-2xl: Bordes redondeados de $1rem$ ($16px$).
    - p-5: Relleno interno (*padding*) de $1.25rem$ ($20px$).
    - relative overflow-hidden: Posicionamiento relativo para contener dentro de sus límites el círculo con luz que sobresale en la esquina.

- group: Permite animar elementos hijos cuando se pasa el cursor sobre la tarjeta (*hover*).
- hover:border-emerald-500/40: Al pasar el ratón, el borde resalta en verde esmeralda con 40% de opacidad.

HTML
```
<div class="absolute -right-4 -bottom-4 w-24 h-24 bg-emerald-500/10 rounded-full blur-xl group-hover:bg-emerald-500/20 transition"></div>
```

- **Línea 5:** Círculo decorativo con desenfoque (blur-xl) colocado fuera de la esquina inferior derecha (-right-4 -bottom-4). Al hacer hover sobre la tarjeta (group-hover), la luz ambiental aumenta su intensidad del 10% al 20%.

HTML
```
<div class="flex items-center justify-between">
    <span class="text-xs font-medium text-slate-400 uppercase tracking-wider">Total Ingresos</span>
    <div class="p-2 rounded-lg bg-emerald-500/10 text-emerald-400 border border-emerald-500/20">
        <i class="fa-solid fa-dollar-sign"></i>
    </div>
</div>
```

- **Líneas 6-11:** Encabezado interno de la tarjeta. Ubica el título con texto en mayúsculas (uppercase tracking-wider) a la izquierda y la caja del icono del signo de dólar (fa-dollar-sign) en verde esmeralda a la derecha.

HTML
```
<div class="mt-4">
    <h3 id="kpi-total-ingresos" class="text-2xl font-extrabold text-white">$0.00</h3>
    <p class="text-xs text-slate-400 mt-1 flex items-center gap-1">
        <span id="kpi-count-transacciones" class="text-emerald-400 font-semibold">0</span> transacciones registradas
    </p>
</div>
</div>
```

- **Líneas 12-18:** Bloque de valores calculados dinámicamente:
  - id="kpi-total-ingresos": Elemento donde JavaScript inyectará la suma total calculada mediante Array.prototype.reduce().
  - id="kpi-count-transacciones": Span que actualizará el número entero de transacciones contadas.

3. Tarjeta 2: Promedio de Venta

HTML
```
<!-- Promedio de Venta Card -->
<div class="glass-card rounded-2xl p-5 relative overflow-hidden group hover:border-cyan-500/40 transition duration-300">
```

```
        <div class="absolute -right-4 -bottom-4 w-24 h-24 bg-cyan-500/10 rounded-full blur-xl
group-hover:bg-cyan-500/20 transition"></div>
        <div class="flex items-center justify-between">
            <span class="text-xs font-medium text-slate-400 uppercase
tracking-wider">Promedio de Venta</span>
            <div class="p-2 rounded-lg bg-cyan-500/10 text-cyan-400 border
border-cyan-500/20">
                <i class="fa-solid fa-calculator"></i>
            </div>
        </div>
        <div class="mt-4">
            <h3 id="kpi-promedio-venta" class="text-2xl font-extrabold text-white">$0.00</h3>
            <p class="text-xs text-slate-400 mt-1">Por ticket de venta</p>
        </div>
    </div>
```

- **Líneas 19-32:** Segunda tarjeta configurada con acentos de color cian.
    - Icono de calculadora (fa-calculator).
    - id="kpi-promedio-venta": Nodo capturado por JavaScript para renderizar la media aritmética del monto por transacción (Total dividido entre el número de elementos).

4. Tarjeta 3: Producto Más Vendido (Frecuencia)

HTML

```
    <!-- Producto Más Vendido Card -->
    <div class="glass-card rounded-2xl p-5 relative overflow-hidden group
hover:border-violet-500/40 transition duration-300">
        <div class="absolute -right-4 -bottom-4 w-24 h-24 bg-violet-500/10 rounded-full blur-xl
group-hover:bg-violet-500/20 transition"></div>
        <div class="flex items-center justify-between">
            <span class="text-xs font-medium text-slate-400 uppercase
tracking-wider">Producto Top (Frecuencia)</span>
            <div class="p-2 rounded-lg bg-violet-500/10 text-violet-400 border
border-violet-500/20">
                <i class="fa-solid fa-award"></i>
            </div>
        </div>
        <div class="mt-4">
            <h3 id="kpi-producto-top" class="text-xl font-bold text-white truncate">N/A</h3>
            <p class="text-xs text-slate-400 mt-1">
                <span id="kpi-producto-top-count" class="text-violet-400 font-semibold">0</span>
ventas realizadas
            </p>
        </div>
    </div>
```

- **Líneas 33-47:** Tercera tarjeta con tono violeta y un icono de medalla/premio (fa-award).

- truncate: Corta el nombre del producto con puntos suspensivos (...) si el texto sobrepasa el ancho de la tarjeta.
- id="kpi-producto-top": Muestra el producto con mayor número de apariciones.
- id="kpi-producto-top-count": Indica el conteo exacto de ventas de dicho producto. En JavaScript, esto ejercita la técnica avanzada de construir un mapa de frecuencias usando reduce() con un objeto acumulador.

5. Tarjeta 4: Categoría Líder por Ingresos

HTML

```
  <!-- Categoría Líder Card -->
   <div class="glass-card rounded-2xl p-5 relative overflow-hidden group
hover:border-amber-500/40 transition duration-300">
      <div class="absolute -right-4 -bottom-4 w-24 h-24 bg-amber-500/10 rounded-full blur-xl
group-hover:bg-amber-500/20 transition"></div>
      <div class="flex items-center justify-between">
         <span class="text-xs font-medium text-slate-400 uppercase
tracking-wider">Categoría Top ($)</span>
         <div class="p-2 rounded-lg bg-amber-500/10 text-amber-400 border
border-amber-500/20">
            <i class="fa-solid fa-layer-group"></i>
         </div>
      </div>
      <div class="mt-4">
         <h3 id="kpi-categoria-top" class="text-xl font-bold text-white truncate">N/A</h3>
         <p class="text-xs text-slate-400 mt-1">
            Ingreso: <span id="kpi-categoria-top-monto" class="text-amber-400
font-semibold">$0.00</span>
         </p>
      </div>
   </div>
</section>
```

- **Líneas 48-63:** Cuarta tarjeta estilizada con tono ámbar/dorado y un icono de capas (fa-layer-group).
  - id="kpi-categoria-top": Nombre de la categoría que generó el mayor volumen monetario.
  - id="kpi-categoria-top-monto": Monto exacto acumulado por esa categoría. Se resuelve mediante la combinación de reduce() con Object.entries().
- **Línea 64:** Cierre del contenedor principal de las 4 tarjetas KPI.

Este bloque contiene la **Columna Izquierda** del layout principal, correspondiente al **Formulario para Registrar Nueva Venta**. Ocupa 4 de las 12 columnas disponibles en pantallas grandes y se fija dinámicamente mediante sticky para permanecer visible al hacer scroll.

Explicación línea por línea

1. Estructura de la Fila Principal y Columna Izquierda

HTML
<!-- Fila Principal: Formulario + Filtros & Controles -->

- **Línea 1:** Comentario HTML.
- **Línea 2:** Abre la rejilla principal del contenido.
    - grid-cols-1: En pantallas móviles, las secciones se muestran apiladas a una sola columna.
    - lg:grid-cols-12: En pantallas grandes ($\geq 1024px$), divide la fila en **12 columnas**.
    - gap-8: Separación de $2rem$ $(32px)$ entre columnas.

HTML
<!-- Columna Izquierda: Formulario para Agregar Venta (4 cols) -->
<section class="lg:col-span-4">

- **Líneas 3-4:** Contenedor de la columna izquierda.
    - lg:col-span-4: Ocupa **4 de las 12 columnas** del layout en pantallas grandes (33.3% del ancho).
    - sticky top-24: **Posicionamiento pegajoso**. Al deslizar la página, la tarjeta se mantendrá fija a $6rem$ $(96px)$ del borde superior para no perderse de vista.

2. Encabezado del Formulario

HTML

<h2 class="text-base font-bold text-white">Registrar Nueva Venta</h2>

- **Líneas 5-8:** Título del panel de registro. Utiliza Flexbox con un icono de agregar (fa-plus-circle) en tono cian y un subtítulo <h2> en negrita.

3. Campo 1: Nombre del Producto

HTML
<form id="form-nueva-venta" class="space-y-4">

<label for="input-producto" class="block text-xs font-semibold text-slate-300 mb-1">Nombre del Producto</label>
<input type="text" id="input-producto" required placeholder="Ej: Monitor Gamer 27""
class="w-full rounded-xl px-3.5 py-2.5 text-sm glass-input">

- **Líneas 9-15:** Inicio del elemento <form> y primer campo.
    - id="form-nueva-venta": Capturado en JavaScript para escuchar el evento submit y prevenir el recargo nativo con e.preventDefault().

- space-y-4: Aplica un espacio vertical de $1rem$ $(16px)$ entre cada grupo del formulario.
- id="input-producto": Entrada de texto requerida (required) para el nombre del artículo. Usa la clase personalizada glass-input.

4. Campos 2 y 3: Categoría y Monto (Fila Doble)

HTML

```
<div class="grid grid-cols-2 gap-3">
  <div>
    <label for="select-categoria" class="block text-xs font-semibold text-slate-300 mb-1">Categoría</label>
    <select id="select-categoria" required class="w-full rounded-xl px-3 py-2.5 text-sm glass-input bg-slate-900">
      <option value="Electrónica">Electrónica</option>
      <option value="Ropa">Ropa</option>
      <option value="Hogar">Hogar</option>
      <option value="Alimentos">Alimentos</option>
      <option value="Servicios">Servicios</option>
    </select>
  </div>
```

- **Líneas 16-27:** Rejilla interna de 2 columnas (grid-cols-2) con separación de $12px$ (gap-3).
  - id="select-categoria": Lista desplegable (<select>) para clasificar la venta en una de las 5 categorías oficiales. Sus valores serán filtrados y agrupados posteriormente en JavaScript.

HTML

```
  <div>
    <label for="input-monto" class="block text-xs font-semibold text-slate-300 mb-1">Monto ($)</label>
    <input type="number" id="input-monto" step="0.01" min="0.01" required placeholder="0.00"
           class="w-full rounded-xl px-3 py-2.5 text-sm glass-input">
  </div>
</div>
```

- **Líneas 28-33:** Entrada numérica para el precio.
  - type="number": Restringe la entrada a números.
  - step="0.01": Permite ingresar centavos (2 decimales).
  - min="0.01": Valida a nivel de navegador que el monto sea un valor positivo.

5. Campos 4 y 5: Fecha y Cliente (Fila Doble)

HTML

```
<div class="grid grid-cols-2 gap-3">
  <div>
    <label for="input-fecha" class="block text-xs font-semibold text-slate-300 mb-1">Fecha</label>
```

```
                    <input type="date" id="input-fecha" required class="w-full rounded-xl px-3
py-2.5 text-sm glass-input">
                </div>
```

- **Líneas 34-38:** Selector de fecha nativo (type="date"). Su valor en formato YYYY-MM-DD permitirá ordenar las ventas cronológicamente en JavaScript.

HTML

```
                <div>
                    <label for="input-cliente" class="block text-xs font-semibold text-slate-300
mb-1">Cliente</label>
                    <input type="text" id="input-cliente" required placeholder="Nombre cliente"
                        class="w-full rounded-xl px-3 py-2.5 text-sm glass-input">
                </div>
            </div>
```

- **Líneas 39-44:** Campo textual para registrar el nombre de la persona o empresa compradora (id="input-cliente").

6. Botón de Envio y Cierre

HTML

```
                <button type="submit" class="w-full py-3 px-4 bg-gradient-to-r from-cyan-500
to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-white font-semibold text-sm
rounded-xl shadow-lg shadow-cyan-500/25 transition duration-200 flex items-center
justify-center gap-2 mt-2">
                    <i class="fa-solid fa-check"></i> Guardar Transacción
                </button>
            </form>
        </div>
    </section>
```

- **Líneas 45-48:** Botón principal de guardado.
  - type="submit": Dispara la validación del formulario y el evento submit.
  - bg-gradient-to-r from-cyan-500 to-blue-600: Degradado horizontal de cian a azul.
  - shadow-cyan-500/25: Proyecta una sombra de resplandor cian al 25% de opacidad.
- **Líneas 49-50:** Cierre del formulario, de la tarjeta contenedora y de la sección izquierda.

Este bloque abre la **Columna Derecha** del layout (8 de las 12 columnas en pantallas grandes) y contiene el **Panel de Filtros y Búsqueda en Tiempo Real**.

Explicación línea por línea

1. Apertura de la Columna Derecha

HTML

```
<!-- Columna Derecha: Filtros, Tabla y Análisis de Categorías (8 cols) -->
```

<section class="lg:col-span-8 space-y-6">

- **Línea 1:** Comentario en HTML que delimita el inicio de la columna de visualización y control.
- **Línea 2:** Apertura de la sección contenedora.
    - lg:col-span-8: Ocupa **8 de las 12 columnas** del grid en pantallas grandes ($\geq 1024px$), correspondiendo al 66.6% del ancho útil.
    - space-y-6: Aplica una separación vertical constante de $1.5rem$ $(24px)$ entre las tarjetas hijas (Panel de Filtros, Tabla de Ventas y Resumen de Categorías).

2. Encabezado del Panel de Filtros

HTML

```
  <!-- Barra de Filtros y Búsqueda -->
   <div class="glass-card rounded-2xl p-5 space-y-4">
      <div class="flex flex-col sm:flex-row sm:items-center justify-between gap-3 border-b
border-slate-800 pb-4">
         <h2 class="text-base font-bold text-white flex items-center gap-2">
            <i class="fa-solid fa-filter text-cyan-400"></i> Filtros de Datos
         </h2>
         <span id="resultados-count" class="text-xs text-slate-400 bg-slate-800 px-3 py-1
rounded-full border border-slate-700">
            Mostrando 0 registros
         </span>
      </div>
```

- **Líneas 3-4:** Inicio de la tarjeta de controles con el estilo traslúcido .glass-card.
- **Líneas 5-11:** Cabecera interna con borde inferior separador (border-b border-slate-800).
    - flex flex-col sm:flex-row: Apila los elementos verticalmente en móviles y los pone en fila a partir de pantallas pequeñas ($\geq 640px$).
    - <h2 class="...">: Título del panel acompañado del icono de embudo/filtro (fa-filter) en color cian.
    - id="resultados-count": Contador dinámico tipo píldora (rounded-full) que JavaScript actualizará en tiempo real indicando cuántas ventas coinciden con los criterios aplicados (ej. *"Mostrando 5 registros"*).

3. Control 1: Campo de Búsqueda de Texto con Icono Superpuesto

HTML

```
      <div class="grid grid-cols-1 sm:grid-cols-3 gap-3">
         <!-- Búsqueda por Nombre -->
         <div class="relative">
            <i class="fa-solid fa-magnifying-glass absolute left-3.5 top-3 text-slate-500
text-xs"></i>
            <input type="text" id="filtro-busqueda" placeholder="Buscar producto o cliente..."
                  class="w-full pl-9 pr-3 py-2 text-xs rounded-xl glass-input">
         </div>
```

- **Línea 12:** Rejilla interna de 3 columnas (sm:grid-cols-3) para distribuir equitativamente los tres controles de filtrado.
- **Líneas 13-18:** Campo de búsqueda por texto.
    - relative: Define el contenedor como relativo para poder posicionar el icono de la lupa con absolutos.
    - <i class="fa-magnifying-glass absolute left-3.5 top-3">: Coloca la lupa dentro del borde izquierdo del campo de texto.
    - id="filtro-busqueda": Input capturado en JavaScript con el evento input para filtrar arreglos dinámicamente mediante Array.prototype.filter() y coincidencia de cadenas con toLowerCase().includes().
    - pl-9: *Padding-left* compensado de $2.25rem$ $(36px)$ para evitar que el texto escrito se encime sobre el icono de la lupa.

4. Control 2: Selector de Categoría

HTML

```
        <!-- Filtro Categoría -->
        <div>
            <select id="filtro-categoria" class="w-full py-2 px-3 text-xs rounded-xl glass-input
bg-slate-900">
                <option value="TODAS">Todas las Categorías</option>
                <option value="Electrónica">Electrónica</option>
                <option value="Ropa">Ropa</option>
                <option value="Hogar">Hogar</option>
                <option value="Alimentos">Alimentos</option>
                <option value="Servicios">Servicios</option>
            </select>
        </div>
```

- **Líneas 19-30:** Lista desplegable para segmentar las ventas.
    - id="filtro-categoria": Escuchado mediante el evento change en JavaScript.
    - value="TODAS": Opción por defecto para desactivar el filtro de categoría y mostrar la totalidad del catálogo.

5. Control 3: Criterio de Ordenamiento

HTML

```
        <!-- Ordenar Por -->
        <div>
            <select id="filtro-orden" class="w-full py-2 px-3 text-xs rounded-xl glass-input
bg-slate-900">
                <option value="reciente">Fecha: Más Reciente</option>
                <option value="antiguo">Fecha: Más Antiguo</option>
                <option value="monto-desc">Monto: Mayor a Menor</option>
                <option value="monto-asc">Monto: Menor a Mayor</option>
            </select>
        </div>
    </div>
</div>
```

- **Líneas 31-41:** Selector del criterio de ordenación.
    - id="filtro-orden": Determina cómo se ordenará el arreglo en JavaScript usando Array.prototype.sort().
    - Permite alternar entre ordenamiento cronológico (Date.parse()) y ordenamiento por magnitudes financieras (comparación directa de montos numéricos).
- **Líneas 42-43:** Cierre del grid de controles y de la tarjeta del panel de filtros.

Este bloque completa la **estructura HTML del Dashboard**, conteniendo la **Tabla Dinámica de Transacciones**, el **Estado Vacío** (cuando no hay resultados), el contenedor para el **Desglose de Categorías** y la etiqueta de cierre del `<main>` junto al contenedor de **Notificaciones Toast**.

Explicación línea por línea

1. Contenedor de la Tabla Dinámica

HTML

```
<!-- Tabla Dinámica de Transacciones -->
<div class="glass-card rounded-2xl overflow-hidden border border-slate-800">
    <div class="px-6 py-4 border-b border-slate-800 flex justify-between items-center bg-slate-900/40">
        <h3 class="text-sm font-bold text-white flex items-center gap-2">
            <i class="fa-solid fa-table-list text-cyan-400"></i> Registro de Ventas
        </h3>
    </div>
```

- **Líneas 1-2:** Comentario HTML y contenedor de la tabla con la clase `.glass-card`.
    - `overflow-hidden`: Asegura que las esquinas redondeadas de la tarjeta (`rounded-2xl`) recorten los bordes de la tabla.
- **Líneas 3-7:** Encabezado superior de la tabla con un icono de lista (`fa-table-list`) en tono cian y el título *"Registro de Ventas"*.

2. Estructura de la Tabla HTML (`<table>`)

HTML

```
    <div class="overflow-x-auto">
        <table class="w-full text-left text-xs">
            <thead class="bg-slate-900/80 uppercase tracking-wider text-slate-400 border-b border-slate-800">
                <tr>
                    <th class="py-3 px-4">Fecha</th>
                    <th class="py-3 px-4">Producto</th>
                    <th class="py-3 px-4">Categoría</th>
                    <th class="py-3 px-4">Cliente</th>
                    <th class="py-3 px-4 text-right">Monto</th>
```

```
            <th class="py-3 px-4 text-center">Acción</th>
          </tr>
        </thead>
```

- **Línea 8:** `overflow-x-auto`: Habilita la barra de desplazamiento horizontal si la pantalla móvil es demasiado estrecha para mostrar todas las columnas.
- **Líneas 9-20:** Definición del encabezado (`<thead>`).
    - `uppercase tracking-wider`: Texto en mayúsculas con espaciado entre letras para mejorar la legibilidad de las columnas.
    - `text-right` en *Monto*: Alinea las cifras numéricas a la derecha por buena práctica financiera.
    - `text-center` en *Acción*: Centra el botón de eliminación de registros.

HTML

```
        <tbody id="tabla-ventas-body" class="divide-y divide-slate-800/60 font-medium">
          <!-- Filas generadas dinámicamente con JavaScript -->
        </tbody>
      </table>
    </div>
```

- **Líneas 21-25:** Cuerpo de la tabla (`<tbody>`).
    - `id="tabla-ventas-body"`: Elemento clave capturado en JavaScript. El script iterará el array de ventas con `.map()` o `.forEach()` e inyectará los nodos `<tr>` con `document.createElement()` y `appendChild()`.
    - `divide-y divide-slate-800/60`: Genera automáticamente líneas separadoras sutiles entre cada fila de datos.

3. Estado Vacío (*Empty State*)

HTML

```
  <!-- Estado Vacío -->
  <div id="empty-state" class="hidden text-center py-12 px-4">
    <div class="inline-flex p-4 rounded-full bg-slate-800/80 text-slate-500 mb-3">
      <i class="fa-solid fa-inbox text-2xl"></i>
    </div>
    <p class="text-sm font-semibold text-slate-300">No se encontraron transacciones</p>
    <p class="text-xs text-slate-500 mt-1">Prueba cambiando los filtros o agregando un
nuevo registro.</p>
  </div>
</div>
```

- **Líneas 26-34:** Mensaje visual que se muestra cuando el array filtrado no contiene elementos (longitud 0).

    - `id="empty-state"`: Inicialmente oculto (`hidden`). Si la búsqueda o filtro no produce coincidencias, JavaScript le removerá la clase `hidden` para guiar al usuario con la ilustración de una bandeja vacía (`fa-inbox`).

4. Sección de Desglose por Categorías

HTML

```
<!-- Desglose Por Categorías (Demostración de Object.entries y reduce) -->
<div class="glass-card rounded-2xl p-6">
  <div class="flex items-center justify-between mb-4">
    <h3 class="text-sm font-bold text-white flex items-center gap-2">
      <i class="fa-solid fa-chart-pie text-violet-400"></i> Distribución de Ventas por Categoría
    </h3>
    <span class="text-xs text-slate-400">Generado dinámicamente con Object.entries()</span>
  </div>
  <div id="categoria-breakdown-container" class="grid grid-cols-1 sm:grid-cols-2 gap-4">
    <!-- Tarjetas de Categorías generadas vía JS -->
  </div>
</div>
```

- **Líneas 35-46:** Panel dinámico para visualizar cuánto dinero aporta cada categoría.
    - `<h3 class="...">`: Título del gráfico/distribución acompañado del icono de pastel (`fa-chart-pie`).
    - `id="categoria-breakdown-container"`: Rejilla contenedora (`grid-cols-1 sm:grid-cols-2`). En JavaScript se agruparán las ventas con `reduce()`, se transformará ese objeto con `Object.entries()` y se insertará una mini-tarjeta con barra de progreso por cada categoría existente.

5. Cierre del Layout Principal y Notificaciones Toast

HTML

```
  </section>
</div>
</main>

<!-- Modal de Notificaciones Toast -->
<div id="toast-container" class="fixed bottom-5 right-5 z-50 flex flex-col gap-2 pointer-events-none"></div>
```

- **Líneas 47-49:** Cierre de la columna derecha (`</section>`), de la rejilla de 12 columnas y del elemento `<main>`.
- **Líneas 50-51:** Contenedor flotante para los avisos emergentes (*Toast Notifications*).
    - `id="toast-container"`: Anclado en la esquina inferior derecha (`fixed bottom-5 right-5 z-50`).

    - `pointer-events-none`: Evita que el contenedor transparente bloquee los clics en la interfaz cuando no hay notificaciones activas.

¡Hemos completado la revisión del 100% de la estructura de marcado HTML y los estilos visuales del proyecto!

Iniciamos el análisis del script principal en JavaScript. Este primer bloque constituye la **columna vertebral de los datos**: define el comentario de arquitectura, el dataset inicial estructurado y la carga con persistencia desde localStorage.

Explicación línea por línea

1. Apertura del Script y Comentario de Arquitectura

HTML

```
<!-- CÓDIGO JAVASCRIPT PRINCIPAL -->
  <script>
```

- **Líneas 1-2:** Comentario HTML y etiqueta de apertura <script> que le indica al navegador que el bloque interno contiene ejecutable en JavaScript Vanilla (ES6+).

JavaScript

```
    /**
     *
================================================================
     * JAVASCRIPT NIVEL 2 - DASHBOARD DE VENTAS & FINANZAS
     *
================================================================
     * Este script implementa todos los conceptos del syllabus:
     * 1. Objetos Literales, Constructores y Manipulación.
     * 2. Métodos Avanzados de Arrays: map(), filter(), reduce(), sort().
     * 3. Uso de Funciones Flecha y Callbacks.
     * 4. Manipulación Avanzada del DOM (createElement, appendChild, removeChild).
     * 5. Almacenamiento Persistente mediante LocalStorage.
     */
```

- **Líneas 3-13:** Bloque de documentación JSDoc/multilinea. Establece la guía didáctica del script e identifica los 5 pilares técnicos del programa de JavaScript Nivel 2 aplicados.

2. Sección de Estado de la Aplicación

JavaScript

```
    // ========================================
    // 1. ESTADO DE LA APLICACIÓN & DATOS DE MUESTRA
    // ========================================
```

- **Líneas 14-16:** Separador conceptual para organizar la estructura del código por módulos.

3. Objeto de Datos de Muestra (DATOS_EJEMPLO)

JavaScript

```
/**
 * Datos de muestra iniciales (Estructura: Array de Objetos)
 */
const DATOS_EJEMPLO = [
```

- **Líneas 17-20:** Declaración de la constante global DATOS_EJEMPLO.
  - Utiliza const porque la referencia al arreglo de datos iniciales no cambiará.
  - Define la estructura de datos principal: un **Array de Objetos Literales** ($Array < Object >$).

JavaScript

```
{ id: "tx-101", producto: "Laptop Pro 15"", categoria: "Electrónica", monto: 1250.00, fecha: "2026-03-10", cliente: "Carlos Mendoza" },
{ id: "tx-102", producto: "Camisa Formal Slim", categoria: "Ropa", monto: 45.50, fecha: "2026-03-11", cliente: "Ana Gómez" },
{ id: "tx-103", producto: "Cafetera Expresso", categoria: "Hogar", monto: 180.00, fecha: "2026-03-12", cliente: "Roberto Silva" },
{ id: "tx-104", producto: "Monitor Gamer 27"", categoria: "Electrónica", monto: 320.00, fecha: "2026-03-14", cliente: "Lucía Fernández" },
{ id: "tx-105", producto: "Consultoría IT (1h)", categoria: "Servicios", monto: 90.00, fecha: "2026-03-15", cliente: "Tech Solutions" },
{ id: "tx-106", producto: "Set de Cuchillos Cocina", categoria: "Hogar", monto: 65.00, fecha: "2026-03-16", cliente: "Mariana López" },
{ id: "tx-107", producto: "Laptop Pro 15"", categoria: "Electrónica", monto: 1250.00, fecha: "2026-03-18", cliente: "Empresa Global" },
{ id: "tx-108", producto: "Paquete Snacks Gourmet", categoria: "Alimentos", monto: 28.00, fecha: "2026-03-19", cliente: "David Ruiz" }
];
```

- **Líneas 21-28:** Colección de 8 registros financieros base. Cada objeto literal contiene 6 propiedades clave:
  - id ($String$): Identificador único prefijado para manipulación y eliminación en el DOM.
  - producto ($String$): Nombre del ítem. Permite evaluar duplicados/frecuencias en reduce().
  - categoria ($String$): Categoría para filtrado y segmentación en agrupadores.
  - monto ($Number$): Valor flotante para agregaciones matemáticas.
  - fecha ($String$): Cadena con formato ISO (YYYY-MM-DD) utilizada para ordenamiento temporal.
  - cliente ($String$): Nombre del comprador para búsquedas por texto completo.

4. Inicialización del Estado Global (transacciones)

JavaScript

```
// Estado Global con persitencia en localStorage
let transacciones = cargarTransacciones();
```

- **Líneas 29-30:** Declaración de la variable de estado principal transacciones.
    - Se declara con let porque su valor cambiará dinámicamente cuando el usuario agregue, elimine o restaure transacciones.
    - Asigna inmediatamente el valor retornado por la función cargarTransacciones(), ejecutando la lectura de la memoria local (localStorage) en el instante de carga del script.

Este bloque implementa el sistema de **Persistencia de Datos en el Navegador** utilizando la API de localStorage, asegurando que la información de las ventas no se pierda al recargar o cerrar la página.

Explicación línea por línea

1. Encabezado del Módulo de Persistencia

```
JavaScript
// ==========================================
// 2. FUNCIONES DE PERSISTENCIA (LOCALSTORAGE)
// ==========================================
```

- **Líneas 1-3:** Comentario de sección que delimita el inicio del módulo de almacenamiento persistente.

2. Función cargarTransacciones()

```
JavaScript
/**
 * Obtiene las transacciones guardadas en localStorage o retorna los datos de ejemplo
 */
function cargarTransacciones() {
```

- **Líneas 4-7:** Declaración de la función tradicional cargarTransacciones. Su objetivo es recuperar la información previa del navegador o inicializar el dashboard con los datos de muestra.

```
JavaScript
  const guardado = localStorage.getItem('js_nivel2_ventas');
```

- **Línea 8:** Utiliza localStorage.getItem('js_nivel2_ventas') para consultar la clave 'js_nivel2_ventas'.
    - Si la clave existe en la memoria local, retorna una cadena de texto en formato **JSON**.
    - Si es la primera vez que el estudiante/usuario abre la aplicación o se limpió el historial, retorna null.

```
JavaScript
  if (guardado) {
```

- **Línea 9:** Evalúa si la variable guardado contiene datos (es decir, no es null o undefined).

```
JavaScript
      try {
          return JSON.parse(guardado);
       } catch (e) {
          console.error("Error al parsear LocalStorage:", e);
          return [...DATOS_EJEMPLO];
       }
   }
```

- **Líneas 10-16:** Bloque de control de excepciones try...catch.
    - JSON.parse(guardado): Convierte el texto JSON recuperado de nuevo a un **Array de Objetos** de JavaScript usable por la aplicación.
    - catch (e): Si la cadena almacenada en localStorage está corrupta o fue alterada manualmente provocando un error de sintaxis, captura la excepción sin romper la ejecución del programa.
    - console.error(...): Muestra el mensaje de fallo en la consola de herramientas de desarrollo.
    - return [...DATOS_EJEMPLO]: Utiliza el **Operador Spread** (...) para devolver una **copia superficial** del arreglo de muestra como mecanismo de defensa (*fallback*).

```
JavaScript
   return [...DATOS_EJEMPLO];
}
```

- **Líneas 17-18:** Si la condición if (guardado) no se cumple (primera carga), la función ejecuta este retorno por defecto.
    - Devuelve [...DATOS_EJEMPLO] para garantizar que el estado inicial contenga los datos de prueba sin mutar el array original DATOS_EJEMPLO.

3. Función Flecha guardarEnLocalStorage()

```
JavaScript
/**
 * Guarda el array actual de transacciones en localStorage
 */
const guardarEnLocalStorage = () => {
    localStorage.setItem('js_nivel2_ventas', JSON.stringify(transacciones));
};
```

- **Líneas 19-24:** Definición de la función flecha (arrow function) encargada de serializar y sincronizar los datos.
    - const guardarEnLocalStorage = () => { ... }: Sintaxis moderna para funciones anónimas asignadas a constantes.

- JSON.stringify(transacciones): **Serialización de Objetos**. Convierte el array global transacciones en una cadena de texto plano codificada en JSON, ya que localStorage solo permite almacenar cadenas de caracteres (strings).
- localStorage.setItem('js_nivel2_ventas', ...): Escribe/sobrescribe la clave 'js_nivel2_ventas' con el contenido JSON actualizado.

Este bloque contiene el **motor del dashboard**. Representa la sección más importante de lógica en JavaScript Nivel 2, ya que combina la **programación funcional**, la **acumulación avanzada de datos con objetos** y el uso de `Object.entries()`.

Explicación línea por línea

1. Encabezado de la Función `calcularMetricas`

JavaScript

```
// ========================================
// 3. MÉTODOS DE ARRAYS & LÓGICA FINANCIERA
// ========================================

/**
 * Calcula las métricas generales utilizando exclusivamente métodos avanzadas de Array
 * @param {Array} lista - Lista de transacciones filtradas o totales
 * @returns {Object} Objeto con los indicadores clave
 */
const calcularMetricas = (lista) => {
```

- **Líneas 1-8:** Comentario del módulo y anotaciones JSDoc.
- **Línea 9:** Define la función flecha `calcularMetricas` que recibe como parámetro `lista` (un arreglo de objetos de venta).

2. Control de Casos Límite (Lista Vacía)

JavaScript

```
  if (lista.length === 0) {
      return { total: 0, promedio: 0, productoTop: "N/A", productoTopCount: 0, catTop: "N/A", catTopMonto: 0 };
  }
```

- **Líneas 10-12:** Cláusula de guarda (**Guard Clause**). Evita errores de división entre cero o la lectura de propiedades inexistentes cuando los filtros aplicados no devuelven ninguna venta. Devuelve un objeto literal con valores por defecto neutros.

3. Cálculo de Ingresos Totales y Promedio

JavaScript

```
  // A) TOTAL INGRESOS: Uso de reduce() para sumar los montos
  const total = lista.reduce((acumulador, tx) => acumulador + parseFloat(tx.monto), 0);
```

- **Líneas 13-14:** Acumulación matemática lineal.
    - `lista.reduce(...)`: Método funcional de arreglos. Transforma el array entero en un único valor numérico.
    - `acumulador`: El valor que va acumulando la suma paso a paso.
    - `tx`: El objeto transacción actual en la iteración.
    - `parseFloat(tx.monto)`: Convierte el valor de `monto` a un número de punto flotante.
    - `, 0`: **Valor inicial del acumulador**. Esencial para evitar comportamientos inesperados.

JavaScript
```
  // B) PROMEDIO DE VENTA: División entre total y la longitud del array
   const promedio = total / lista.length;
```

- **Líneas 15-16:** Cálculo de la media aritmética dividiendo el monto total entre la cantidad de transacciones procesadas (`lista.length`).

4. Cálculo del Producto Más Vendido (Mapa de Frecuencias)

JavaScript
```
  // C) PRODUCTO MÁS VENDIDO (Frecuencia): Uso de reduce() con un objeto acumulador
   const conteoProductos = lista.reduce((acc, tx) => {
      acc[tx.producto] = (acc[tx.producto] || 0) + 1;
      return acc;
   }, {});
```

- **Líneas 17-21:** Construcción de un mapa de frecuencias.
    - `, {}`: Define un **objeto vacío** como acumulador inicial (`acc`).
    - `acc[tx.producto]`: Acceso a propiedades dinámicas mediante corchetes.
    - `(acc[tx.producto] || 0) + 1`: Evalúa si el producto ya existe en el objeto. Si no existe (`undefined`), usa `0` como valor base y le suma `1`. Si ya existe, incrementa su valor previo en `1`.
    - `return acc`: Retorna el objeto acumulador actualizado a la siguiente iteración.

JavaScript
```
  // Determinar cuál clave (producto) tiene la mayor frecuencia mediante Object.entries()
   let productoTop = "N/A";
   let productoTopCount = 0;

   Object.entries(conteoProductos).forEach(([prod, count]) => {
      if (count > productoTopCount) {
         productoTopCount = count;
```

```
        productoTop = prod;
    }
});
```

- **Líneas 22-31:** Búsqueda del valor máximo dentro de la estructura.
    - `Object.entries(conteoProductos)`: Convierte el objeto de frecuencias en una matriz de pares clave-valor (ej. `[['Laptop', 2], ['Monitor', 1]]`).
    - `.forEach(([prod, count]) => ...)`: Recorre los pares desestructurando la clave (`prod`) y el valor (`count`).
    - `if (count > productoTopCount)`: Algoritmo de búsqueda de máximo. Actualiza el nombre y la cantidad máxima encontrada.

5. Categoría Líder en Ingresos (Agregación por Clave)

JavaScript

```
// D) CATEGORÍA LÍDER EN INGRESOS: Uso de reduce() agregando montos por categoría
const ingresosPorCategoria = lista.reduce((acc, tx) => {
    acc[tx.categoria] = (acc[tx.categoria] || 0) + parseFloat(tx.monto);
    return acc;
}, {});
```

- **Líneas 32-36:** Suma de importes agrupados por categoría.
    - En lugar de sumar `1` como en el conteo, le suma el valor financiero `parseFloat(tx.monto)` a la clave `tx.categoria`. Produce un objeto resultante con los totales monetarios por categoría (ej. `{ Electrónica: 2820.00, Ropa: 45.50 }`).

JavaScript

```
let catTop = "N/A";
let catTopMonto = 0;

Object.entries(ingresosPorCategoria).forEach(([cat, monto]) => {
    if (monto > catTopMonto) {
        catTopMonto = monto;
        catTop = cat;
    }
});
```

- **Líneas 37-45:** Evalúa los datos generados para hallar la categoría con mayores ingresos brutos acumulados.

JavaScript

```
  return { total, promedio, productoTop, productoTopCount, catTop, catTopMonto,
ingresosPorCategoria };
};
```

- **Líneas 46-47:** Devuelve un objeto literal que empaqueta todas las métricas procesadas. Este objeto será consumido por las funciones encargas de actualizar la interfaz gráfica.

Este bloque implementa el **pipeline de procesamiento de datos** del dashboard. Combina las técnicas de **filtrado reactivo con `filter()`** y **ordenamiento dinámico multinivel con `sort()`**.

Explicación línea por línea

1. Firma y Captura de Entradas del DOM

```
JavaScript
/**
 * Filtra y ordena la lista de transacciones según la configuración del usuario
 */
const obtenerTransaccionesFiltradas = () => {
```

- **Líneas 1-4:** Comentario explicativo en formato JSDoc y declaración de la función flecha `obtenerTransaccionesFiltradas`. No recibe parámetros porque consulta de forma reactiva el estado global `transacciones` y los valores actuales de la interfaz.

```
JavaScript
  const busqueda = document.getElementById('filtro-busqueda').value.toLowerCase().trim();
```

- **Línea 5:** Obtiene el texto ingresado en el campo de búsqueda (`id="filtro-busqueda"`).
  - `.value`: Extrae la cadena de texto actual del `<input>`.
  - `.toLowerCase()`: Normaliza todo el texto a minúsculas para realizar una búsqueda insensible a mayúsculas y minúsculas (*case-insensitive*).
  - `.trim()`: Elimina espacios en blanco accidentales al inicio y al final de la cadena de texto.

```
JavaScript
  const categoria = document.getElementById('filtro-categoria').value;
   const orden = document.getElementById('filtro-orden').value;
```

- **Líneas 6-7:** Lee las opciones seleccionadas en las listas desplegables del panel de control: la categoría activa (`'TODAS'`, `'Electrónica'`, etc.) y el criterio de ordenamiento (`'reciente'`, `'monto-desc'`, etc.).

2. Filtrado Combinado (`Array.prototype.filter`)

JavaScript
```
  // 1. Filtrado combinado (filter)
   let resultado = transacciones.filter(tx => {
```

- **Líneas 8-9:** Inicia el filtrado sobre el arreglo principal `transacciones`.
    - `.filter(...)`: Método inmutable de arrays de JavaScript. Recorre cada elemento (`tx`) y crea un nuevo arreglo únicamente con las transacciones que devuelvan `true` en la función callback.

JavaScript
```
      const coincideTexto = tx.producto.toLowerCase().includes(busqueda) ||
                            tx.cliente.toLowerCase().includes(busqueda);
```

- **Líneas 10-11:** Evalúa si la búsqueda por texto coincide con el **producto** o con el **cliente**.
    - `.includes(busqueda)`: Verifica si el término buscado existe como subcadena dentro de la propiedad normalizada a minúsculas. El operador lógico `||` **(OR)** permite buscar en ambos campos de forma simultánea.

JavaScript
```
      const coincideCat = (categoria === 'TODAS') || (tx.categoria === categoria);
```

- **Línea 12:** Evalúa la coincidencia de categoría. Si el selector está en `'TODAS'`, la primera parte se cumple (`true`) y deja pasar cualquier transacción. Si se seleccionó una categoría específica, exige una igualdad estricta (`===`) con `tx.categoria`.

JavaScript
```
      return coincideTexto && coincideCat;
   });
```

- **Líneas 13-14:** Retorna el resultado del operador **`&&` (AND)**. La transacción solo sobrevivirá al filtro si cumple **ambas condiciones al mismo tiempo** (coincidencia de texto Y coincidencia de categoría).

3. Criterio de Ordenamiento Dinámico (`Array.prototype.sort`)

JavaScript
```
  // 2. Ordenamiento (sort)
   resultado.sort((a, b) => {
```

- **Líneas 15-16:** Aplica el método `.sort()` sobre el arreglo ya filtrado (`resultado`).
    - `(a, b) => { ... }`: Función de comparación que toma dos elementos consecutivos (`a` y `b`) para determinar su orden relativo.

JavaScript
```
    if (orden === 'reciente') return new Date(b.fecha) - new Date(a.fecha);
     if (orden === 'antiguo') return new Date(a.fecha) - new Date(b.fecha);
```

- **Líneas 17-18:** Ordenamiento cronológico mediante conversión de fechas.
    - `new Date(b.fecha) - new Date(a.fecha)`: Convierte los strings ISO (`YYYY-MM-DD`) a marcas de tiempo en milisegundos. Restar `b - a` produce un orden **descendente** (la fecha más reciente primero).
    - `new Date(a.fecha) - new Date(b.fecha)`: Restar `a - b` produce un orden **ascendente** (la fecha más antigua primero).

JavaScript
```
    if (orden === 'monto-desc') return b.monto - a.monto;
     if (orden === 'monto-asc') return a.monto - b.monto;
     return 0;
  });
```

- **Líneas 19-22:** Ordenamiento numérico por magnitud financiera.
    - `b.monto - a.monto`: Ordena de mayor a menor importe monetario.
    - `a.monto - b.monto`: Ordena de menor a mayor importe monetario.
    - `return 0`: Mantiene la posición relativa de los elementos si no coincide ningún criterio conocido.

4. Retorno del Conjunto Procesado

JavaScript
```
  return resultado;
};
```

- **Líneas 23-24:** Retorna el arreglo final procesado (filtrado y ordenado), listo para ser consumido por las funciones de renderizado del DOM y cálculo de KPIs.

Este bloque abre la sección dedicada a la **Manipulación y Renderizado del DOM**, incluyendo una función utilitaria para formato financiero y la función encargada de actualizar dinámicamente las Tarjetas de Métricas Principales (KPIs).

Explicación línea por línea

1. Encabezado de la Sección y Utilidad de Formato Moneda

JavaScript

```
// ========================================
// 4. MANIPULACIÓN Y RENDERIZADO DEL DOM
// ========================================

/**
 * Formatea números a moneda USD
 */
const formatearMoneda = (valor) => {
    return new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' }).format(valor);
};
```

- **Líneas 1-3:** Comentario de encabezado que identifica la cuarta sección del script.
- **Líneas 4-6:** Documentación JSDoc y declaración de la función flecha formatearMoneda.
- **Línea 7:** Utiliza la API nativa de JavaScript Intl.NumberFormat.
    - 'en-US': Establece la localización para aplicar separadores de miles con coma (,) y decimales con punto (.).
    - { style: 'currency', currency: 'USD' }: Especifica el formato de moneda en Dólares Estadounidenses (agrega el símbolo $).
    - .format(valor): Toma el valor numérico (ej. 1250) y devuelve la cadena formateada (ej. "$1,250.00").

2. Encabezado de la Función renderizarKPIs

JavaScript

```
/**
 * Renderiza las Tarjetas KPI principales
 */
const renderizarKPIs = (metricas, countTotal) => {
```

- **Líneas 8-11:** Declaración de la función renderizarKPIs. Recibe dos argumentos:
    - metricas: El objeto retornado previamente por la función calcularMetricas() (con totales, promedios, producto top, etc.).
    - countTotal: El número entero de registros filtrados para mostrar en el contador.

3. Actualización de Tarjetas: Ingresos, Contador y Promedio

JavaScript

```
    document.getElementById('kpi-total-ingresos').textContent =
formatearMoneda(metricas.total);
    document.getElementById('kpi-count-transacciones').textContent = countTotal;
    document.getElementById('kpi-promedio-venta').textContent =
formatearMoneda(metricas.promedio);
```

- **Línea 12:** Selecciona el nodo con id="kpi-total-ingresos" en el DOM y reemplaza su contenido textual (textContent) con el resultado de formatearMoneda(metricas.total).

- **Línea 13:** Actualiza el nodo id="kpi-count-transacciones" con la cantidad de transacciones registradas.
- **Línea 14:** Formatea e inyecta el importe del ticket promedio en id="kpi-promedio-venta".

4. Actualización de Tarjetas: Producto Top y Categoría Líder

JavaScript

```
  document.getElementById('kpi-producto-top').textContent = metricas.productoTop;
   document.getElementById('kpi-producto-top-count').textContent =
metricas.productoTopCount;

   document.getElementById('kpi-categoria-top').textContent = metricas.catTop;
   document.getElementById('kpi-categoria-top-monto').textContent =
formatearMoneda(metricas.catTopMonto);
};
```

- **Líneas 15-17:** Asigna el nombre del producto más vendido a id="kpi-producto-top" y su cantidad de unidades/ventas a id="kpi-producto-top-count".
- **Líneas 18-20:** Escribe el nombre de la categoría con mayor facturación en id="kpi-categoria-top" y su monto formateado en id="kpi-categoria-top-monto".
- **Línea 21:** Cierra la función renderizarKPIs.

Este bloque constituye una de las partes centrales del curso: la **generación dinámica de nodos del DOM**, la **gestión de estados vacíos** y la **asignación de atributos de datos (`data-attributes`)** para acciones individuales sobre registros.

Explicación línea por línea

1. Firma y Captura de Contenedores Base

JavaScript

```
/**
 * Genera dinámicamente las filas de la tabla mediante document.createElement()
 */
const renderizarTabla = (lista) => {
   const tbody = document.getElementById('tabla-ventas-body');
   const emptyState = document.getElementById('empty-state');
```

- **Líneas 1-5:** Declaración de la función flecha `renderizarTabla` que acepta la lista de datos procesados (filtrados/ordenados).
- **Líneas 6-7:** Obtiene la referencia al cuerpo de la tabla (`id="tabla-ventas-body"`) y al contenedor del mensaje sin resultados (`id="empty-state"`).

2. Limpieza del Contenido Previo y Control de Estado Vacío

JavaScript
```
  // Limpiar contenido previo del tbody
   tbody.innerHTML = '';
```

- **Línea 9:** Vacía por completo las filas previas del `<tbody>` antes de volver a pintar, evitando duplicados en cada renderizado.

JavaScript
```
  if (lista.length === 0) {
      emptyState.classList.remove('hidden');
   } else {
      emptyState.classList.add('hidden');
   }
```

- **Líneas 10-14:** Evalúa si hay registros a renderizar.
  - Si `lista.length === 0`: Remueve la clase `hidden` para visibilizar el *empty state*.
  - Si hay elementos: Le agrega la clase `hidden` para ocultar el aviso visual.

3. Mapeo de Diccionario de Colores (Objeto Literal para Badges)

JavaScript
```
  // Badge de colores para las categorías
   const BadgesCategoria = {
      'Electrónica': 'bg-cyan-500/10 text-cyan-400 border-cyan-500/20',
      'Ropa': 'bg-violet-500/10 text-violet-400 border-violet-500/20',
      'Hogar': 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20',
      'Alimentos': 'bg-amber-500/10 text-amber-400 border-amber-500/20',
      'Servicios': 'bg-rose-500/10 text-rose-400 border-rose-500/20'
   };
```

- **Líneas 16-23:** Crea un **Objeto Literal como Diccionario (Lookup Table)**. Asocia cada categoría de producto con sus clases CSS de Tailwind para darle un color distintivo al badge sin necesidad de escribir múltiples estructuras condicionales `if/else`.

4. Iteración y Creación Dinámica con `createElement` y Template Literals

JavaScript
```
  // Iteración con map / forEach para la creación de elementos DOM
   lista.forEach((tx) => {
      const tr = document.createElement('tr');
      tr.className = 'hover:bg-slate-800/40 transition duration-150 group';
```

- **Línea 25:** Recorre cada objeto de transacción con el método de array `.forEach()`.

- **Línea 26: Paso clave de manipulación del DOM**: Usa `document.createElement('tr')` para construir un nuevo nodo de fila en memoria.
- **Línea 27:** Asigna las clases de estilos y hover a la fila directamente sobre su propiedad `.className`.

JavaScript
```
const badgeClass = BadgesCategoria[tx.categoria] || 'bg-slate-700 text-slate-300';
```

- **Línea 29:** Busca la clase correspondiente en el objeto `BadgesCategoria` utilizando notación de corchetes. El operador `||` asigna una clase neutra gris por defecto en caso de no hallar coincidencia.

JavaScript
```
// Inserción modular del HTML interno
tr.innerHTML = `
    <td class="py-3 px-4 text-slate-400 whitespace-nowrap">${tx.fecha}</td>
    <td class="py-3 px-4 font-semibold text-white">${tx.producto}</td>
    <td class="py-3 px-4">
        <span class="px-2.5 py-1 text-[10px] font-semibold rounded-md border ${badgeClass}">
            ${tx.categoria}
        </span>
    </td>
    <td class="py-3 px-4 text-slate-300">${tx.cliente}</td>
    <td class="py-3 px-4 font-bold text-right text-emerald-400">${formatearMoneda(tx.monto)}</td>
    <td class="py-3 px-4 text-center">
        <button class="btn-eliminar p-1.5 text-slate-500 hover:text-rose-400 transition"
                data-id="${tx.id}" title="Eliminar transacción">
            <i class="fa-solid fa-trash-can pointer-events-none"></i>
        </button>
    </td>
`;
```

- **Líneas 31-48:** Inyecta la estructura interna de la celda mediante un **Template Literal** (`${...}`).
    - `data-id="${tx.id}"`: Inserta un atributo personalizado **DataSet HTML5** en el botón de eliminar. Esto permite identificar la transacción específica al momento de hacer clic.
    - `pointer-events-none`: Se le aplica al icono interno (`<i>`) para asegurar que cuando el usuario haga clic en la papelera, el evento capture directamente el botón y no el icono.

5. Inserción en el DOM con `appendChild` y Actualización de Contadores

JavaScript
```
      tbody.appendChild(tr);
   });
```

- **Línea 50: Paso final de renderizado**: Adiciona la fila recién creada (`tr`) al árbol del DOM dentro de `tbody` con el método `appendChild()`.
- **Línea 51:** Cierra el bloque del `forEach`.

JavaScript
```
  // Actualizar contador
   document.getElementById('resultados-count').textContent = `Mostrando ${lista.length} de ${transacciones.length} registros`;
};
```

- **Líneas 53-54:** Actualiza la etiqueta del contador en la barra de filtros con una interpolación que compara la cantidad de registros filtrados contra el total global de la base de datos.
- **Línea 55:** Cierra la función `renderizarTabla`.

Este bloque comprende la **generación de las tarjetas con barra de porcentaje por categoría** (transformando objetos a arrays con Object.entries()) y la **función principal orquestadora**, encargada de reactivar toda la vista cuando los datos cambian.

Explicación línea por línea

1. Firma e Inicialización del Contenedor

JavaScript
```
/**
 * Genera las tarjetas de resumen por categoría utilizando Object.entries() y reduce()
 */
const renderizarDesgloseCategorias = (metricasGlobales) => {
   const container = document.getElementById('categoria-breakdown-container');
   container.innerHTML = '';
```

- **Líneas 1-5:** Declaración de la función flecha renderizarDesgloseCategorias. Recibe metricasGlobales, que contiene la agregación de montos procesados por calcularMetricas().
- **Línea 6:** Obtiene la referencia del contenedor en el DOM (id="categoria-breakdown-container").
- **Línea 7:** Vacía el contenido previo (innerHTML = '') para realizar una renderización limpia.

2. Preparación y Transformación de Datos con Object.entries()

JavaScript

```
const ingresosCat = metricasGlobales.ingresosPorCategoria || {};
 const totalGlobal = metricasGlobales.total || 1; // Prevenir división por cero
```

- **Línea 9:** Extrae el objeto ingresosPorCategoria (ej. { Electrónica: 2820, Ropa: 45.5 }). Si por algún motivo no existiera, asigna un objeto vacío {} por seguridad.
- **Línea 10:** Obtiene el gran total financiero. Aplica el operador || 1 como un seguro para evitar divisiones entre cero ($x/0 = NaN$) en caso de que no haya ninguna transacción guardada.

JavaScript
```
const listaCategorias = Object.entries(ingresosCat);
```

- **Línea 12: Concepto Clave de Objetos**: Aplica Object.entries(ingresosCat) para convertir las propiedades del objeto en una **matriz de pares [clave, valor]** iterables.
    - Convierte { "Electrónica": 2820, "Ropa": 45.5 } en [["Electrónica", 2820], ["Ropa", 45.5]].

3. Control de Estado Vacío

JavaScript
```
if (listaCategorias.length === 0) {
    container.innerHTML = `<p class="text-xs text-slate-500 col-span-2">Sin datos de categoría.</p>`;
    return;
 }
```

- **Líneas 14-17:** Verifica si la matriz transformada no tiene elementos. Si está vacía, imprime una leyenda informativa en span de 2 columnas (col-span-2) e interrumpe la función de inmediato mediante return.

4. Iteración y Creación de Tarjetas con Barras de Progreso

JavaScript
```
listaCategorias.forEach(([cat, monto]) => {
    const porcentaje = ((monto / totalGlobal) * 100).toFixed(1);
```

- **Línea 19:** Recorre los pares clave-valor desestructurando directamente en los parámetros [cat, monto].
- **Línea 20:** Calcula la proporción o participación porcentual de esa categoría sobre el total financiero global.
    - .toFixed(1): Redondea el valor decimal a **1 solo dígito después de la coma** (ej. "32.4").

JavaScript
```
    const card = document.createElement('div');
    card.className = 'bg-slate-900/60 p-4 rounded-xl border border-slate-800 flex flex-col justify-between';
```

- **Líneas 22-23:** Crea la tarjeta contenedora en memoria con document.createElement('div') y le asigna sus clases de maquetación Flexbox y fondo.

JavaScript
```
      card.innerHTML = `
          <div class="flex justify-between items-center mb-2">
             <span class="text-xs font-semibold text-slate-200">${cat}</span>
             <span class="text-xs font-bold
text-cyan-400">${formatearMoneda(monto)}</span>
          </div>
          <div class="w-full bg-slate-800 rounded-full h-2 overflow-hidden mb-1">
             <div class="bg-gradient-to-r from-cyan-500 to-violet-500 h-2 rounded-full
transition-all duration-500"
                  style="width: ${porcentaje}%"></div>
          </div>
          <span class="text-[10px] text-slate-500 text-right">${porcentaje}% del total</span>
      `;
```

- **Líneas 25-35:** Construcción del HTML interno mediante Template Literals:
  - Inyecta el nombre de la categoría (${cat}) y su monto formateado (${formatearMoneda(monto)}).
  - **Efecto visual dinámico**: Aplica un atributo inline style="width: ${porcentaje}%" sobre el div del relleno de la barra para ajustar su ancho proporcionalmente de forma fluida (transition-all duration-500).

JavaScript
```
      container.appendChild(card);
   });
};
```

- **Líneas 37-39:** Agrega el nodo de la tarjeta al contenedor general mediante appendChild() y cierra la función.

5. Función Orquestadora Central (actualizarDashboard)

JavaScript
```
/**
 * Función orquestadora que actualiza la interfaz por completo
 */
const actualizarDashboard = () => {
```

- **Líneas 41-44:** Declaración de la función master actualizarDashboard. Sirve como el **punto único de verdad** para redibujar la pantalla.

JavaScript
```
   const filtradas = obtenerTransaccionesFiltradas();
    const metricasFiltradas = calcularMetricas(filtradas);
    const metricasTotales = calcularMetricas(transacciones);
```

- **Líneas 45-47:** Obtiene los conjuntos de datos necesarios:
    - filtradas: Transacciones que cumplen con el texto, la categoría y el orden actual.
    - metricasFiltradas: Métricas (KPIs, promedios) calculadas **solamente** sobre el conjunto filtrado.
    - metricasTotales: Métricas calculadas sobre la **totalidad de transacciones** (para que el desglose por categorías muestre la distribución global).

JavaScript
```
  renderizarKPIs(metricasFiltradas, filtradas.length);
   renderizarTabla(filtradas);
   renderizarDesgloseCategorias(metricasTotales);
};
```

- **Líneas 49-52:** Invoca en secuencia ordenada a las 3 funciones de renderizado, garantizando que los KPIs, la tabla de filas y las tarjetas de categorías queden totalmente sincronizados.

Este bloque abarca la **captura de eventos de formulario**, la **validación de campos**, el uso de **Shorthand Property Names** en objetos literales, la mutación de arrays con `unshift()` y la **sincronización completa con el estado y LocalStorage**.

Explicación línea por línea

1. Encabezado y Asignación del Listener al Formulario

JavaScript
```
// ========================================
// 5. GESTIÓN DE EVENTOS & INTERACTIVIDAD
// ========================================

// Evento Formulario: Agregar Nueva Transacción
document.getElementById('form-nueva-venta').addEventListener('submit', (e) => {
   e.preventDefault();
```

- **Líneas 1-6:** Comentario de sección que inicia la quinta y última fase del script.
- **Línea 7:** Busca el formulario en el DOM (`id="form-nueva-venta"`) y suscribe un escucha con `.addEventListener('submit', ...)`. Escuchar el evento `submit` en el formulario (en lugar del `click` en el botón) permite procesar la venta tanto al hacer clic en *"Guardar Transacción"* como al presionar la tecla *Enter* desde cualquier campo.
- **Línea 8:** `e.preventDefault()`: **Método fundamental**. Detiene el comportamiento predeterminado del navegador de recargar la página o enviar una petición HTTP

GET/POST tradicional, permitiendo que JavaScript tome el control total (*Single Page Application*).

2. Lectura y Saneamiento de Entradas (`value`, `trim`, `parseFloat`)

JavaScript
```
// Captura de valores de los inputs
const producto = document.getElementById('input-producto').value.trim();
const categoria = document.getElementById('select-categoria').value;
const monto = parseFloat(document.getElementById('input-monto').value);
const fecha = document.getElementById('input-fecha').value;
const cliente = document.getElementById('input-cliente').value.trim();
```

- **Líneas 10-15:** Lee el contenido de cada campo del formulario.
  - `.trim()`: Limpia espacios en blanco accidentales al inicio/final del texto escrito en producto y cliente.
  - `parseFloat(...)`: Convierte la cadena del valor numérico del `<input type="number">` a un número de punto flotante usable en operaciones aritméticas.

3. Validación Defensiva de Datos

JavaScript
```
if (!producto || isNaN(monto) || !fecha || !cliente) {
    mostrarToast('Por favor completa todos los campos correctamente.', 'error');
    return;
}
```

- **Líneas 17-20:** Evalúa si alguna de las variables requeridas está vacía o es inválida.
  - `!producto`, `!fecha`, `!cliente`: Evalúan como verdaderos si el string resultante del `.trim()` quedó vacío (`""`).
  - `isNaN(monto)`: Verifica si el usuario ingresó un valor que no pudo convertirse a un número (`NaN`).
  - `mostrarToast(...)`: Llama a la función de alertas flotantes pasando la etiqueta de error.
  - `return`: Interrumpe inmediatamente la ejecución para no agregar registros corruptos al estado global.

4. Construcción del Objeto Transacción y Generación de ID Único

JavaScript
```
// Creación del nuevo objeto transacción
const nuevaVenta = {
    id: 'tx-' + Date.now(),
    producto,
    categoria,
    monto,
    fecha,
```

```
    cliente
};
```

- **Líneas 22-30:** Construcción de un nuevo objeto literal.
    - `id: 'tx-' + Date.now()`: Genera una clave primaria pseudo-única concatenando el prefijo `'tx-'` con la marca de tiempo en milisegundos retornada por `Date.now()`.
    - **Shorthand Property Names (ES6)**: Sintaxis reducida `producto, categoria, monto, fecha, cliente`. Equivale a escribir `producto: producto, categoria: categoria...`, aprovechando que las variables capturadas coinciden exactamente con los nombres de las propiedades del objeto.

5. Actualización del Estado Array (`unshift`), Persistencia y Renderizado

```
JavaScript
  // Agregar al inicio del array
   transacciones.unshift(nuevaVenta);

   // Persistencia
   guardarEnLocalStorage();

   // Actualizar vista
   actualizarDashboard();
```

- **Línea 33:** `transacciones.unshift(nuevaVenta)`: Añade el nuevo registro al **principio del arreglo** (índice `0`). A diferencia de `.push()` (que agrega al final), `unshift()` garantiza que la nueva venta aparezca como la más reciente en la cima de la tabla.
- **Línea 36:** Ejecuta `guardarEnLocalStorage()` para guardar inmediatamente la lista actualizada en el almacenamiento interno del navegador.
- **Línea 39:** Llama a la función orquestadora `actualizarDashboard()`, redibujando los KPIs, la tabla y los porcentajes en tiempo real.

6. Limpieza del Formulario y Notificación al Usuario

```
JavaScript
  // Resetear formulario
   e.target.reset();
   // Restaurar fecha por defecto
   document.getElementById('input-fecha').valueAsDate = new Date();

   mostrarToast('Transacción agregada exitosamente', 'exito');
});
```

- **Línea 42:** `e.target.reset()`: Restablece todos los campos del `<form>` a sus valores iniciales en una sola llamada de código.
- **Línea 44:** Reasigna el campo fecha con el día actual (`new Date()`) mediante la propiedad `valueAsDate`.
- **Línea 46:** Muestra una notificación emergente tipo toast indicando el éxito de la operación.
- **Línea 47:** Cierra la función callback del evento `submit`.

Este bloque aborda patrones clave de interactividad avanzada: **Delegación de Eventos**, **reactividad con callbacks directos**, **manipulación de clases CSS (toggle)** y la **construcción de notificaciones efímeras (Toasts) con temporizadores**.

Explicación línea por línea

1. Delegación de Eventos para Eliminar Registros

JavaScript

```
// Evento Delegado: Eliminar Transacción de la Tabla
document.getElementById('tabla-ventas-body').addEventListener('click', (e) => {
```

- **Líneas 1-2:** Aplica la técnica de **Event Delegation** (Delegación de Eventos).
    - En lugar de adjuntar un escucha individual a cada botón de eliminar en cada fila (lo cual sobrecargaría la memoria), se adjunta un **único listener al contenedor padre** (id="tabla-ventas-body"). Cualquier clic en una celda o botón "burbujeará" (*Event Bubbling*) hasta este elemento.

JavaScript

```
  if (e.target.classList.contains('btn-eliminar')) {
      const id = e.target.getAttribute('data-id');
```

- **Línea 3:** if (e.target.classList.contains('btn-eliminar')): Filtra el clic comprobando si el elemento exactamente presionado (e.target) contiene la clase identificadora btn-eliminar.
- **Línea 4:** Extrae el identificador único guardado en el atributo personalizable HTML5 usando .getAttribute('data-id').

JavaScript

```
      // Confirmación simple mediante array filter
      transacciones = transacciones.filter(tx => tx.id !== id);
      guardarEnLocalStorage();
      actualizarDashboard();

      mostrarToast('Registro eliminado correctamente', 'info');
  }
});
```

- **Línea 7: Inmutabilidad con .filter()**: Remueve el registro reasignando el arreglo global transacciones con una versión filtrada que excluye la transacción cuya propiedad tx.id coincida con el id capturado.
- **Líneas 8-9:** Sincroniza el cambio en el localStorage y vuelve a renderizar la interfaz con actualizarDashboard().
- **Línea 11:** Despliega un mensaje informativo.

2. Escuchas de Eventos para Filtros Reactivos

JavaScript
```
// Listeners para Filtros
document.getElementById('filtro-busqueda').addEventListener('input', actualizarDashboard);
document.getElementById('filtro-categoria').addEventListener('change',
actualizarDashboard);
document.getElementById('filtro-orden').addEventListener('change', actualizarDashboard);
```

- **Línea 16:** Escucha el evento 'input' en el campo de texto. Cada vez que el estudiante teclea o borra un carácter, invoca la función orquestadora actualizarDashboard para refrescar los datos en tiempo real.
- **Líneas 17-18:** Escuchan el evento 'change' en las listas desplegables. Al cambiar de categoría o criterio de orden, actualizan inmediatamente la vista.
    - **Uso limpio de callbacks**: Se pasa la función actualizarDashboard por referencia directa sin necesidad de envolverla en () => actualizarDashboard().

3. Botón para Restaurar Datos de Muestra (Seed)

JavaScript
```
// Botón para Restaurar Datos de Muestra
document.getElementById('btn-seed').addEventListener('click', () => {
   transacciones = [...DATOS_EJEMPLO];
   guardarEnLocalStorage();
   actualizarDashboard();
   mostrarToast('Datos de muestra restaurados', 'info');
});
```

- **Líneas 20-22:** Escucha el clic en el botón *"Restaurar Datos"* (id="btn-seed").
- **Línea 23:** Sobrescribe la variable global transacciones asignándole una **copia superficial** del arreglo original DATOS_EJEMPLO utilizando el operador **Spread** (...).
- **Líneas 24-26:** Persiste la restauración en localStorage, redibuja todo el dashboard y emite un toast de confirmación.

4. Control del Banner / Guía Pedagógica (toggle y classList)

JavaScript
```
// Mostrar / Ocultar Guía Pedagógica
const guideEl = document.getElementById('pedagogical-guide');
const codeToggleText = document.getElementById('code-toggle-text');

document.getElementById('btn-toggle-code').addEventListener('click', () => {
```

```
    guideEl.classList.toggle('hidden');
    const isHidden = guideEl.classList.contains('hidden');
    codeToggleText.textContent = isHidden ? 'Ver Guía Pedagógica' : 'Ocultar Guía';
});
```

- **Líneas 29-31:** Captura el panel pedagógico (guideEl) y la etiqueta del botón (codeToggleText).
- **Línea 34:** guideEl.classList.toggle('hidden'): Alterna la clase hidden. Si el elemento la tiene, se la quita; si no la tiene, se la agrega.
- **Líneas 35-36:** Utiliza un **Operador Ternario** (condición ? valorTrue : valorFalse) para conmutar dinámicamente el texto del botón según el estado de visibilidad del panel.

JavaScript
```
document.getElementById('btn-close-guide').addEventListener('click', () => {
    guideEl.classList.add('hidden');
    codeToggleText.textContent = 'Ver Guía Pedagógica';
});
```

- **Líneas 39-42:** Asigna el evento de cierre al botón de equis (id="btn-close-guide") dentro del banner, forzando la adición de la clase hidden y reajustando la etiqueta del botón principal.

5. Función de Notificaciones Efímeras (mostrarToast)

JavaScript
```
// Notificación Toast
function mostrarToast(mensaje, tipo = 'exito') {
    const container = document.getElementById('toast-container');
    const toast = document.createElement('div');
```

- **Líneas 45-47:** Define la función mostrarToast. Utiliza **parámetros por defecto** (ES6) en tipo = 'exito'.

JavaScript
```
    const colorBg = tipo === 'exito' ? 'bg-emerald-600' : tipo === 'error' ? 'bg-rose-600' :
'bg-cyan-600';
```

- **Línea 49:** Operador ternario anidado para mapear el fondo según el estado:
  - 'exito' → Verde (bg-emerald-600).
  - 'error' → Rojo (bg-rose-600).
  - Por defecto / 'info' → Azul Cian (bg-cyan-600).

JavaScript
```
    toast.className = `${colorBg} text-white text-xs font-semibold px-4 py-3 rounded-xl
shadow-xl flex items-center gap-2 transform transition duration-300 pointer-events-auto`;
    toast.innerHTML = `<i class="fa-solid fa-circle-info"></i> <span>${mensaje}</span>`;

    container.appendChild(toast);
```

- **Líneas 51-54:** Asigna los estilos visuales, construye el contenido interno con la cadena del mensaje y lo inserta en el DOM con appendChild().

```
JavaScript
  setTimeout(() => {
      toast.classList.add('opacity-0', 'translate-y-2');
      setTimeout(() => container.removeChild(toast), 300);
   }, 3000);
}
```

- **Líneas 56-61: Ciclo de vida efímero con asincronía (setTimeout)**:
  - **Primer temporizador (3000 ms = 3s)**: Tras 3 segundos en pantalla, le agrega las clases CSS opacity-0 y translate-y-2 para iniciar una transición suave de desvanecimiento hacia abajo.
  - **Segundo temporizador interno (300 ms)**: Espera a que termine la animación visual de Tailwind CSS y luego ejecuta container.removeChild(toast), **eliminando físicamente el nodo del DOM** para liberar memoria.

Este bloque de cierre completa la **fase de inicialización y el ciclo de vida de la aplicación**, asegurando que la interfaz esté lista y sincronizada desde el primer milisegundo en que la página web termina de cargar.

Explicación línea por línea

1. Encabezado y Asignación del Evento de Carga de la Ventana

```
JavaScript
// ========================================
// 6. INICIALIZACIÓN
// ========================================
window.onload = () => {
```

- **Línea 1-3:** Comentario de encabezado de la sexta y última sección del script.
- **Línea 4:** Asigna una función flecha al evento global `window.onload`.
  - `window.onload`: Se dispara cuando todo el documento (estructura HTML, estilos CSS, fuentes e imágenes) se ha cargado por completo en la memoria del navegador. Es el punto de entrada (**Entry Point**) idóneo para ejecutar la lógica de arranque.

2. Configuración de Fecha Predeterminada en el Formulario

```
JavaScript
  // Establecer fecha de hoy en el formulario por defecto
   document.getElementById('input-fecha').valueAsDate = new Date();
```

- **Líneas 5-6:** Captura el campo de fecha del formulario (`id="input-fecha"`).
    - `new Date()`: Crea una nueva instancia del objeto nativo `Date` que contiene la fecha y hora exacta del sistema en ese momento.
    - `.valueAsDate`: Propiedad nativa de los campos `<input type="date">` que permite asignarle directamente un objeto `Date` de JavaScript, formateando automáticamente el valor al estándar que exige el navegador (`YYYY-MM-DD`). Esto mejora la experiencia de usuario (UX) al evitar que deba seleccionar manualmente la fecha actual para ventas de hoy.

3. Primer Renderizado del Dashboard

JavaScript

```
  // Cargar datos y renderizar interfaz
   actualizarDashboard();
};
```

- **Líneas 7-8:** Llama a la función orquestadora máster `actualizarDashboard()`.
    - Al ejecutarse aquí, desencadena en cadena todo el flujo inicial:
        1. Lee los datos persistidos en `localStorage` (o carga los `DATOS_EJEMPLO` si es la primera vez).
        2. Aplica el filtrado y ordenamiento por defecto.
        3. Calcula los KPIs globales y filtrados con `reduce()`.
        4. Genera las filas de la tabla mediante `createElement()` y `appendChild()`.
        5. Dibuja las tarjetas con barras de progreso de categorías con `Object.entries()`.
- **Línea 9:** Cierra la función callback del evento `window.onload`.

4. Cierre del Script y del Documento HTML

HTML

```
  </script>
</body>
</html>
```

- **Línea 10:** Cierra la etiqueta `<script>` donde residía todo el código de lógica en JavaScript Vanilla.
- **Líneas 11-12:** Cierran las etiquetas principales `</body>` y `</html>`, marcando el final de la estructura completa del archivo.

¡Felicidades! Hemos desglosado y explicado **línea por línea** la totalidad del código (HTML, Tailwind CSS y JavaScript Vanilla Nivel 2) del **Dashboard de Ventas y Análisis Financiero**.

Proyecto 2: Catálogo Interactivo de Cine/Series con Consumo de API

Dificultad: 🟠 **Media** (Enfoque en Asincronismo, Eventos, DOM dinámico y Estado)

Una aplicación estilo *Mini-Netflix* o *TMDB Explorer* que busca películas mediante una API pública (o un simulador con `fetch`/`Promises`), permite ver detalles en un modal, gestionar favoritos y simular llamadas concurrentes.

- **¿Qué hace el proyecto?**
  - Busca películas en tiempo real controlando eventos de teclado (`keydown`/`keyup`) o formulario (`submit`).
  - Consulta información asíncrona mediante `async/await` con manejo de errores (`try...catch`).
  - Implementa un comparador de 2 o más películas al mismo tiempo usando `Promise.all()`.
  - Guarda la lista de películas favoritas en `localStorage`.
- **Temas cubiertos del temario:**
  - **Asincronismo:** Promesas, `async/await`, manejo de errores (`try...catch`), `Promise.all()`, `Promise.race()` (ej. para un timeout de carga).
  - **Gestión de Eventos:** `addEventListener()`, `preventDefault()`, captura/propagación (`bubbling`).
  - **DOM:** Creación/eliminación de modales y tarjetas, modificación de atributos y clases CSS.
  - **Compatibilidad y Recursividad:** Algoritmo recursivo para renderizar categorías anidadas o comentarios.
- **Puntos de dificultad para los estudiantes:**
  - Comprender el flujo asíncrono y evitar renderizar antes de que la promesa se resuelva.
  - Dominar la propagación de eventos (`Event Bubbling`) al hacer clic en botones generados dinámicamente en el DOM.

https://github.com/aristides1000/Cat-logo-Interactivo-de-Cine-Series-con-Consumo-de-API-Uneweb-Javasscript-nivel-2

https://aristides1000.github.io/Cat-logo-Interactivo-de-Cine-Series-con-Consumo-de-API-Uneweb-Javasscript-nivel-2/

Nota, recuerda activar el API Key en el siguiente enlace:

https://www.omdbapi.com/

En la pestaña API Key

Comencemos con la explicación **línea por línea** del código HTML, CSS y la configuración inicial de esta aplicación de cine.

1. Cabecera HTML y Configuración de Estilos (Líneas 1 - 50)

HTML
```
<!DOCTYPE html>
<html lang="es" class="dark">
```

- **Línea 1:** Declaración <!DOCTYPE html>, especifica la versión Estándar **HTML5**.
- **Línea 2:** Tag raíz <html lang="es" class="dark">. Activa el idioma español y establece la clase dark a nivel global para que **Tailwind CSS** aplique el modo oscuro por defecto.

HTML
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CineStream Explorer - Paginación & Fallbacks</title>
```

- **Líneas 3-6:** Configuración de caracteres UTF-8, ajuste de escala para móviles (*responsive viewport*) y el título visible en la pestaña del navegador.

HTML
```
  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      darkMode: 'class',
      theme: {
        extend: {
          colors: {
            slate: { 850: '#111827', 900: '#0f172a', 950: '#050914' },
            brand: { red: '#E50914', crimson: '#B81D24', cyan: '#06B6D4', gold:
'#F59E0B' }
          },
          fontFamily: { sans: ['Inter', 'sans-serif'] }
        }
      }
    }
  </script>
```

- **Línea 8:** Carga la librería framework **Tailwind CSS** desde su CDN.

- **Líneas 9-22:** Personalización del objeto global tailwind.config. Extiende la paleta añadiendo tonos oscuros de azul (slate-850, 900, 950) y la gama corporativa brand (rojo tipo Netflix #E50914, cian y dorado).

HTML
```
  <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&displ
ay=swap" rel="stylesheet">
   <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
```

- **Líneas 23-27:** Carga la tipografía moderna **Inter** desde Google Fonts y la librería de iconos **FontAwesome 6**.

HTML
```
  <style>
      body { font-family: 'Inter', sans-serif; background-color: #050914; color: #f3f4f6; }
      ::-webkit-scrollbar { width: 8px; height: 8px; }
      ::-webkit-scrollbar-track { background: #0f172a; }
      ::-webkit-scrollbar-thumb { background: #334155; border-radius: 4px; }
      ::-webkit-scrollbar-thumb:hover { background: #475569; }
```

- **Líneas 28-35:** Estilos globales CSS. Aplica el fondo #050914 al cuerpo de la página y personaliza la barra de desplazamiento (*scrollbar*) para darle una estética oscura moderna.

HTML
```
      .glass-card {
          background: rgba(15, 23, 42, 0.75);
          backdrop-filter: blur(16px);
          border: 1px solid rgba(255, 255, 255, 0.08);
          box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.4);
      }
      .glass-input {
          background: rgba(30, 41, 59, 0.6);
          border: 1px solid rgba(255, 255, 255, 0.12);
          color: #f8fafc;
      }
      .glass-input:focus {
          border-color: #E50914;
          outline: none;
          box-shadow: 0 0 0 2px rgba(229, 9, 20, 0.3);
      }
      .movie-card:hover .movie-overlay { opacity: 1; transform: translateY(0); }
      .movie-overlay { transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1); }
  </style>
```

</head>

- **Líneas 36-54:** Clases personalizadas de **Glassmorphism**:
  - .glass-card: Oculta parcialmente el fondo aplicando un desenfoque de cristal (backdrop-filter: blur(16px)) con bordes sutiles.
  - .glass-input: Da estilo semitransparente a las cajas de texto y cambia al rojo #E50914 cuando se selecciona (:focus).
  - .movie-card:hover .movie-overlay: Revela la capa flotante sobre el póster de la película al pasar el cursor por encima.

2. Encabezado de Navegación (Header) (Líneas 51 - 92)

HTML

```
<body class="min-h-screen pb-16 antialiased selection:bg-brand-red selection:text-white">

    <!-- Header Principal -->
    <header class="border-b border-slate-800/80 bg-slate-950/80 sticky top-0 z-40 backdrop-blur-xl">
        <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-3.5">
            <div class="flex flex-col md:flex-row md:items-center md:justify-between gap-4">
```

- **Líneas 51-56:** Elemento <body> con altura mínima de pantalla (min-h-screen).
- **Línea 54:** Encabezado con navegación fija en la parte superior (sticky top-0 z-40) y desenfoque traslúcido.

HTML

```
                <div class="flex items-center space-x-3">
                    <div class="p-2.5 bg-gradient-to-tr from-red-600 to-rose-500 rounded-xl shadow-lg shadow-red-600/30">
                        <i class="fa-solid fa-film text-xl text-white"></i>
                    </div>
                    <div>
                        <div class="flex items-center space-x-2">
                            <h1 class="text-xl font-extrabold tracking-tight text-white flex items-center gap-1.5">
                                CineStream <span class="text-brand-red">Explorer</span>
                            </h1>
                            <span class="px-2.5 py-0.5 text-[10px] font-bold bg-red-500/10 text-red-400 border border-red-500/20 rounded-full">PAGINACIÓN API</span>
                        </div>
                        <p class="text-xs text-slate-400">Proyecto 2 JS Nivel 2: Paginación Real, Manejo de Errores de Imagen & Async/Await</p>
                    </div>
                </div>
```

- **Líneas 57-70:** Bloque de marca e identidad visual. Muestra el logotipo con un degradado rojo y la insignia informativa del proyecto.

HTML

```
            <div class="flex flex-wrap items-center gap-2.5">
                <button id="btn-open-comparator" class="relative px-3.5 py-2 text-xs font-semibold text-slate-200 bg-slate-800 hover:bg-slate-700 rounded-xl border border-slate-700 transition flex items-center gap-2">
                    <i class="fa-solid fa-scale-balanced text-amber-400"></i> Comparar
                    <span id="badge-compare-count" class="px-1.5 py-0.2 bg-amber-500 text-slate-950 font-extrabold rounded-full text-[10px]">0</span>
                </button>

                <button id="btn-show-favorites" class="px-3.5 py-2 text-xs font-semibold text-slate-200 bg-slate-800 hover:bg-slate-700 rounded-xl border border-slate-700 transition flex items-center gap-2">
                    <i class="fa-solid fa-heart text-brand-red"></i> Favoritos
                    <span id="badge-fav-count" class="px-1.5 py-0.2 bg-brand-red text-white font-bold rounded-full text-[10px]">0</span>
                </button>

                <button id="btn-config-api" class="p-2 text-xs text-slate-400 hover:text-white bg-slate-800/80 rounded-xl border border-slate-700" title="Configurar API Key">
                    <i class="fa-solid fa-key"></i>
                </button>
            </div>
        </div>
    </div>
</header>
```

- **Líneas 71-88:** Botones de acción rápida en el encabezado:
    - id="btn-open-comparator": Abre el modal comparador e incluye el badge id="badge-compare-count" para mostrar la cantidad de películas seleccionadas.
    - id="btn-show-favorites": Alterna a la vista de favoritos guardados en localStorage.
    - id="btn-config-api": Permite al usuario cambiar dinámicamente la API Key de OMDb.

Aquí tienes la explicación detallada **línea por línea** de la sección de la Barra de Búsqueda de tu proyecto:

1. Tag Contenedor Principal `<main>`

HTML

```
<main class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 pt-6 space-y-8">
```

- **`<main>`**: Etiqueta semántica HTML5 que delimita el contenido principal de la aplicación.
- **`max-w-7xl mx-auto`**: Establece un ancho máximo de **80rem (1280px)** y centra el contenedor horizontalmente en la pantalla (`mx-auto`).
- **`px-4 sm:px-6 lg:px-8`**: Padding horizontal adaptable (*responsive*): `16px` en móviles, `24px` en pantallas medianas (`sm`) y `32px` en pantallas grandes (`lg`).
- **`pt-6`**: Padding superior de `1.5rem (24px)` para separar el contenido del header.
- **`space-y-8`**: Agrega automáticamente un margen superior de `2rem (32px)` entre cada hijo directo dentro del `<main>`.

2. Tarjeta Contenedora de la Búsqueda (`<section>`)

HTML

<!-- Barra de Búsqueda -->

<section class="glass-card rounded-2xl p-5 border border-slate-800/80">

- **`<section>`**: Bloque semántico que agrupa la interfaz del formulario de búsqueda.
- **`glass-card`**: Aplica la clase CSS personalizada de **Glassmorphism** (efecto traslúcido con `backdrop-filter: blur(16px)`).
- **`rounded-2xl`**: Bordes fuertemente redondeados (`1rem` / `16px`).
- **`p-5`**: Padding interno uniforme de `1.25rem (20px)`.
- **`border border-slate-800/80`**: Borde sutil de color azul oscuro con 80% de opacidad.

3. Formulario y Maquetación Grid (`<form>`)

HTML

<form id="form-search" class="grid grid-cols-1 md:grid-cols-12 gap-3 items-center">

- **`<form id="form-search">`**: Formulario HTML con un identificador único para capturar el evento `submit` desde JavaScript.
- **`grid grid-cols-1 md:grid-cols-12`**: Sistema de maquetación CSS Grid. En móviles usa **1 sola columna**; a partir de pantallas medianas (`md:`) se divide en **12 columnas**.
- **`gap-3`**: Espaciado interno de `0.75rem (12px)` entre los campos del formulario.
- **`items-center`**: Alinea verticalmente todos los elementos en el centro de la fila.

4. Campo de Texto e Indicador de Carga (Columnas 1 a 6)

HTML

- **`relative`**: Establece posicionamiento relativo para permitir posicionar de forma absoluta el icono de lupa y el spinner de carga.
- **`md:col-span-6`**: En pantallas medianas y grandes, este contenedor ocupa **6 de las 12 columnas** del grid (el 50% del ancho del formulario).

HTML

- **`<i>`**: Icono de lupa de FontAwesome.
- **`absolute left-4 top-3.5`**: Posicionamiento absoluto fija el icono en el interior del campo de texto (`16px` a la izquierda y `14px` desde arriba).
- **`text-slate-400 text-sm`**: Color gris atenuado y tamaño de fuente pequeño (`14px`).

HTML

<input type="text" id="input-search" placeholder="Buscar película o serie (Ej: Batman, Marvel, Star Wars)..."

class="w-full pl-11 pr-4 py-3 rounded-xl text-sm glass-input font-medium" autocomplete="off" required>

- **`<input type="text">`**: Campo de entrada principal de búsqueda.
- **`id="input-search"`**: ID para extraer su valor (`value.trim()`) desde JavaScript.
- **`pl-11`**: Padding izquierdo amplio (`2.75rem` / `44px`) para dejar espacio y no sobreponer el texto con el icono de la lupa.
- **`pr-4 py-3`**: Relleno interno a la derecha (`16px`) y verticalmente (`12px`).
- **`rounded-xl text-sm font-medium`**: Bordes redondeados de `12px`, fuente de `14px` y grosor medio.
- **`glass-input`**: Clase CSS personalizada que aplica fondo oscuro semitransparente y resalte rojo (`#E50914`) al enfocar (`:focus`).
- **`autocomplete="off" required`**: Desactiva el historial emergente del navegador y hace obligatorio llenar el campo antes de enviar.

HTML

```
    <i class="fa-solid fa-spinner text-sm"></i>
</span>
</div>
```

- **`<span id="search-spinner">`**: Contenedor para el indicador visual de carga durante las peticiones `fetch()`.
- **`hidden`**: Oculto por defecto; JavaScript le remueve la clase `hidden` al iniciar la búsqueda.
- **`absolute right-4 top-3.5`**: Posicionado a la derecha del `input`.
- **`text-brand-red animate-spin`**: Aplica el color rojo de la marca e inicia la animación continua de rotación CSS de Tailwind (`animate-spin`).

5. Filtro Selector de Tipo (Columnas 7 a 9)

HTML

```
<div class="md:col-span-3">
    <select id="select-type" class="w-full py-3 px-3.5 text-sm rounded-xl glass-input bg-slate-900 font-medium">
        <option value="">Todos los tipos</option>
        <option value="movie">Películas</option>
        <option value="series">Series de TV</option>
    </select>
</div>
```

- **`md:col-span-3`**: Ocupa **3 de las 12 columnas** (25% del ancho del formulario).
- **`<select id="select-type">`**: Menú desplegable para filtrar los resultados mediante el parámetro `&type=` de la API de OMDb.
- **`option value=""`**: Envia una cadena vacía a la API para traer tanto películas como series.
- **`option value="movie"`** / **`option value="series"`**: Envía `movie` o `series` a la petición `fetch()` para acotar los resultados.

6. Botones de Acción (Columnas 10 a 12)

HTML

- **`md:col-span-3`**: Ocupa las últimas **3 de las 12 columnas** (25% del ancho).
- **`flex items-center gap-2`**: Alinea los dos botones horizontalmente con un espacio de `8px` entre ellos.

HTML

<button type="submit" class="w-full py-3 px-4 bg-gradient-to-r from-red-600 to-rose-600 hover:from-red-500 hover:to-rose-500 text-white font-bold text-sm rounded-xl shadow-lg shadow-red-600/25 transition flex items-center justify-center gap-2">

 Buscar

</button>

- **`type="submit"`**: Dispara el evento `submit` del formulario al hacer clic o presionar *Enter*.
- **`w-full`**: Hace que el botón ocupe todo el espacio disponible dentro de su columna.
- **`bg-gradient-to-r from-red-600 to-rose-600`**: Gradiente de color horizontal de rojo a rosado.
- **`hover:from-red-500 hover:to-rose-500`**: Aumenta el brillo del degradado al pasar el cursor.
- **`shadow-lg shadow-red-600/25`**: Sombra flotante roja con 25% de opacidad para dar un efecto *neon/glow*.
- **`transition`**: Suaviza las animaciones al pasar el cursor (`hover`).

HTML

<button type="button" id="btn-quick-trends" class="py-3 px-3 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-xl border border-slate-700 text-xs font-semibold transition" title="Buscar Aleatorio">

</button>

- **`type="button"`**: Importante especificarlo para evitar que este botón ejecute el `submit` predeterminado del formulario.
- **`id="btn-quick-trends"`**: Escuchado por JavaScript para seleccionar un título aleatorio del arreglo `TITULOS_ALEATORIOS` y ejecutar la búsqueda.

- **`<i class="fa-solid fa-shuffle text-amber-400">`**: Icono de reproducción aleatoria en amarillo dorado.

Aquí tienes el desglose detallado **línea por línea** de esta sección final del HTML, que abarca la interfaz de Estado Vacío, el Modal de Detalles, el Modal Comparador de películas y el contenedor de Notificaciones Toast:

1. Componente de Estado Vacío (`<div id="empty-state">`)

HTML

<!-- Estado Vacío -->

- **`id="empty-state"`**: Contenedor manipulado por JavaScript a través de la función `mostrarEstadoVacio()`. Se muestra cuando la API devuelve un error (ej. sin coincidencias o API Key inválida).
- **`hidden`**: Oculto por defecto mediante la utilidad de Tailwind (`display: none`).
- **`text-center py-16`**: Centra el texto e íconos, con un relleno vertical amplio (`4rem` / `64px`).
- **`glass-card rounded-2xl border border-slate-800`**: Aplica la estética traslúcida con bordes redondeados y un contorno azul oscuro.

HTML

- **`inline-flex p-4 rounded-full bg-slate-800/80`**: Crea una insignia circular (`rounded-full`) alrededor del icono con un relleno de `16px` y fondo azul de opacidad reducida.
- **`<i class="fa-solid fa-film text-3xl text-brand-red">`**: Icono de cinta de película en tamaño grande (`30px`) con el rojo de la marca (`#E50914`).

HTML

<h3 id="empty-title" class="text-base font-bold text-white">No se encontraron películas</h3>

Intenta con otro término de búsqueda o selecciona una opción distinta.

</main>

- **`id="empty-title"` y `id="empty-desc"`**: Título y descripción con IDs dinámicos. JavaScript altera su contenido (`textContent`) según el contexto (por ejemplo: "Sin favoritos", "Error de conexión", etc.).
- **`</main>`**: Cierra formalmente la etiqueta de contenido principal de la página.

2. Modal de Detalles de Película (`<div id="modal-details">`)

HTML

<!-- MODAL DETALLES -->

- **`id="modal-details"`**: Telón de fondo (*backdrop*) del modal de detalles.
- **`fixed inset-0 z-50`**: Posicionamiento fijo que cubre el 100% de la ventana visual (`top: 0, right: 0, bottom: 0, left: 0`) situándose en la capa superior (`z-index: 50`).
- **`hidden`**: Oculto por defecto hasta que el usuario presiona "Ver Detalles".
- **`bg-slate-950/80 backdrop-blur-md`**: Oscurece el fondo con un azul muy oscuro al 80% y aplica desenfoque gaussiano (`backdrop-filter`).
- **`flex items-center justify-center p-4 overflow-y-auto`**: Centra la tarjeta modal en pantalla y permite desplazamiento vertical (*scroll*) si el contenido excede la pantalla en móviles.

HTML

- **`w-full max-w-3xl`**: Ventana del modal con un ancho adaptable hasta un máximo de `48rem` (`768px`).
- **`rounded-3xl overflow-hidden`**: Bordes suavizados de `24px`. Oculta cualquier elemento o imagen que sobresalga de la esquina.

- **`shadow-2xl relative`**: Sombra pronunciada para dar profundidad tridimensional respecto al fondo.

HTML

<button id="btn-close-modal" class="absolute top-4 right-4 z-10 w-9 h-9 rounded-full bg-slate-900/80 text-slate-300 hover:text-white hover:bg-red-600 transition flex items-center justify-center">

</button>

- **`id="btn-close-modal"`**: Botón de cierre posicionado en la esquina superior derecha (`absolute top-4 right-4`).
- **`w-9 h-9 rounded-full`**: Botón circular de `36px x 36px`.
- **`hover:bg-red-600 transition`**: Transición suave que tiñe el botón de rojo intenso al pasar el cursor.
- **`<i class="fa-solid fa-xmark text-lg">`**: Icono de la "X" para cerrar.

HTML

- **`id="modal-content"`**: Contenedor Grid (1 columna en móvil, 12 columnas en escritorio) donde la función `renderizarModalDetalles()` inyecta el póster (5 columnas) y la información/sinopsis traducida al español latino (7 columnas).

3. Modal Comparador de Películas (`<div id="modal-comparator">`)

HTML

<!-- MODAL COMPARADOR -->

- **`id="modal-comparator"`**: Telón de fondo fijo para el módulo de comparación concurrente de películas.

- **`bg-slate-950/85 backdrop-blur-md`**: Fondo oscuro con desenfoque de cristal.

HTML

- **`max-w-5xl`**: Ancho máximo extendido de `64rem` (`1024px`) para acomodar hasta 3 películas lado a lado horizontalmente.
- **`p-6 space-y-6`**: Relleno interno de `24px` y separación vertical de `24px` entre la cabecera, la grilla comparativa y las acciones.

HTML

<h3 class="text-lg font-bold text-white flex items-center gap-2">

 Comparador de Películas (Promise.all)

</h3>

Consulta asíncrona concurrente de múltiples títulos en tiempo real.

<button id="btn-close-comparator" class="text-slate-400 hover:text-white text-lg"></button>

- **Líneas del encabezado del modal**: Muestra el título con el icono de la balanza en amarillo/ámbar, un subtítulo descriptivo de la tecnología utilizada (`Promise.all`) y el botón `btn-close-comparator` para ocultar la ventana.

HTML

- **`id="comparator-grid"`**: Contenedor objetivo formateado como Grid de hasta 3 columnas (`lg:grid-cols-3`), donde `renderizarComparador()` renderiza las fichas dinámicas con los datos de taquilla, calificación IMDb y género de las películas seleccionadas.

HTML

```
      <div class="flex justify-end gap-3 pt-2">
        <button id="btn-clear-compare" class="px-4 py-2 text-xs font-semibold text-rose-400 bg-rose-950/40 hover:bg-rose-900/60 rounded-xl border border-rose-800/60 transition">
          Vaciar Comparador
        </button>
      </div>
    </div>
</div>
```

- **`id="btn-clear-compare"`**: Botón alineado a la derecha (`flex justify-end`) que vacía el arreglo `EstadoApp.listaComparar` y limpia el `localStorage`.

4. Contenedor de Notificaciones Toast (`<div id="toast-container">`)

HTML

```
<!-- Toast Notifications -->
<div id="toast-container" class="fixed bottom-5 right-5 z-50 flex flex-col gap-2 pointer-events-none"></div>
```

- **`id="toast-container"`**: Contenedor flotante donde la función utilitaria `mostrarToast()` inserta las alertas temporales que informan acciones al usuario (ej. *"Guardado en favoritos"*, *"Error de API"*, etc.).
- **`fixed bottom-5 right-5 z-50`**: Posicionamiento fijo flotante en la esquina inferior derecha (`20px` desde abajo y `20px` desde la derecha).
- **`flex flex-col gap-2`**: Apila las notificaciones verticalmente con una separación de `8px`.
- **`pointer-events-none`**: Permite que los clics atraviesen el contenedor cuando no hay mensajes activos, asegurando que no bloquee la interacción con los elementos que quedan debajo.

Aquí tienes la explicación detallada **línea por línea** del bloque de Configuración, Constantes y Estado Global en JavaScript:

1. Etiqueta de Script y Declaración de la API Key

JavaScript

```
<script>

    // 1. CONFIGURACIÓN Y ESTADO GLOBAL

    // ----------------------------------------------------------------

    let OMDB_API_KEY = "1f23a416";
```

- **<script>**: Apertura de la etiqueta HTML donde se ejecuta la lógica JavaScript.
- **let OMDB_API_KEY = "1f23a416";**: Declara la variable para la clave de autenticación de OMDb API. Se utiliza let (en lugar de const) porque su valor puede ser reasignado dinámicamente si el usuario decide cambiarla desde el botón de configuración de la interfaz.

2. URL Base y la Imagen Fallback

JavaScript

```
    const OMDB_BASE_URL = "https://www.omdbapi.com/";
```

- **const OMDB_BASE_URL**: Constante con el endpoint principal de la API de OMDb. Evita repetir la cadena de texto fija al construir las peticiones HTTP mediante fetch().

JavaScript

```
    // Imagen de reemplazo por defecto cuando falla o no existe póster en la API

     const POSTER_DEFAULT =
'https://images.unsplash.com/photo-1489599849927-2ee91cede3ba?q=80&w=400&auto=format&fit=crop';
```

- **const POSTER_DEFAULT**: URL de una imagen estática almacenada en Unsplash (un proyector de cine a oscuras). Funciona como **imagen de reserva o fallback** en dos escenarios:
    1. Cuando la API responde con Poster: "N/A".
    2. Cuando la URL del póster enviada por la API falla al cargar o da error 404 (capturado mediante el evento onerror en el HTML).

3. El Objeto de Estado Global (**EstadoApp**)

JavaScript

```
// Estado de la Aplicación ampliado con Paginación

const EstadoApp = {
```

- **const EstadoApp**: Objeto literal central que gestiona el **Single Source of Truth** (fuente única de verdad) de toda la aplicación. Almacena en memoria el estado reactivo sin necesidad de consultar el DOM continuamente.

JavaScript

```
busquedaActual: "Batman",

tipoActual: "",
```

- **busquedaActual: "Batman"**: Guarda el término de búsqueda activo (por defecto *"Batman"* al cargar la página por primera vez).
- **tipoActual: ""**: Guarda el filtro de tipo de contenido seleccionado ("" para todos, "movie" para películas o "series" para series).

JavaScript

```
paginaActual: 1,

totalPaginas: 1,

totalResultados: 0,
```

- **paginaActual: 1**: Mantiene el control del número de página consultado actualmente en la API para enviar en el parámetro &page=N.
- **totalPaginas: 1**: Almacena el cálculo total de páginas disponibles ($\lceil \text{totalResultados} / 10 \rceil$).
- **totalResultados: 0**: Guarda la cantidad total bruta de ítems coincidentes devueltos por la propiedad totalResults de OMDb.

JavaScript

```
resultados: [],
```

- **resultados: []**: Arreglo que almacena los 10 objetos de películas de la página activa que retornó la búsqueda.

JavaScript

```
    favoritos: JSON.parse(localStorage.getItem('cinestream_favs')) || [],
```

- **favoritos: ...**: Carga las películas guardadas por el usuario desde la memoria del navegador (localStorage).
  - **localStorage.getItem('cinestream_favs')**: Lee la cadena JSON almacenada.
  - **JSON.parse(...)**: Convierte la cadena en un arreglo de JavaScript.
  - **|| []**: Evaluador lógico fallback. Si localStorage está vacío (null), inicializa un arreglo vacío [] para prevenir errores de tipo en tiempo de ejecución.

JavaScript

```
    listaComparar: JSON.parse(localStorage.getItem('cinestream_compare')) || [],
```

- **listaComparar: ...**: Carga los IDs (imdbID) de las películas agregadas al comparador desde localStorage. Si no existen datos previos, arranca con [].

JavaScript

```
    vistaActiva: "busqueda"

};
```

- **vistaActiva: "busqueda"**: Controla el contexto visual actual de la interfaz ("busqueda" o "favoritos"). Permite saber si la paginación debe dibujarse o esconderse.

4. Colección de Títulos Aleatorios

JavaScript

```
const TITULOS_ALEATORIOS = ["Marvel", "Spider-Man", "Star Wars", "Harry Potter", "Jurassic", "Matrix", "Lord of the Rings"];
```

- **const TITULOS_ALEATORIOS**: Arreglo de cadenas de texto con franquicias populares. Se utiliza en la función asociada al botón aleatorio (#btn-quick-trends) para realizar una consulta rápida al azar mediante Math.random().

Este bloque contiene el **módulo principal de consumo asíncrono y paginación** de la API de OMDb. Aplica conceptos clave de JavaScript Nivel 2 como async/await, peticiones HTTP con fetch(), control defensivo con try...catch...finally y sincronización directa con el objeto de estado reactivo EstadoApp.

Explicación línea por línea

1. Firma de la Función y Parámetros por Defecto (ES6)

JavaScript

```
// 2. PETICIONES ASÍNCROANAS A LA API CON PAGINACIÓN

// ------------------------------------------------------------------


async function buscarPeliculasAPI(query, tipo = "", pagina = 1) {
```

- **Líneas 1-3:** Comentario organizador del script.
- **Línea 4:** Declaración de la función asíncrona buscarPeliculasAPI anteponiendo la palabra clave async. Esto permite utilizar el operador await en su interior para pausar la ejecución de forma no bloqueante hasta que las promesas se resuelvan.
- **Parámetros con valores por defecto (ES6):**
  - query: La cadena de texto a buscar (obligatorio, ej. "Batman").
  - tipo = "": Tipo de medio ("movie", "series" o "" por defecto para consultar ambos).
  - pagina = 1: Número de página de la API que se desea consultar (por defecto inicia en la primera página).

2. Actualización del Estado Pre-Petición e Indicador de Carga

JavaScript

```
  mostrarCargando(true);

   EstadoApp.busquedaActual = query;

   EstadoApp.tipoActual = tipo;

   EstadoApp.paginaActual = pagina;
```

- **Línea 5:** Llama a la función helper mostrarCargando(true) para hacer visible el spinner animado de Tailwind (#search-spinner) en la interfaz.
- **Líneas 6-8: Persistencia en el estado reactivo (EstadoApp)**: Guarda en memoria los tres parámetros de la consulta actual (busquedaActual, tipoActual, paginaActual).

Esto garantiza que si el usuario cambia de página o aplica filtros, la app recuerde qué término estaba buscando.

3. Construcción del Endpoint URL y Petición HTTP (fetch + await)

JavaScript

```
  try {

      // Parámetro &page=N para la paginación nativa de OMDb API

      const url =
`${OMDB_BASE_URL}?apikey=${OMDB_API_KEY}&s=${encodeURIComponent(query)}&t
ype=${tipo}&page=${pagina}`;

      const respuesta = await fetch(url);

      const datos = await respuesta.json();
```

- **Línea 9:** Inicio del bloque defensivo try...catch.
- **Línea 11:** Construcción de la URL mediante un **Template Literal** (${...}):
  - ${OMDB_BASE_URL}: Endpoint base (https://www.omdbapi.com/).
  - apikey=${OMDB_API_KEY}: Credencial de acceso.
  - s=${encodeURIComponent(query)}: **Seguridad en la URL**. encodeURIComponent codifica caracteres especiales o espacios en la búsqueda (ej. transforma "Spider Man" en "Spider%20Man").
  - type=${tipo}: Filtro por película o serie.
  - page=${pagina}: Parámetro nativo de OMDb que devuelve los resultados paginados en bloques de 10.
- **Línea 12:** const respuesta = await fetch(url): Realiza la petición HTTP asíncrona. La palabra clave await detiene la ejecución dentro de la función hasta que el servidor devuelva los encabezados de la respuesta.
- **Línea 13:** const datos = await respuesta.json(): Procesa y parsea el cuerpo de la respuesta desde formato JSON a un objeto JavaScript ejecutable.

4. Procesamiento de Respuesta Exitosa (datos.Response === "True")

JavaScript

```
      if (datos.Response === "True") {

          EstadoApp.resultados = datos.Search;

          EstadoApp.totalResultados = parseInt(datos.totalResults, 10);

          EstadoApp.totalPaginas = Math.ceil(EstadoApp.totalResultados / 10);
```

- **Línea 15:** OMDb API devuelve una propiedad de texto Response: "True" cuando la búsqueda tuvo éxito.
- **Línea 16:** Asigna el arreglo de 10 películas encontradas (datos.Search) a EstadoApp.resultados.
- **Línea 17:** parseInt(datos.totalResults, 10): Convierte el conteo total de coincidencias devuelto como cadena por la API a un número entero base 10.
- **Línea 18: Cálculo matemático de páginas**: Utiliza Math.ceil(totalResultados / 10) para redondear hacia arriba el total de páginas. Si la API reporta 23 resultados, $23/10 = 2.3$, que al aplicar Math.ceil() se convierte en **3 páginas necesarias**.

JavaScript

```
        renderizarGrillaPeliculas(datos.Search);

        renderizarPaginacion();

        actualizarContadorResultados(`Mostrando página ${pagina} de
${EstadoApp.totalPaginas} (${EstadoApp.totalResultados} títulos)`);
```

- **Líneas 20-22:** Invocación de las funciones de UI para actualizar la pantalla:
    1. Dibuja las 10 tarjetas de las películas devueltas.
    2. Genera los controles y botones numerados de la paginación.
    3. Actualiza el indicador del header con la información de la página actual.

5. Manejo de Búsquedas Sin Coincidencias (Response === "False")

JavaScript

```
    } else {

        EstadoApp.resultados = [];

        EstadoApp.totalResultados = 0;

        EstadoApp.totalPaginas = 1;



        renderizarGrillaPeliculas([]);

        renderizarPaginacion();

        mostrarEstadoVacio("Sin resultados", datos.Error || "No se encontraron
coincidencias.");

    }
```

- **Líneas 23-26:** Si la API devuelve Response: "False" (por ejemplo, al teclear texto sin sentido), resetea los indicadores de estado a cero/vacío.
- **Líneas 28-30:** Oculta la grilla de películas, oculta la paginación e invoca a mostrarEstadoVacio() pasando el mensaje de error que proviene de la API (datos.Error).

6. Captura de Errores de Red y Limpieza Global (catch & finally)

JavaScript

```
  } catch (error) {

      console.error("Error al buscar en la API:", error);

      mostrarToast("Error de conexión con la API", "error");

      mostrarEstadoVacio("Error de red", "Por favor verifica tu conexión o tu API Key.");

  } finally {

      mostrarCargando(false);

  }

}
```

- **Líneas 32-36 (catch):** Si ocurre un error crítico en la red (ej. sin acceso a internet, endpoint caído o fallo de DNS), captura la excepción evitando que la aplicación se rompa, emite un aviso toast flotante de color rojo y muestra el componente de estado vacío con la leyenda "Error de red".
- **Líneas 37-39 (finally): Bloque de garantía de ejecución**. Sin importar si la petición fue exitosa o falló con un error, la clausula finally ejecuta mostrarCargando(false) para ocultar el spinner de carga, evitando que se quede girando indefinidamente en la pantalla.

Esta función es la **orquestadora de la traducción**. Aplica un algoritmo de fragmentación (**chunking**) para evitar el límite de 500 caracteres de la API pública y resuelve los fragmentos simultáneamente usando `Promise.all()`.

Explicación línea por línea

1. Firma y Cláusulas de Guarda

JavaScript

```
/**

 * Traduce de forma asíncrona la sinopsis (Plot) y los premios (Awards) al español latino.
```

```
 * Maneja automáticamente el límite de 500 caracteres dividiendo el texto en fragmentos.
 */
async function traducirAlEspanolLatino(texto, esPremios = false) {
```

- **Líneas 1-4:** Comentario JSDoc explicativo del método.
- **Línea 5:** Declaración de la función asíncrona (`async`).
  - `texto`: La cadena enviada por OMDb (ej. la sinopsis completa).
  - `esPremios = false`: Flag booleano opcional que indica si el texto a procesar pertenece a la sección de premios.

JavaScript

```
  if (!texto || texto === 'N/A') return 'Información no disponible';
```

- **Línea 6: Guard Clause (Validación previa)**. Si el parámetro está vacío, es `undefined` o OMDb retornó `'N/A'`, interrumpe la función de inmediato y retorna la cadena en español por defecto.

JavaScript

```
  // 1. Limpieza de frases de error previas si quedaron cacheadas
   if (texto.includes("QUERY LENGTH LIMIT EXCEEDED")) {
      return "Sinopsis no disponible en español.";
   }
```

- **Líneas 8-11:** Filtro de seguridad. Evita reprocesar o enviar a traducir un texto que ya contenga el mensaje de error de límite previo.

2. Caso Rápido: Texto Corto

JavaScript

```
  try {
      // 2. Si el texto es corto (menor a 450 caracteres), se traduce directamente
      if (texto.length <= 450) {
         return await pedirTraduccionMyMemory(texto, esPremios);
      }
```

- **Línea 12:** Inicio del bloque defensivo `try...catch`.
- **Líneas 14-17:** Evaluación de longitud. Si la sinopsis o texto de premios no supera los 450 caracteres (por debajo del límite de 500 de la API), hace la llamada directa mediante `await pedirTraduccionMyMemory(...)` y retorna el resultado sin fragmentar.

3. Algoritmo de Fragmentación (Chunking por Oraciones)

JavaScript

```
// 3. Si el texto supera los 450 caracteres, se divide por oraciones/puntos (.)
const oraciones = texto.match(/[^.!?]+[.!?]+/g) || [texto];
```

- **Línea 19-20:** Expresión Regular (**RegEx**) para fragmentar.
  - `/[^.!?]+[.!?]+/g`: Busca bloques de texto que terminen en punto (`.`), signo de exclamación (`!`) o interrogación (`?`).
  - `|| [texto]`: Fallback si el texto no contiene signos de puntuación, creando un array de un único elemento con la cadena entera.

JavaScript

```
let fragmentoActual = "";
const promesasFragmentos = [];
```

- **Líneas 21-22:** Inicialización de variables acumuladoras:
  - `fragmentoActual`: Almacena temporalmente la concatenación de oraciones antes de enviarlas.
  - `promesasFragmentos`: Array donde se guardarán las **promesas pendientes** de cada fragmento.

JavaScript

```
for (const oracion of oraciones) {
    if ((fragmentoActual + oracion).length > 400) {
        promesasFragmentos.push(pedirTraduccionMyMemory(fragmentoActual, esPremios));
        fragmentoActual = oracion;
    } else {
```

```
            fragmentoActual += " " + oracion;
        }
    }
```

- **Líneas 24-31:** Bucle de empaquetado `for...of`:
    - `if ((fragmentoActual + oracion).length > 400)`: Verifica si agregar la siguiente oración supera los 400 caracteres.
    - `promesasFragmentos.push(...)`: Si los supera, inicia la petición asíncrona llamando a `pedirTraduccionMyMemory()` (sin anteponer `await`, por lo que **retorna una Promesa no resuelta**) y la guarda en la colección. Luego reinicia `fragmentoActual` con la nueva oración.
    - `else`: Si aún cabe dentro del margen de 400 caracteres, la concatena con un espacio.

JavaScript

```
    if (fragmentoActual.trim().length > 0) {
        promesasFragmentos.push(pedirTraduccionMyMemory(fragmentoActual, esPremios));
    }
```

- **Líneas 32-34:** Procesa el remanente. Si al terminar el bucle queda algún texto guardado en `fragmentoActual`, añade su correspondiente promesa al arreglo.

4. Resolución Concurrente con `Promise.all` y Ensamblado

JavaScript

```
    // 4. Se traducen todos los bloques en paralelo y se unen
    const resultadosTraduccion = await Promise.all(promesasFragmentos);
    return resultadosTraduccion.join(" ");
```

- **Línea 37:** `await Promise.all(promesasFragmentos)`: **Eje de asincronismo en paralelo**. Ejecuta todas las solicitudes HTTP a la API de traducción de manera simultánea. Espera a que todas se completen y devuelve un arreglo con las cadenas traducidas en el mismo orden original.
- **Línea 38:** `.join(" ")`: Reconstruye la sinopsis uniendo los fragmentos del arreglo con un espacio en blanco.

5. Manejo de Excepciones y Fallback Local

JavaScript

```
  } catch (error) {
      console.warn("Fallo en la traducción, aplicando fallback local:", error);
      return aplicarFallbackLocal(texto, esPremios);
   }
}
```

- **Líneas 40-44:** Si cualquier punto de la red o del parseo falla, captura la excepción con `catch`, emite una advertencia en la consola y ejecuta el diccionario local de reemplazo (`aplicarFallbackLocal`) para garantizar que la interfaz no se rompa ni muestre errores al usuario.

Este bloque constituye el **núcleo de ejecución de traducciones HTTP y resiliencia**. Se encarga de procesar las peticiones individuales hacia la API pública de traducción, sanear las respuestas devueltas mediante expresiones regulares (RegEx) para dar un tono en **Español Latino** y proporcionar un sistema de respaldo (**fallback local**) en caso de fallos de red o errores de la API.

Explicación línea por línea

1. Firma e Inicio de la Función pedirTraduccionMyMemory

JavaScript

```
/**
 * Petición individual a MyMemory API (máximo 450 caracteres)
 */
async function pedirTraduccionMyMemory(textoParte, esPremios) {
```

- **Líneas 1-4:** Comentario JSDoc explicativo del método.
- **Línea 5:** Declaración de la función asíncrona pedirTraduccionMyMemory. Recibe dos argumentos:
  - textoParte: Un fragmento individual de texto que no supera los 450 caracteres.
  - esPremios: Booleano que indica si se trata del texto de reconocimientos para el tratamiento de términos.

2. Petición HTTP e Invocación de MyMemory API

JavaScript

```
  try {

      const url =
`https://api.mymemory.translated.net/get?q=${encodeURIComponent(textoParte.trim())}&lan
gpair=en|es`;

      const res = await fetch(url);

      const data = await res.json();
```

- **Línea 6:** Apertura del bloque de control de errores try...catch.
- **Línea 7:** Construcción de la URL de consulta:
    - q=${encodeURIComponent(textoParte.trim())}: Limpia espacios en los extremos (.trim()) y codifica la cadena para evitar caracteres no válidos en la URL.
    - langpair=en|es: Especifica el par de idiomas de traducción (Inglés a Español).
- **Líneas 8-9:** Realiza la petición asíncrona con fetch(url) y parsea la respuesta JSON de vuelta (data).

3. Validación de Respuesta y Filtro Anti-Límite

JavaScript

```
    if (data && data.responseData && data.responseData.translatedText) {

        let traducido = data.responseData.translatedText;



        // Filtro por si MyMemory devuelve el error de límite

        if (traducido.includes("QUERY LENGTH LIMIT EXCEEDED")) {

            return aplicarFallbackLocal(textoParte, esPremios);

        }
```

- **Líneas 11-12:** Verifica la presencia segura de las propiedades deeply nested (data.responseData.translatedText) en la respuesta JSON.
- **Líneas 14-17: Filtro Defensivo**. Si la API devuelve el mensaje de error "QUERY LENGTH LIMIT EXCEEDED" empaquetado dentro de la propiedad de texto, interrumpe el flujo y redirige al fallback local.

4. Post-procesamiento y Adaptación a Español Latino (RegEx)

JavaScript

```
        // Adaptación de términos a Español Latino
        return traducido
            .replace(/\bNominated for\b/gi, 'Nominada a')
            .replace(/\bWon\b/gi, 'Ganadora de')
            .replace(/\bOscars?\b/gi, 'Premios Óscar')
            .replace(/\bBAFTA Film Award\b/gi, 'Premio BAFTA')
            .replace(/\bwins\b/gi, 'victorias')
            .replace(/\bnominations\b/gi, 'nominaciones');
    }
```

- **Líneas 19-27:** Aplica encadenamiento de métodos de reemplazo con **Expresiones Regulares (RegEx)**:
    - \b: Límite de palabra (*Word Boundary*) para evitar reemplazar coincidencias parciales dentro de otras palabras.
    - g (Global): Reemplaza todas las apariciones en la cadena.
    - i (Insensitive): Ignora mayúsculas y minúsculas.
    - Transforma términos formales del inglés audiovisual a frases naturales en español latino (ej. *"Won 3 Oscars"* → *"Ganadora de 3 Premios Óscar"*).

5. Control de Excepciones y Retorno por Defecto

JavaScript

```
  } catch (e) {
      return aplicarFallbackLocal(textoParte, esPremios);
  }
  return textoParte;
}
```

- **Líneas 29-31:** Si la petición HTTP falla (ej. pérdida de conexión), ejecuta aplicarFallbackLocal.
- **Línea 32:** Si la API no devolvió un objeto válido pero no lanzó un error de red, retorna la cadena original en inglés sin alterar.

6. Función de Resguardo Local (aplicarFallbackLocal)

JavaScript

```
/**
 * Fallback de seguridad en caso de fallo de API o límite alcanzado
 */
function aplicarFallbackLocal(texto, esPremios) {
    if (esPremios) {
        return texto
            .replace(/Nominated for/g, 'Nominada a')
            .replace(/Won/g, 'Ganadora de')
            .replace(/Oscars/g, 'Premios Óscar')
            .replace(/Oscar/g, 'Premio Óscar')
            .replace(/wins/g, 'victorias')
            .replace(/nominations/g, 'nominaciones')
            .replace(/Another/g, 'Otras')
            .replace(/win/g, 'victoria');
    }
    return texto;
}
```

- **Líneas 34-39:** Definición de la función sincrónica aplicarFallbackLocal.
- **Líneas 40-52:** Si el texto procesado corresponde a la categoría de premios (esPremios === true), ejecuta un diccionario directo de sustitución mediante .replace() sobre el texto en inglés original para que la interfaz mantenga traducciones clave aunque la red o la API externa no estén disponibles.

Este bloque implementa dos patrones fundamentales de **asincronismo avanzado**: la hidratación/enriquecimiento de un objeto remoto con datos traducidos y la **ejecución concurrente masiva** de múltiples peticiones HTTP en paralelo con `Promise.all()`.

Explicación línea por línea

1. Función `obtenerDetallesPeliculaAPI` (Consulta individual de ficha completa)
JavaScript

async function obtenerDetallesPeliculaAPI(imdbID) {

- **Línea 1:** Declaración de la función asíncrona (`async`). Recibe como parámetro el identificador único `imdbID` (ej. `"tt0371746"` para Iron Man).

JavaScript

```
  try {
      const url = `${OMDB_BASE_URL}?apikey=${OMDB_API_KEY}&i=${imdbID}&plot=full`;
      const res = await fetch(url);
      const datos = await res.json();
```

- **Línea 2:** Abre el bloque de manejo de errores `try...catch`.
- **Líneas 3-5:** Construye la URL de detalle cambiando el parámetro de búsqueda `s=` por **`i=${imdbID}`** para solicitar la ficha completa de un solo título. Incluye `plot=full` para que la API devuelva la sinopsis extensa. Realiza la petición con `fetch()` y procesa el cuerpo JSON.

JavaScript

```
      if (datos.Response === "True") {
          // Traducción concurrente de la sinopsis y premios al español latino
          const [sinopsisLatino, premiosLatino] = await Promise.all([
              traducirAlEspanolLatino(datos.Plot, false),
              traducirAlEspanolLatino(datos.Awards, true)
          ]);
```

- **Línea 7:** Verifica si la API devolvió una respuesta válida (`datos.Response === "True"`).

- **Líneas 9-12: Punto pedagógico clave (Concurrencia con `Promise.all`)**:
    - En lugar de esperar secuencialmente a que termine de traducirse la sinopsis para recién empezar a traducir los premios (`await traducicion1; await traduccion2;`), se envían **ambas solicitudes en paralelo**.
    - `Promise.all([...])` toma el arreglo de promesas no resueltas.
    - **Desestructuración de Arrays**: `const [sinopsisLatino, premiosLatino] = ...` asigna de forma directa las respuestas una vez que ambas operaciones asíncronas han finalizado.

JavaScript

```
        datos.PlotEs = sinopsisLatino;

        datos.AwardsEs = premiosLatino;



        return datos;

    }

    return null;
```

- **Líneas 14-15: Hidratación/Enriquecimiento de objetos**: Añade dinámicamente dos nuevas propiedades al objeto `datos` (`PlotEs` y `AwardsEs`) con el contenido en español latino, conservando también los originales en inglés (`Plot` y `Awards`).
- **Líneas 17-18:** Retorna el objeto mutado y enriquecido. Si la API devolvió `Response: "False"`, retorna `null`.

JavaScript

```
  } catch (e) {

      return null;

   }

}
```

- **Líneas 20-22:** Captura cualquier fallo de red y retorna `null` para no interrumpir el flujo que consuma esta función.

2. Función `cargarComparacionConcurrente` (Comparador Multihilo)

JavaScript

async function cargarComparacionConcurrente() {

- **Línea 24:** Declaración de la función asíncrona encargada de consultar las películas seleccionadas para el comparador.

JavaScript

```
  if (EstadoApp.listaComparar.length === 0) {
      document.getElementById('comparator-grid').innerHTML = `
          <p class="text-xs text-slate-400 col-span-full text-center py-8">
              No has seleccionado películas para comparar.
          </p>`;
      return;
  }
```

- **Líneas 25-31: Guard Clause (Cláusula de guarda)**. Si el arreglo de IDs a comparar (`EstadoApp.listaComparar`) está vacío, inyecta un aviso textual en el contenedor `#comparator-grid` e interrumpe la función con `return`.

JavaScript

```
  document.getElementById('comparator-grid').innerHTML = `
      <div class="col-span-full text-center py-12">
          <i class="fa-solid fa-spinner animate-spin text-3xl text-amber-400 mb-2"></i>
          <p class="text-xs text-slate-300">Cargando datos concurrentes con Promise.all()...</p>
      </div>`;
```

- **Líneas 33-37:** Inyecta un *feedback* visual con un spinner animado mientras se resuelven las peticiones HTTP en segundo plano.

JavaScript

```
  try {
```

```
    const promesas = EstadoApp.listaComparar.map(id => obtenerDetallesPeliculaAPI(id));

    const resultados = await Promise.all(promesas);

    renderizarComparador(resultados.filter(p => p !== null));
```

- **Línea 40: Uso de `.map()` para generar arreglos de promesas**: Recorre cada `id` guardado en la lista de comparación y llama a `obtenerDetallesPeliculaAPI(id)`. Como no se usa `await` dentro del `map`, este devuelve un **Array de Promesas pendientes** (`[Promise, Promise, ...]`).
- **Línea 41:** `await Promise.all(promesas)`: Lanza la ejecución concurrente de todas las consultas de la lista en simultáneo.
- **Línea 42:** `resultados.filter(p => p !== null)`: Limpia el arreglo resultante descartando cualquier película que haya fallado o devuelto `null` antes de enviarlo a `renderizarComparador()`.

JavaScript

```
  } catch (e) {

    mostrarToast("Error en comparación concurrente", "error");

   }

}
```

- **Líneas 43-46:** Si ocurre una falla en el bloque de comparación, captura la excepción y notifica al usuario mediante un aviso Toast emergente.

Este bloque implementa el **módulo de renderizado dinámico del catálogo de películas**, destacando la creación interactiva de elementos en el DOM, la validación defensiva de imágenes en dos capas y la parametrización de estados (favoritos y comparador).

Explicación línea por línea

1. Firma e Inicialización del Contenedor DOM

JavaScript

```
// 3. RENDERIZADO DEL DOM & MANEJO DE IMÁGENES DEFENSIVO

// ----------------------------------------------------------------


function renderizarGrillaPeliculas(lista) {
```

```
const grid = document.getElementById('movies-grid');

const emptyState = document.getElementById('empty-state');

grid.innerHTML = '';
```

- **Líneas 1-4:** Comentario de sección y declaración de la función `renderizarGrillaPeliculas(lista)`, que recibe el arreglo de películas a dibujar.
- **Líneas 5-6:** Captura los nodos contenedores de la rejilla principal (`#movies-grid`) y del mensaje de lista vacía (`#empty-state`).
- **Línea 7:** `grid.innerHTML = ''`: Vacía por completo las tarjetas renderizadas anteriormente para evitar duplicados antes de pintar el nuevo conjunto de datos.

2. Control de Estado Vacío (Guard Clause)

JavaScript

```
if (!lista || lista.length === 0) {

    grid.classList.add('hidden');

    emptyState.classList.remove('hidden');

    return;

}


grid.classList.remove('hidden');

emptyState.classList.add('hidden');
```

- **Líneas 9-13:** Evalúa si la lista no existe (`null`/`undefined`) o no tiene elementos (`lista.length === 0`).
    - `grid.classList.add('hidden')`: Oculta el contenedor de la rejilla.
    - `emptyState.classList.remove('hidden')`: Muestra el mensaje visual de estado vacío.
    - `return`: Corta la ejecución de la función.
- **Líneas 15-16:** Si hay elementos en la lista, asegura que la rejilla sea visible y el estado vacío permanezca oculto.

3. Iteración e Inspección de Estado Local (`some` e `includes`)

JavaScript

```
lista.forEach(item => {

    const esFavorito = EstadoApp.favoritos.some(f => f.imdbID === item.imdbID);

    const estaEnComparar = EstadoApp.listaComparar.includes(item.imdbID);
```

- **Línea 18:** Recorre cada objeto de película dentro del arreglo usando `.forEach()`.
- **Línea 19:** `EstadoApp.favoritos.some(...)`: Utiliza el método de array `.some()` para verificar si al menos un elemento guardado en favoritos coincide con el `imdbID` del ítem actual. Devuelve un valor booleano (`true`/`false`).
- **Línea 20:** `EstadoApp.listaComparar.includes(...)`: Consulta si el arreglo plano de IDs seleccionados para comparar contiene el `imdbID` actual.

4. Validación Defensiva de Imágenes (Nivel 1: Prevención en Código)

JavaScript

```
    // Validación de imagen 1: Comprobar si la API devuelve 'N/A'

    const tienePosterValido = item.Poster && item.Poster !== 'N/A';

    const posterUrl = tienePosterValido ? item.Poster : POSTER_DEFAULT;
```

- **Líneas 22-24: Capa 1 de seguridad de imágenes**. Verifica que la propiedad `item.Poster` contenga texto y no sea estrictamente la cadena `"N/A"` (que envía la API de OMDb cuando no posee el póster). Si la imagen no es válida, asigna la constante estática `POSTER_DEFAULT`.

5. Creación del Nodo e Inyección del HTML (Nivel 2: Control de Errores HTTP)

JavaScript

```
    const card = document.createElement('div');

    card.className = 'glass-card rounded-2xl overflow-hidden relative group movie-card border border-slate-800 hover:border-red-500/50 transition duration-300 flex flex-col justify-between';
```

- **Líneas 26-27:** Crea un elemento `<div>` en memoria con `document.createElement()` y le asigna las clases de estilos Glassmorphism, bordes redondeados y transiciones.

JavaScript

```
    card.innerHTML = `
```

```
<div class="relative overflow-hidden aspect-[2/3] bg-slate-900">

<!-- Validación de imagen 2: Evento onerror para capturar URLs rotas de la API
-->

<img src="${posterUrl}"

alt="${item.Title}"

onerror="this.onerror=null; this.src='${POSTER_DEFAULT}';"

class="w-full h-full object-cover group-hover:scale-105 transition duration-500"

loading="lazy">
```

- **Líneas 29-37: Capa 2 de seguridad de imágenes (`onerror`)**:
  - Si la API envió una URL que parecía válida pero la imagen remota está rota, devolvió error 404 o fue bloqueada por CORS, el navegador dispara el evento `onerror`.
  - `this.onerror=null`: Desconecta el listener para evitar un bucle infinito en caso de que la propia imagen por defecto también falle.
  - `this.src='${POSTER_DEFAULT}'`: Reemplaza dinámicamente el atributo `src` en caliente por la imagen por defecto.
  - `loading="lazy"`: Aplica carga perezosa nativa del navegador para optimizar el rendimiento.

6. Renderizado Condicional de Botones y Metadatos

JavaScript

```
<div class="movie-overlay absolute inset-0 bg-gradient-to-t from-slate-950
via-slate-950/60 to-transparent opacity-0 p-4 flex flex-col justify-between">

<div class="flex justify-between items-start">

<span class="px-2 py-0.5 text-[10px] font-extrabold bg-red-600 text-white
rounded-md uppercase tracking-wider">

${item.Type}

</span>

<button class="btn-toggle-fav p-2 rounded-full ${esFavorito ? 'bg-red-600
text-white' : 'bg-slate-900/80 text-slate-300 hover:text-red-400'} transition"
data-id="${item.imdbID}">

<i class="fa-solid fa-heart pointer-events-none"></i>

</button>
```

```
            </div>
```

- **Líneas 39-48:** Inyecta la capa emergente (*overlay*). Usa un operador ternario `${esFavorito ? 'bg-red-600 text-white' : ...}` para pintar el botón del corazón de color rojo si el producto ya está en favoritos.
- `data-id="${item.imdbID}"`: Atributo dataset clave para la posterior **delegación de eventos**.

JavaScript

```
            <div class="space-y-2">

                <button class="btn-open-details w-full py-2 bg-red-600 hover:bg-red-500 text-white font-bold text-xs rounded-xl transition flex items-center justify-center gap-1.5 shadow-lg shadow-red-600/30" data-id="${item.imdbID}">

                    <i class="fa-solid fa-circle-info pointer-events-none"></i> Ver Detalles

                </button>

                <button class="btn-toggle-compare w-full py-1.5 ${estaEnComparar ? 'bg-amber-500 text-slate-950' : 'bg-slate-800 text-slate-200 hover:bg-slate-700'} font-semibold text-[11px] rounded-xl transition flex items-center justify-center gap-1" data-id="${item.imdbID}">

                    <i class="fa-solid fa-scale-balanced pointer-events-none"></i> ${estaEnComparar ? 'En Comparador' : 'Comparar'}

                </button>

            </div>

        </div>

    </div>
```

- **Líneas 50-57:** Botones de acción dinámica dentro de la tarjeta. Modifica la etiqueta y los colores del botón comparador de acuerdo a la variable `estaEnComparar`.

JavaScript

```
    <div class="p-3.5 space-y-1 bg-slate-900/40">

        <h3 class="font-bold text-xs text-white truncate" title="${item.Title}">${item.Title}</h3>
```

```
            <div class="flex items-center justify-between text-[11px] text-slate-400 font-medium">

                <span><i class="fa-regular fa-calendar text-brand-red"></i> ${item.Year}</span>

                <span class="text-amber-400 font-semibold"><i class="fa-solid fa-star"></i> IMDb</span>

            </div>

        </div>

    `;



    grid.appendChild(card);

  });

}
```

- **Líneas 59-67:** Pie de la tarjeta con el título truncado (`truncate`), año de lanzamiento e insignias.
- **Línea 69:** `grid.appendChild(card)`: Inserta físicamente la tarjeta recién construida dentro del contenedor del DOM `#movies-grid`.

Este bloque constituye el **algoritmo de paginación dinámica**. Gestiona la visibilidad del panel, actualiza las etiquetas de recuento, habilita/deshabilita botones de navegación y calcula una **ventana flotante centrado de páginas numeradas** (máximo 5 visibles a la vez).

Explicación línea por línea

1. Firma y Captura de Referencias del DOM

JavaScript

```
/**

 * Renderiza el Panel de Paginación Dinámico

 */

function renderizarPaginacion() {

  const container = document.getElementById('pagination-container');
```

```
const btnPrev = document.getElementById('btn-page-prev');

const btnNext = document.getElementById('btn-page-next');

const lblCurrent = document.getElementById('lbl-page-current');

const lblTotal = document.getElementById('lbl-page-total');

const lblItems = document.getElementById('lbl-total-items');

const numbersContainer = document.getElementById('pagination-numbers');
```

- **Líneas 1-4:** Comentario JSDoc explicativo.
- **Línea 5:** Declaración de la función `renderizarPaginacion()`.
- **Líneas 6-12:** Obtiene todas las referencias requeridas del DOM: el contenedor general, los botones de dirección (*Anterior* / *Siguiente*), los textos informativos y el contenedor interno donde se generarán los números de página.

2. Cláusula de Guarda y Control de Visibilidad

JavaScript

```
// Si estamos en la vista de favoritos o no hay páginas suficientes, oculta la paginación

if (EstadoApp.vistaActiva === 'favoritos' || EstadoApp.totalPaginas <= 1) {

    container.classList.add('hidden');

    return;

}
```

- **Líneas 14-18: Guard Clause de visibilidad**:
  - Evalúa si la vista activa es `'favoritos'` (ya que los favoritos están almacenados localmente y no paginados desde la API) **O** si el total de páginas devuelto por la API es menor o igual a `1`.
  - En cualquiera de esos dos casos, le añade la clase `hidden` al contenedor para ocultar la barra por completo y sale inmediatamente de la función con `return`.

3. Actualización de Indicadores y Estado de Botones

JavaScript

```
container.classList.remove('hidden');

lblCurrent.textContent = EstadoApp.paginaActual;
```

lblTotal.textContent = EstadoApp.totalPaginas;

lblItems.textContent = EstadoApp.totalResultados;

- **Línea 20:** Remueve la clase `hidden` para hacer visible el panel de paginación.
- **Líneas 21-23:** Actualiza el texto de los indicadores del DOM (`textContent`) consumiendo las propiedades del objeto de estado global `EstadoApp`.

JavaScript

btnPrev.disabled = EstadoApp.paginaActual <= 1;

btnNext.disabled = EstadoApp.paginaActual >= EstadoApp.totalPaginas;

- **Línea 25:** Deshabilita el botón *Anterior* (`btnPrev.disabled = true`) si la página actual es la primera (`<= 1`).
- **Línea 26:** Deshabilita el botón *Siguiente* (`btnNext.disabled = true`) si el usuario se encuentra en la última página disponible.

4. Algoritmo de Ventana Flotante (Cálculo de `inicio` y `fin`)

JavaScript

// Renderizar botones numerados alrededor de la página actual

numbersContainer.innerHTML = '';

const maxVisible = 5;

let inicio = Math.max(1, EstadoApp.paginaActual - 2);

let fin = Math.min(EstadoApp.totalPaginas, inicio + maxVisible - 1);

- **Línea 29:** Vacía los botones numéricos creados en el renderizado anterior.
- **Línea 30:** Establece un límite constante `maxVisible = 5` para no saturar la pantalla con decenas de botones si la API devuelve miles de resultados.
- **Línea 31:** `Math.max(1, EstadoApp.paginaActual - 2)`: Intenta centrar la página actual mostrando 2 números a la izquierda. `Math.max` garantiza que `inicio` nunca sea inferior a `1`.
- **Línea 32:** `Math.min(...)`: Define el extremo derecho de la ventana sumando `4` al índice inicial. `Math.min` asegura que `fin` no exceda el límite total de páginas devuelto por la API.

JavaScript

```
  if (fin - inicio < maxVisible - 1) {

      inicio = Math.max(1, fin - maxVisible + 1);

   }
```

- **Líneas 34-36: Ajuste de límite superior**: Si el usuario está cerca del final (por ejemplo, en la página 10 de 10), el cálculo inicial de `inicio` quedaría descalibrado. Este bloque ajusta `inicio` hacia atrás para garantizar que siempre se muestre el bloque completo de 5 botones mientras existan suficientes páginas disponibles.

5. Bucle de Creación e Inyección de Botones Numerados

JavaScript

```
  for (let i = inicio; i <= fin; i++) {

      const btnNum = document.createElement('button');

      const esActual = i === EstadoApp.paginaActual;
```

- **Línea 38:** Bucle `for` que itera desde el índice `inicio` calculado hasta `fin`.
- **Línea 39:** Crea dinámicamente el elemento `<button>` en memoria con `document.createElement('button')`.
- **Línea 40:** Define un booleano `esActual` verificando si el número del ciclo (`i`) coincide con la página activa en el estado.

JavaScript

```
      btnNum.className = `w-8 h-8 text-xs font-bold rounded-xl transition ${

          esActual

          ? 'bg-red-600 text-white shadow-lg shadow-red-600/30'

          : 'bg-slate-900 text-slate-400 hover:bg-slate-800 hover:text-white border
border-slate-800'

      }`;
```

- **Líneas 41-45:** Aplica clases CSS con un operador ternario:
    - Si `esActual === true`: Pinta el botón con el rojo característico de la marca (`bg-red-600`) y sombra brillante.

- Si es una página secundaria: Le otorga un fondo oscuro (`bg-slate-900`) con un efecto hover de aclarado.

JavaScript

```
      btnNum.textContent = i;
       btnNum.onclick = () => cambiarPagina(i);
       numbersContainer.appendChild(btnNum);
   }
}
```

- **Línea 46:** Asigna el número ordinal `i` como texto visible del botón.
- **Línea 47:** Asigna el evento click llamando a `cambiarPagina(i)` con el número de página específico.
- **Línea 48:** Inserta el botón individual dentro del contenedor `#pagination-numbers`.
- **Línea 49:** Cierra la función `renderizarPaginacion()`.

Este bloque de código combina el **mecanismo de navegación entre páginas con retroalimentación de desplazamiento suave** y la **inyección dinámica de datos enriquecidos (traducidos al español latino)** dentro del modal de detalles.

Explicación línea por línea

1. Función `cambiarPagina` (Navegación y Scroll Suave)

JavaScript

```
function cambiarPagina(nuevaPagina) {
```

- **Línea 1:** Declaración de la función `cambiarPagina`. Recibe el entero `nuevaPagina` que indica el número de página solicitado.

JavaScript

```
  if (nuevaPagina < 1 || nuevaPagina > EstadoApp.totalPaginas || nuevaPagina === EstadoApp.paginaActual) return;
```

- **Línea 2: Guard Clause (Validación de rangos)**. Interrumpe la ejecución si el número solicitado:
    - Es menor a 1 (`nuevaPagina < 1`).
    - Desborda el total de páginas disponibles (`nuevaPagina > EstadoApp.totalPaginas`).
    - Es exactamente la misma página en la que el usuario ya se encuentra (`nuevaPagina === EstadoApp.paginaActual`).

JavaScript

```
  // Solicita la nueva página a la API
   buscarPeliculasAPI(EstadoApp.busquedaActual, EstadoApp.tipoActual, nuevaPagina);
```

- **Líneas 4-5:** Dispara la petición a la API de OMDb reutilizando la consulta guardada en el estado (`EstadoApp.busquedaActual` y `EstadoApp.tipoActual`), enviando la variable `nuevaPagina` en el parámetro `&page=N`.

JavaScript

```
  // Scroll suave hacia la sección de resultados
   document.getElementById('anchor-results').scrollIntoView({ behavior: 'smooth' });
}
```

- **Líneas 7-8: Mejora de Experiencia de Usuario (UX)**. Localiza el nodo `#anchor-results` (situado arriba de la rejilla de películas) y ejecuta `.scrollIntoView()`.
- `{ behavior: 'smooth' }`: Le indica al navegador que realice un **desplazamiento animado suave** hacia arriba en lugar de un salto brusco e instantáneo.

2. Función `renderizarModalDetalles` (Inyección HTML y Presentación de Ficha)

JavaScript

```
function renderizarModalDetalles(p) {
   const container = document.getElementById('modal-content');
   const poster = (p.Poster && p.Poster !== 'N/A') ? p.Poster : POSTER_DEFAULT;
```

- **Línea 11:** Declaración de la función `renderizarModalDetalles(p)`. Recibe el objeto enriquecido `p` devuelto previamente por `obtenerDetallesPeliculaAPI()`.
- **Línea 12:** Obtiene la referencia al contenedor interno del modal (`#modal-content`).
- **Línea 13:** Operador ternario de validación de póster. Si la propiedad no existe o es `'N/A'`, usa la constante de fallback `POSTER_DEFAULT`.

JavaScript

```
  // Se utilizan p.PlotEs y p.AwardsEs con el texto en español latino

   container.innerHTML = `

      <div class="md:col-span-5 relative bg-slate-900">

         <img src="${poster}"

            alt="${p.Title}"

            onerror="this.onerror=null; this.src='${POSTER_DEFAULT}';"

            class="w-full h-full object-cover min-h-[320px]">

      </div>
```

- **Líneas 16-23:** Construcción del HTML interno mediante Template Literals.
- **`md:col-span-5`**: En escritorio asigna 5 de las 12 columnas del modal al póster principal.
- **`onerror="this.onerror=null; this.src='${POSTER_DEFAULT}';"`**: Captura errores de red en la carga de la imagen (links rotos o 404) y reemplaza la fuente en caliente por la imagen por defecto.

JavaScript

```
      <div class="md:col-span-7 p-6 space-y-4 flex flex-col justify-between">

         <div class="space-y-2">

            <div class="flex items-center gap-2">

               <span class="px-2.5 py-0.5 text-[10px] font-bold bg-amber-500/10
text-amber-400 border border-amber-500/20 rounded-md">

                  ⭐ ${p.imdbRating} / 10

               </span>
```

${p.Runtime}

Español Latino

<h2 class="text-xl font-extrabold text-white">${p.Title}</h2>

${p.PlotEs || p.Plot}

- **Línea 24:** Contenedor de la información que ocupa las 7 columnas restantes (`md:col-span-7`).
- **Líneas 27-31:** Inyecta la calificación de IMDb (`${p.imdbRating}`), la duración (`${p.Runtime}`) y muestra una etiqueta informativa *"Español Latino"*.
- **Línea 33:** Inyecta la **sinopsis traducida al español latino** (`${p.PlotEs}`). El operador **OR (`||`)** actúa como fallback defensivo: si por algún motivo la traducción no se generó, muestra el texto original en inglés (`p.Plot`).

JavaScript

Director: ${p.Director}

Elenco: ${p.Actors}

Premios: ${p.AwardsEs || p.Awards}

- **Líneas 36-40:** Sección de ficha técnica con borde superior.
- **Línea 39:** Inyecta el texto de **Premios traducido** (`${p.AwardsEs || p.Awards}`) con un resaltado en color amarillo dorado (`text-amber-300`).

JavaScript

<a href="https://www.imdb.com/title/${p.imdbID}" target="_blank" class="w-full py-2.5 bg-amber-500 hover:bg-amber-400 text-slate-950 font-bold text-xs rounded-xl transition text-center flex items-center justify-center gap-1.5">

 Ver Ficha Completa en IMDb

```
            </a>

        </div>

    `;


    document.getElementById('modal-details').classList.remove('hidden');

}
```

- **Líneas 42-44:** Botón de hipervínculo externo que redirige a la página oficial de IMDb usando la propiedad `p.imdbID`. `target="_blank"` abre el enlace en una pestaña nueva.
- **Línea 47: Revelado del Modal**: Remueve la clase `hidden` del contenedor `#modal-details` para hacer visible la ventana emergente en pantalla.

Este bloque implementa el **renderizado dinámico de las tarjetas del comparador**. Toma el arreglo de películas procesadas concurrentemente con Promise.all() y genera las columnas comparativas con sus métricas clave (recaudación, calificación, votos) y la opción de quitar cada título individualmente.

Explicación línea por línea

1. Firma de la Función y Limpieza del Contenedor

JavaScript

```
function renderizarComparador(lista) {

    const container = document.getElementById('comparator-grid');

    container.innerHTML = '';
```

- **Línea 1:** Declaración de la función sincrónica renderizarComparador(lista). Recibe el arreglo lista con los objetos de películas devueltos por cargarComparacionConcurrente().
- **Línea 2:** Obtiene la referencia al contenedor Grid del modal comparador (#comparator-grid).
- **Línea 3:** container.innerHTML = '': Vacía el contenido previo del modal (eliminando el spinner de carga o tarjetas anteriores) para evitar duplicados al redibujar.

2. Iteración y Manejo Defensivo del Póster

JavaScript

```
lista.forEach(p => {

    const poster = (p.Poster && p.Poster !== 'N/A') ? p.Poster : POSTER_DEFAULT;
```

- **Línea 5:** Recorre cada objeto de película p del arreglo usando .forEach().
- **Línea 6:** Operador ternario de validación de póster. Si la API devuelve un valor válido (distinto de null, undefined o 'N/A'), asigna p.Poster; de lo contrario, asigna la constante de respaldo POSTER_DEFAULT.

3. Creación de la Columna Comparativa (DOM Element)

JavaScript

```
    const col = document.createElement('div');

    col.className = 'bg-slate-900/80 p-4 rounded-2xl border border-slate-800 space-y-3 flex flex-col justify-between';
```

- **Línea 8:** Crea un elemento en memoria con document.createElement('div').
- **Línea 9:** Asigna las clases CSS de Tailwind:
  - bg-slate-900/80 p-4 rounded-2xl border border-slate-800: Fondo oscuro semitransparente con relleno de 16px, bordes de 16px y contorno azul sutil.
  - flex flex-col justify-between: Estructura Flexbox vertical que empuja el botón de "Quitar" al fondo de la tarjeta, manteniendo alineadas todas las columnas sin importar la longitud del título.

4. Inyección del Template HTML (Metadatos y Métrica)

JavaScript

```
    col.innerHTML = `
        <div class="space-y-3">
            <div class="flex gap-3">
                <img src="${poster}" onerror="this.onerror=null; this.src='${POSTER_DEFAULT}';" class="w-16 h-24 object-cover rounded-lg shadow">
                <div>
                    <h4 class="font-bold text-xs text-white">${p.Title}</h4>
                    <p class="text-[11px] text-slate-400">${p.Year} • ${p.Genre}</p>
                    <span class="inline-block mt-1 px-2 py-0.5 text-[10px] font-bold bg-amber-500/20 text-amber-400 rounded">
```

```
                ⭐ ${p.imdbRating}
            </span>
        </div>
    </div>
```

- **Líneas 11-23:** Construcción del encabezado de la ficha dentro del Template Literal:
  - Muestra la miniatura de la imagen (w-16 h-24) con el evento defensivo onerror para prevenir imágenes rotas.
  - Muestra el título (${p.Title}), año de estreno (${p.Year}), género (${p.Genre}) y la puntuación de IMDb (${p.imdbRating}) resaltada en dorado.

5. Métricas Financieras/Populares y Botón de Eliminación

JavaScript

```
        <div class="text-[11px] text-slate-300 space-y-1 bg-slate-950/50 p-2.5 rounded-xl border border-slate-800">
            <p><strong>Recaudación:</strong> ${p.BoxOffice || 'N/A'}</p>
            <p><strong>Votos IMDb:</strong> ${p.imdbVotes || 'N/A'}</p>
        </div>
    </div>
```

- **Líneas 24-27:** Bloque de métricas comparativas clave:
  - ${p.BoxOffice || 'N/A'}: Muestra la cifra de recaudación en taquilla (ej. $292,576,195). Si el campo no viene en la API, el operador || muestra 'N/A'.
  - ${p.imdbVotes || 'N/A'}: Muestra el total de votos registrados en IMDb.

JavaScript

```
        <button class="btn-remove-compare w-full py-1.5 bg-rose-950/60 hover:bg-rose-900 text-rose-300 text-xs font-semibold rounded-xl border border-rose-800/60 transition" data-id="${p.imdbID}">
            Quitar
        </button>
    `;
```

```
        container.appendChild(col);
    });
}
```

- **Líneas 29-31:** Botón de acción para eliminar la película del comparador:
  - class="btn-remove-compare ...": Identificador de clase utilizado por la **delegación de eventos**.
  - data-id="${p.imdbID}": Atributo dataset que almacena el imdbID para saber qué película remover al hacer clic.
- **Línea 33:** container.appendChild(col): Inserta físicamente la columna terminada dentro del contenedor #comparator-grid.
- **Línea 35:** Cierra la función renderizarComparador.

Este bloque de código administra la **gestión de estado reactivo, la persistencia en almacenamiento local (localStorage) y la actualización de badges en el encabezado**.

Explicación línea por línea

1. Función toggleFavorito (Alternar película en Favoritos)

JavaScript

```
// 4. LÓGICA DE FAVORITOS Y ESTADO
// ----------------------------------------------------------------

function toggleFavorito(imdbID) {
    const idx = EstadoApp.favoritos.findIndex(f => f.imdbID === imdbID);
```

- **Líneas 1-3:** Comentario de sección y declaración de la función toggleFavorito(imdbID).
- **Línea 4:** Utiliza el método de arreglo .findIndex() para buscar la posición dentro de EstadoApp.favoritos donde coincida el imdbID. Si la película ya existe en el arreglo, devuelve su índice (≥ 0); de lo contrario, devuelve -1.

JavaScript

```
    if (idx >= 0) {
        EstadoApp.favoritos.splice(idx, 1);
```

```
        mostrarToast("Película removida de favoritos", "info");
    } else {
        const item = EstadoApp.resultados.find(r => r.imdbID === imdbID);
        if (item) {
            EstadoApp.favoritos.push(item);
            mostrarToast("Guardada en favoritos", "exito");
        }
    }
```

- **Línea 6: Eliminación**: Si el índice es mayor o igual a 0 (idx >= 0), significa que la película ya figuraba como favorita.
- **Línea 7:** EstadoApp.favoritos.splice(idx, 1): Modifica el arreglo mutándolo directamente para remover 1 elemento en la posición idx.
- **Línea 8:** Emite una notificación flotante de información (*Toast*).
- **Línea 9: Inclusión**: Si la película no estaba en favoritos (idx < 0).
- **Línea 10:** Localiza la referencia del objeto completo de la película dentro de EstadoApp.resultados con .find().
- **Líneas 11-14:** Si el ítem existe, lo añade al arreglo EstadoApp.favoritos mediante .push(item) y lanza la notificación de éxito.

JavaScript

```
    localStorage.setItem('cinestream_favs', JSON.stringify(EstadoApp.favoritos));
    actualizarBadgesHeader();
```

- **Línea 17: Persistencia en LocalStorage**: Convierte el arreglo de objetos JavaScript a una cadena de texto JSON con JSON.stringify() y lo guarda bajo la clave 'cinestream_favs'.
- **Línea 18:** Invoca actualizarBadgesHeader() para refrescar el número visible en el botón del header.

JavaScript

```
    if (EstadoApp.vistaActiva === 'favoritos') {
        renderizarGrillaPeliculas(EstadoApp.favoritos);
        renderizarPaginacion();
```

```
    } else {
        renderizarGrillaPeliculas(EstadoApp.resultados);
    }
}
```

- **Líneas 20-25: Redibujado condicional según la vista activa**:
  - Si el usuario está viendo la pestaña de favoritos (vistaActiva === 'favoritos'), vuelve a renderizar la grilla enviando EstadoApp.favoritos (lo que remueve la tarjeta inmediatamente si fue desmarcada) y actualiza los controles de paginación.
  - Si está en la vista general de búsqueda, redibuja la grilla enviando EstadoApp.resultados para refrescar el color del icono del corazón en la tarjeta.

2. Función toggleComparar (Alternar película en Comparador)

JavaScript

```
function toggleComparar(imdbID) {
    const idx = EstadoApp.listaComparar.indexOf(imdbID);
```

- **Línea 27:** Declaración de la función toggleComparar(imdbID).
- **Línea 28:** Utiliza .indexOf(imdbID) sobre EstadoApp.listaComparar (que es un arreglo plano de cadenas con los IDs) para verificar si la película ya fue agregada al comparador.

JavaScript

```
    if (idx >= 0) {
        EstadoApp.listaComparar.splice(idx, 1);
        mostrarToast("Removida del comparador", "info");
    } else {
        if (EstadoApp.listaComparar.length >= 3) {
            mostrarToast("Máximo 3 películas para comparar", "error");
            return;
        }
```

```
        EstadoApp.listaComparar.push(imdbID);

        mostrarToast("Agregada al comparador", "exito");

    }
```

- **Líneas 30-32:** Si ya está en la lista, remueve el ID usando splice(idx, 1) y notifica al usuario.
- **Líneas 33-37: Validación de Límite Máximo**: Si el usuario intenta agregar una cuarta película (length >= 3), muestra una alerta de error y detiene la ejecución con return.
- **Líneas 38-39:** Si no excede el límite, agrega el imdbID al arreglo con .push() y confirma la acción.

JavaScript

```
    localStorage.setItem('cinestream_compare', JSON.stringify(EstadoApp.listaComparar));

    actualizarBadgesHeader();

    renderizarGrillaPeliculas(EstadoApp.resultados);

}
```

- **Línea 42:** Serializa y guarda la lista de comparación en el almacenamiento del navegador con la clave 'cinestream_compare'.
- **Líneas 43-44:** Actualiza los badges del header y redibuja la grilla de búsqueda para actualizar la etiqueta del botón (*"Comparar"* vs. *"En Comparador"*).

3. Función actualizarBadgesHeader (Actualización de Indicadores Visuales)
JavaScript

```
function actualizarBadgesHeader() {

    document.getElementById('badge-fav-count').textContent = EstadoApp.favoritos.length;

    document.getElementById('badge-compare-count').textContent =
EstadoApp.listaComparar.length;

}
```

- **Línea 47:** Declaración de la función auxiliar actualizarBadgesHeader().
- **Línea 48:** Actualiza el texto del badge #badge-fav-count con la cantidad total de favoritos guardados (EstadoApp.favoritos.length).

- **Línea 49:** Actualiza el texto del badge #badge-compare-count con el total de ítems en el comparador (EstadoApp.listaComparar.length).

Este bloque constituye el **sistema central de listeners e interacciones de usuario**. Aplica técnicas clave de arquitectura web como la **Delegación de Eventos (*Event Delegation*)**, captura de formularios con prevención de recarga y el control interactivo de estados visuales.

Explicación línea por línea

1. Delegación de Eventos en la Grilla Principal (`movies-grid`)

JavaScript

```
// 5. MANEJO DE EVENTOS

// ------------------------------------------------------------------


document.getElementById('movies-grid').addEventListener('click', async (e) => {

    const btnDetails = e.target.closest('.btn-open-details');

    const btnFav = e.target.closest('.btn-toggle-fav');

    const btnCompare = e.target.closest('.btn-toggle-compare');
```

- **Líneas 1-4:** Asigna un único *Event Listener* de clic al contenedor padre `'movies-grid'` usando una función flecha asíncrona (`async (e)`).
- **Líneas 5-7 (Delegación de Eventos / Event Bubbling):** En lugar de adjuntar decenas de listeners a cada tarjeta generada dinámicamente, se escucha en el padre y se utiliza `e.target.closest('selector')`:
    - `e.target`: El elemento exacto que recibió el clic (por ejemplo, el texto o el icono interno `<i class="fa-solid...">`).
    - `.closest(...)`: Sube por el árbol del DOM hasta encontrar el ancestro más cercano que coincida con la clase especificada (`.btn-open-details`, `.btn-toggle-fav` o `.btn-toggle-compare`). Si el clic ocurrió fuera de estos botones, devuelve `null`.

JavaScript

```
    if (btnDetails) {

        const id = btnDetails.getAttribute('data-id');
```

```
        mostrarToast("Cargando detalles...", "info");
        const detalles = await obtenerDetallesPeliculaAPI(id);
        if (detalles) renderizarModalDetalles(detalles);
    }

    if (btnFav) toggleFavorito(btnFav.getAttribute('data-id'));
    if (btnCompare) toggleComparar(btnCompare.getAttribute('data-id'));
});
```

- **Líneas 9-14:** Si se presionó el botón de detalles (`btnDetails` no es `null`):
    1. Extrae el atributo personal `data-id` (`imdbID`).
    2. Muestra un mensaje flotante (*Toast*) de carga.
    3. Ejecuta la llamada asíncrona `await obtenerDetallesPeliculaAPI(id)` (que además traduce la sinopsis y los premios en segundo plano).
    4. Si la consulta retorna datos válidos, abre y renderiza el modal.
- **Líneas 16-17:** Si se presionó el botón de favorito o comparador, extrae la propiedad `data-id` e invoca a sus respectivas funciones de alternancia de estado (`toggleFavorito` o `toggleComparar`).

2. Delegación de Eventos en el Modal Comparador

JavaScript

```
document.getElementById('comparator-grid').addEventListener('click', (e) => {
    if (e.target.classList.contains('btn-remove-compare')) {
        toggleComparar(e.target.getAttribute('data-id'));
        cargarComparacionConcurrente();
    }
});
```

- **Líneas 19-20:** Escucha clics dentro del contenedor de tarjetas del comparador (`'comparator-grid'`).

- **Líneas 21-23:** Evalúa si el elemento presionado contiene la clase `'btn-remove-compare'`. Si es así, elimina el `imdbID` del estado mediante `toggleComparar()` y vuelve a ejecutar `cargarComparacionConcurrente()` para refrescar la vista en vivo con `Promise.all()`.

3. Captura y Envío del Formulario de Búsqueda
JavaScript

```
document.getElementById('form-search').addEventListener('submit', (e) => {

    e.preventDefault();

    const query = document.getElementById('input-search').value.trim();

    const tipo = document.getElementById('select-type').value;



    if (query) {

        EstadoApp.vistaActiva = 'busqueda';

        document.getElementById('section-title').innerHTML = `<i class="fa-solid fa-search text-brand-red"></i> Resultados para "${query}"`;

        buscarPeliculasAPI(query, tipo, 1);

    }

});
```

- **Línea 26:** Escucha el evento `'submit'` del formulario principal (`'form-search'`).
- **Línea 27:** `e.preventDefault()`: **Fundamental en Single Page Applications (SPA)**. Cancela el comportamiento nativo del navegador de recargar toda la página al enviar un formulario.
- **Líneas 28-29:** Obtiene el texto tecleado limpiando espacios al inicio/final (`.trim()`) y el tipo de contenido seleccionado (`movie`, `series` o todos).
- **Líneas 31-34:** Si hay un texto válido, cambia la vista activa a `'busqueda'`, actualiza el título visual de la sección e inicia la búsqueda desde la página 1 (`buscarPeliculasAPI(query, tipo, 1)`).

4. Navegación de Vistas y Búsqueda Aleatoria
JavaScript

```
document.getElementById('btn-show-favorites').addEventListener('click', () => {

    EstadoApp.vistaActiva = 'favoritos';
```

```
    document.getElementById('section-title').innerHTML = `<i class="fa-solid fa-heart text-brand-red"></i> Mis Películas Favoritas`;

    renderizarGrillaPeliculas(EstadoApp.favoritos);

    renderizarPaginacion();

    actualizarContadorResultados(`Mostrando ${EstadoApp.favoritos.length} guardados`);

});
```

- **Líneas 37-43:** Listener para el botón de la cabecera "Favoritos":
    1. Cambia el estado a `vistaActiva = 'favoritos'`.
    2. Actualiza el título a *"Mis Películas Favoritas"*.
    3. Renderiza únicamente la colección guardada en `localStorage`.
    4. Oculta la paginación de la API invocando a `renderizarPaginacion()`.

JavaScript

```
document.getElementById('btn-open-comparator').addEventListener('click', () => {

    document.getElementById('modal-comparator').classList.remove('hidden');

    cargarComparacionConcurrente();

});
```

- **Líneas 45-48:** Abre el modal comparador removiendo la clase `'hidden'` y ejecuta la carga concurrente.

JavaScript

```
document.getElementById('btn-quick-trends').addEventListener('click', () => {

    const randomQuery = TITULOS_ALEATORIOS[Math.floor(Math.random() * TITULOS_ALEATORIOS.length)];

    document.getElementById('input-search').value = randomQuery;

    EstadoApp.vistaActiva = 'busqueda';

    buscarPeliculasAPI(randomQuery, "", 1);

});
```

- **Líneas 50-55:** Selección aleatoria de franquicias:
    - `Math.floor(Math.random() * TITULOS_ALEATORIOS.length)`: Genera un índice aleatorio dentro del rango del arreglo.
    - Escribe la franquicia elegida en el campo de texto y dispara la búsqueda inmediatamente.

5. Botones de Paginación Anterior / Siguiente
JavaScript

```
// Eventos de Botones Anterior / Siguiente

document.getElementById('btn-page-prev').addEventListener('click', () => {

    cambiarPagina(EstadoApp.paginaActual - 1);

});

document.getElementById('btn-page-next').addEventListener('click', () => {

    cambiarPagina(EstadoApp.paginaActual + 1);

});
```

- **Líneas 58-63:** Asigna los listeners a los botones direccionales de la paginación invocando `cambiarPagina()` con el decreciente (`paginaActual - 1`) o el creciente (`paginaActual + 1`).

6. Controles de Cierre y Configuración de API Key
JavaScript

```
// Cierre de Modales

document.getElementById('btn-close-modal').addEventListener('click', () => {

    document.getElementById('modal-details').classList.add('hidden');

});

document.getElementById('btn-close-comparator').addEventListener('click', () => {

    document.getElementById('modal-comparator').classList.add('hidden');

});
```

- **Líneas 66-71:** Ocultan los modales de detalles y comparador añadiéndoles la clase `'hidden'`.

JavaScript

```
document.getElementById('btn-clear-compare').addEventListener('click', () => {
    EstadoApp.listaComparar = [];
    localStorage.setItem('cinestream_compare', JSON.stringify([]));
    actualizarBadgesHeader();
    cargarComparacionConcurrente();
});
```

- **Líneas 72-77:** Resetea por completo la lista de comparación (`[]`), limpia el `localStorage`, refresca los badges y vuelve a renderizar el modal (que ahora mostrará el aviso de comparador vacío).

JavaScript

```
document.getElementById('btn-config-api').addEventListener('click', () => {
    const key = prompt("Ingresa tu OMDb API Key:", OMDB_API_KEY);
    if (key) {
        OMDB_API_KEY = key.trim();
        mostrarToast("API Key guardada", "exito");
    }
});
```

- **Líneas 79-85:** Permite ingresar una nueva clave mediante la ventana emergente `prompt()`. Si el usuario provee un valor, actualiza la variable `OMDB_API_KEY` en tiempo de ejecución.

Este bloque final cierra la arquitectura de la aplicación, agrupando las **funciones helper de interfaz de usuario (UI)** y el **punto de entrada global de arranque (`window.onload`)**.

Explicación línea por línea

1. Control del Spinner de Carga (`mostrarCargando`)

JavaScript

```
// UTILERÍAS

function mostrarCargando(flag) {

    const spinner = document.getElementById('search-spinner');

    if (flag) spinner.classList.remove('hidden');

    else spinner.classList.add('hidden');

}
```

- **Líneas 1-3:** Comentario de encabezado y firma de la función `mostrarCargando`. Acepta el parámetro booleano `flag` (`true`/`false`).
- **Línea 4:** Selecciona el nodo del indicador de carga (`#search-spinner`).
- **Líneas 5-6:** Alterna la visibilidad con Tailwind: si `flag` es `true`, remueve la clase `hidden` para hacer visible la animación de rotación; si es `false`, oculta el spinner.

2. Actualización de Texto en Header (`actualizarContadorResultados`)
JavaScript

```
function actualizarContadorResultados(texto) {

    document.getElementById('results-count').textContent = texto;

}
```

- **Líneas 8-10:** Función utilitaria de una sola línea. Recibe una cadena de texto y actualiza directamente la propiedad `textContent` del elemento `#results-count` para reflejar el conteo de páginas y resultados de la API.

3. Presentación del Estado Vacío (`mostrarEstadoVacio`)
JavaScript

```
function mostrarEstadoVacio(titulo, desc) {

    document.getElementById('movies-grid').classList.add('hidden');

    document.getElementById('pagination-container').classList.add('hidden');

    const emptyState = document.getElementById('empty-state');

    document.getElementById('empty-title').textContent = titulo;
```

```
    document.getElementById('empty-desc').textContent = desc;

    emptyState.classList.remove('hidden');
}
```

- **Línea 12:** Firma de la función `mostrarEstadoVacio`, que recibe el título y la descripción personalizada para el error o estado sin resultados.
- **Líneas 13-14:** Ocasiona la salida visual limpia agregando la clase `hidden` tanto a la rejilla de tarjetas (`#movies-grid`) como a la barra de paginación (`#pagination-container`).
- **Líneas 15-17:** Selecciona el contenedor de estado vacío e inyecta dinámicamente los valores en `#empty-title` y `#empty-desc`.
- **Línea 18:** Muestra el componente llamando a `classList.remove('hidden')`.

4. Sistema de Notificaciones Flotantes / Toast (`mostrarToast`)

JavaScript

```
function mostrarToast(mensaje, tipo = 'exito') {

    const container = document.getElementById('toast-container');

    const toast = document.createElement('div');

    const colorBg = tipo === 'exito' ? 'bg-emerald-600' : tipo === 'error' ? 'bg-rose-600' :
'bg-cyan-600';
```

- **Línea 20:** Firma de la función `mostrarToast`. Define un parámetro opcional con valor por defecto `tipo = 'exito'`.
- **Línea 21:** Captura la referencia del contenedor flotante `#toast-container`.
- **Línea 22:** Crea en memoria una nueva etiqueta `<div>` con `document.createElement('div')`.
- **Línea 23:** Evaluador ternario doble para la paleta de colores:
  - `'exito'`: Fondo verde esmeralda (`bg-emerald-600`).
  - `'error'`: Fondo rojo rosa (`bg-rose-600`).
  - Cualquier otro tipo (`'info'`): Fondo cian (`bg-cyan-600`).

JavaScript

```
   toast.className = `${colorBg} text-white text-xs font-semibold px-4 py-3 rounded-xl
shadow-xl flex items-center gap-2 transform transition duration-300 pointer-events-auto`;

    toast.innerHTML = `<i class="fa-solid fa-circle-info"></i> <span>${mensaje}</span>`;
```

```
    container.appendChild(toast);
```

- **Línea 25:** Aplica las clases de estilo con un Template Literal. `pointer-events-auto` restaura los eventos de interacción dentro del elemento, a pesar de que el contenedor padre tenga `pointer-events-none`.
- **Línea 26:** Inyecta el icono de información y la cadena `${mensaje}`.
- **Línea 28:** `container.appendChild(toast)`: Inserta físicamente la alerta flotante en el árbol del DOM.

JavaScript

```
  setTimeout(() => {
      toast.classList.add('opacity-0', 'translate-y-2');
      setTimeout(() => container.removeChild(toast), 300);
   }, 3000);
}
```

- **Líneas 30-34 (Ciclo de Vida del Toast):**
  1. Primer `setTimeout` a los **3000 milisegundos (3 segundos)**: Inicia la animación de desvanecimiento añadiendo las clases de opacidad 0 y desplazamiento suave hacia abajo (`translate-y-2`).
  2. Segundo `setTimeout` anidado a los **300 milisegundos** (duración del CSS `duration-300`): Ejecuta `container.removeChild(toast)` para eliminar el nodo del DOM y evitar fuga de memoria (*memory leaks*).

5. Punto de Entrada e Inicialización (`window.onload`)

JavaScript

```
// INICIALIZACIÓN
window.onload = () => {
    actualizarBadgesHeader();
    buscarPeliculasAPI("Batman", "", 1);
};
```

- **Línea 37:** Comentario del bloque de inicialización.
- **Línea 38:** Asigna la función callback al evento `window.onload`, garantizando que todo el HTML, CSS y recursos hayan cargado completamente antes de ejecutar JavaScript.
- **Línea 39:** `actualizarBadgesHeader()`: Lee la memoria `localStorage` y coloca el conteo inicial en los badges de Favoritos y Comparador.
- **Línea 40:** `buscarPeliculasAPI("Batman", "", 1)`: Dispara la primera búsqueda automática de portada para poblar la grilla inicial.
- **Línea 41:** Cierra la función callback del evento `window.onload`.

6. Cierre del Script y del Documento HTML

HTML
```
  </script>
</body>
</html>
```

- **Línea 42:** Cierra la etiqueta `<script>` del código JavaScript.
- **Líneas 43-44:** Cierran el cuerpo (`</body>`) y el documento HTML (`</html>`).

¡Excelente! Hemos completado el análisis y explicación **línea por línea** de todo el código de este segundo proyecto (**CineStream Explorer**).

Proyecto 3: Sistema de Gestión de Tareas y Proyectos "Kanban/Trello"

Dificultad: 🔴 **Media-Alta** (Enfoque en POO, Herencia, Estado Complejo y Eventos del Ratón)

Un sistema visual completo tipo tablero Kanban con columnas (Por hacer, En proceso, Completado), donde cada tarea es un objeto instanciado desde clases.

- **¿Qué hace el proyecto?**
  - Define una estructura jerárquica: `Task` (clase base), `BugTask` y `FeatureTask` (clases hijas con atributos específicos).
  - Gestor general `ProjectManager` (clase estática o singleton) para orquestar la lógica.
  - Permite mover tareas entre estados, asignar prioridades y eliminarlas del DOM.
  - Sincroniza el estado de las clases directamente con `localStorage` serializando los objetos.
- **Temas cubiertos del temario:**

    - **Programación Orientada a Objetos:** Clases, `constructor`, `extends`, `super()`, métodos y propiedades estáticas, encapsulamiento.
    - **Objetos Avanzados:** `Object.create()`, `Constructor Functions` (para comparar sintaxis moderna vs tradicional).
    - **Eventos:** Eventos de ratón (`click`, `mouseover`, `mouseout`), drag & drop sencillo o manejo de formularios.
    - **Buenas Prácticas:** Principio de responsabilidad única (Single Responsibility Principle) repartido entre las clases y las funciones del DOM.
- **Puntos de dificultad para los estudiantes:**
    - Entender la diferencia entre las instancias de las Clases y la representación visual en el DOM.
    - Reconstruir los objetos e instancias al recuperar los datos JSON desde `localStorage` (pérdida de métodos tras `JSON.parse`).

https://github.com/aristides1000/Sistema-de-Gestion-de-Tareas-y-Proyectos-Kanban-Trello-

https://aristides1000.github.io/Sistema-de-Gestion-de-Tareas-y-Proyectos-Kanban-Trello-/

Aquí tienes la explicación detallada **línea por línea** del bloque de cabecera HTML, la configuración de Tailwind CSS y las reglas personalizadas de CSS para el tablero Kanban:

1. Declaración HTML e Idioma

HTML
<!DOCTYPE html>
<html lang="es" class="dark">

- **Línea 1:** Declaración `<!DOCTYPE html>` que le indica al navegador que el documento utiliza el estándar **HTML5**.
- **Línea 2:** Tag `<html lang="es" class="dark">`. Define el idioma del contenido en español y asigna la clase `dark` global para activar el modo oscuro nativo de Tailwind CSS.

2. Metadatos de la Cabecera (`<head>`)

HTML
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TaskFlow Kanban Pro - POO & Hydration System</title>

- **Líneas 3-6:** Establece la codificación de caracteres en `UTF-8` (admite acentos, tildes y emojis), configura el *viewport* responsivo (`width=device-width, initial-scale=1.0`) para que la pantalla se ajuste al ancho de dispositivos móviles y fija el título en la pestaña del navegador.

3. Carga y Personalización de Tailwind CSS

HTML

```
  <!-- Tailwind CSS CDN -->
   <script src="https://cdn.tailwindcss.com"></script>
   <script>
      tailwind.config = {
          darkMode: 'class',
          theme: {
              extend: {
                  colors: {
                      slate: { 850: '#111827', 900: '#0f172a', 950: '#050914' },
                      brand: { blue: '#3B82F6', purple: '#8B5CF6', rose: '#F43F5E', amber:
'#F59E0B' }
                  },
                  fontFamily: { sans: ['Inter', 'sans-serif'] }
              }
          }
      }
   </script>
```

- **Línea 8:** Importa el motor de Tailwind CSS directamente desde su CDN oficial.
- **Líneas 9-22:** Extensión del objeto de configuración global `tailwind.config`:
  - `darkMode: 'class'`: Configura el cambio de modo oscuro basado en la presencia de la clase `dark` en la etiqueta `<html>`.
  - `extend.colors`: Amplía la paleta de colores predeterminada añadiendo tonos oscuros de la gama `slate` (`#111827`, `#0f172a`, `#050914`) y la paleta personalizada `brand` (azul, púrpura `#8B5CF6`, rosa y ámbar).
  - `fontFamily`: Fija la fuente tipográfica predeterminada a la familia `Inter`.

4. Importación de Fuentes e Iconos Externos

HTML

```
  <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&displ
ay=swap" rel="stylesheet">
   <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
```

- **Líneas 23-27:** Realiza la preconexión y carga de la tipografía **Inter** (pesos desde 300 hasta 800) mediante Google Fonts, además de la librería de iconos vectoriales **FontAwesome 6.4.0**.

5. Estilos CSS Personalizados (`<style>`)

HTML

```
  <style>
      body { font-family: 'Inter', sans-serif; background-color: #050914; color: #f3f4f6; }
      ::-webkit-scrollbar { width: 6px; height: 6px; }
      ::-webkit-scrollbar-track { background: #0f172a; }
      ::-webkit-scrollbar-thumb { background: #334155; border-radius: 4px; }
```

- **Líneas 29-33:** Asigna el color de fondo ultra-oscuro `#050914` y texto claro `#f3f4f6` al cuerpo de la página. Personaliza la barra de desplazamiento (*scrollbar*) para que sea delgada (`6px`), discreta y acorde al tema.

HTML

```
      .glass-card {
          background: rgba(15, 23, 42, 0.75);
          backdrop-filter: blur(16px);
          border: 1px solid rgba(255, 255, 255, 0.08);
          box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
      }
      .glass-input {
          background: rgba(30, 41, 59, 0.6);
          border: 1px solid rgba(255, 255, 255, 0.12);
          color: #f8fafc;
      }
      .glass-input:focus {
          border-color: #8B5CF6;
          outline: none;
          box-shadow: 0 0 0 2px rgba(139, 92, 246, 0.3);
      }
```

- **Líneas 35-51:** Implementa el diseño **Glassmorphism**:
  - `.glass-card`: Otorga transparencia con desenfoque de fondo (`backdrop-filter: blur(16px)`), bordes translúcidos y sombra suave.
  - `.glass-input`: Da apariencia esmerilada a las cajas de texto y cambia su borde al color púrpura `#8B5CF6` al recibir el foco del cursor (`:focus`).

HTML

```
      .kanban-column {
          min-height: 520px;
          transition: background-color 0.2s ease;
      }
```

```
.kanban-column.drag-over {
    background-color: rgba(139, 92, 246, 0.12);
    border-style: dashed;
}
```

- **Líneas 52-59:** Reglas específicas para las columnas del tablero Kanban:
    - `.kanban-column`: Fija una altura mínima de `520px` para asegurar un área amplia de arrastre.
    - `.kanban-column.drag-over`: Clase aplicada por JavaScript durante los eventos *Drag & Drop*. Ilumina el fondo de la columna de púrpura y cambia el borde a punteado (`dashed`) cuando se sostiene una tarjeta sobre ella.

HTML

```
.task-card {
    cursor: grab;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.task-card:active { cursor: grabbing; }
.task-card.dragging {
    opacity: 0.4;
    transform: scale(0.96);
}
</style>
</head>
```

- **Líneas 60-70:** Reglas de cursor y animaciones para las tarjetas de tareas:
    - `cursor: grab`: Muestra el icono de la mano abierta al pasar por encima de la tarjeta.
    - `:active { cursor: grabbing; }`: Cambia a la mano cerrada al presionar y arrastrar.
    - `.task-card.dragging`: Reduce la opacidad al 40% y escala ligeramente hacia abajo (`scale(0.96)`) la tarjeta original mientras está en movimiento durante el arrastre.

Esta sección define la **etiqueta de apertura del cuerpo (`<body>`)** y el **encabezado principal (`<header>`) fijado en la parte superior**, organizando tanto la identidad visual de la aplicación como los botones de control global.

Explicación línea por línea

1. Apertura del `<body>` y Estilos de Fondo Globales

HTML

<body class="min-h-screen pb-12 antialiased selection:bg-brand-purple selection:text-white">

- **`min-h-screen`**: Garantiza que el cuerpo ocupe como mínimo el 100% de la altura de la ventana del navegador (`100vh`).
- **`pb-12`**: Agrega un padding inferior de `3rem` (`48px`) para despejar el espacio del tablero antes del final de la página.
- **`antialiased`**: Aplica suavizado de fuentes mediante propiedades CSS (`-webkit-font-smoothing: antialiased`), logrando que la tipografía luzca definida sobre el fondo oscuro.
- **`selection:bg-brand-purple selection:text-white`**: Personaliza el resaltado cuando el usuario selecciona texto en la pantalla, pintando el fondo púrpura y las letras blancas.

2. Contenedor Flotante del Encargado (`<header>`)

HTML

```
  <!-- Header Principal -->
   <header class="border-b border-slate-800/80 bg-slate-950/80 sticky top-0 z-40 backdrop-blur-xl">
```

- **`border-b border-slate-800/80`**: Traza una línea divisoria inferior de 1px en tono azul oscuro con 80% de opacidad.
- **`bg-slate-950/80`**: Fondo semi-transparente muy oscuro (`#050914` al 80%).
- **`sticky top-0`**: Convierte el header en una **barra fija adhesiva**: permanece pegado al borde superior de la pantalla mientras el usuario hace scroll.
- **`z-40`**: Asigna un índice de capas elevado para que las tarjetas de las columnas pasen por debajo del header al deslizarse.
- **`backdrop-blur-xl`**: Aplica un desenfoque de fondo agresivo (*Glassmorphic blur*), difuminando el contenido que se desplaza detrás de la barra.

3. Maquetación Responsiva y Marca de la Aplicación

HTML

```
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4">
      <div class="flex flex-col md:flex-row md:items-center md:justify-between gap-4">
```

- **`max-w-7xl mx-auto ...`**: Limita el ancho máximo del contenido a `1280px` y lo centra horizontalmente en pantallas anchas con padding adaptativo (`px-4`, `sm:px-6`, `lg:px-8`).
- **`flex flex-col md:flex-row ...`**: Disposición en columna en dispositivos móviles (`flex-col`) que se transforma en una fila horizontal (`md:flex-row`) a partir del punto de interrupción mediano (`768px`).

HTML

```
            <div class="flex items-center space-x-3">
                <div class="p-2.5 bg-gradient-to-tr from-purple-600 to-indigo-500 rounded-xl shadow-lg shadow-purple-600/30">
                    <i class="fa-solid fa-kanban text-xl text-white"></i>
                </div>
```

- **`p-2.5 bg-gradient-to-tr ...`**: Crea la caja del logotipo con un degradado en diagonal (de púrpura a índigo), bordes redondeados (`12px`) y una resplandeciente sombra Púrpura (`shadow-purple-600/30`).
- **`<i class="fa-solid fa-kanban ...">`**: Inyecta el icono de tablero Kanban de FontAwesome en blanco.

HTML

```
            <div>
                <div class="flex items-center space-x-2">
                    <h1 class="text-xl font-extrabold tracking-tight text-white flex items-center gap-1.5">
                        TaskFlow <span class="text-brand-purple">Kanban</span>
                    </h1>
                    <span class="px-2.5 py-0.5 text-[10px] font-bold bg-purple-500/10 text-purple-400 border border-purple-500/20 rounded-full">POO & HERENCIA</span>
                </div>
                <p class="text-xs text-slate-400">Proyecto 3: Clases, Polimorfismo, Encargados & Drag and Drop</p>
            </div>
        </div>
```

- **`h1`**: Título de la aplicación con la palabra *"Kanban"* resaltada en color púrpura.
- **`<span class="...">POO & HERENCIA</span>`**: Badge informativo con bordes semitransparentes que destaca el objetivo pedagógico del proyecto.
- **`<p class="text-xs text-slate-400">`**: Subtítulo descriptivo en letra pequeña y gris atenuado.

4. Barra de Botones de Acción Global

HTML

```
            <div class="flex items-center gap-3">
                <button id="btn-open-modal" class="px-4 py-2.5 bg-gradient-to-r from-purple-600 to-indigo-600 hover:from-purple-500 hover:to-indigo-500 text-white font-bold text-xs rounded-xl shadow-lg shadow-purple-600/25 transition flex items-center gap-2">
                    <i class="fa-solid fa-plus"></i> Nueva Tarea
                </button>
```

- **`#btn-open-modal`**: Botón primario de llamado a la acción (*CTA*).

    - **Estilos**: Fondo con degradado horizontal púrpura a índigo, sombra con brillo púrpura y efecto hover de aclarado.
    - **Función JS**: Abre la ventana modal para instanciar una nueva tarea en el estado.

HTML
```
                <button id="btn-seed-data" class="px-3 py-2.5 bg-slate-800 hover:bg-slate-700 text-slate-300 font-semibold text-xs rounded-xl border border-slate-700 transition" title="Generar Tareas Ejemplo">
                    <i class="fa-solid fa-wand-magic-sparkles text-amber-400"></i>
                </button>
```

- `#btn-seed-data`: Botón secundario con icono de varita mágica.
    - **Función JS**: Dispara el método `seedDefaultData()` en el `ProjectManager` para sembrar instancias predeterminadas de `BugTask` y `FeatureTask`.

HTML
```
                <button id="btn-clear-all" class="px-3 py-2.5 bg-slate-800 hover:bg-slate-700 text-rose-400 font-semibold text-xs rounded-xl border border-slate-700 transition" title="Vaciar Tablero">
                    <i class="fa-solid fa-trash"></i>
                </button>
            </div>

        </div>
    </div>
</header>
```

- `#btn-clear-all`: Botón de peligro representado por una papelera de reciclaje roja (`text-rose-400`).
    - **Función JS**: Invoca a `clearAll()` para limpiar el arreglo de tareas y reiniciar la clave en `localStorage`.
- **Líneas finales**: Cierran la maquetación del contenedor Flexbox y el bloque `<header>`.

Esta sección define la **estructura principal del Tablero Kanban (`<main>`)**, organizada en una grilla responsiva de tres columnas para los distintos estados de trabajo: **Por Hacer**, **En Proceso** y **Completado**.

Explicación línea por línea

1. Estructura Principal y Grilla Responsiva (`<main>`)

HTML
<!-- Tablero Kanban -->
<main class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 pt-8">

- **Línea 2:** Etiqueta semántica `<main>`. Centra la vista con `max-w-7xl mx-auto`, añade relleno lateral adaptativo (`px-4 sm:px-6 lg:px-8`) y un margen superior `pt-8` (`32px`) para separarla del header pegajoso (*sticky*).
- **Línea 3:** `<div class="grid grid-cols-1 md:grid-cols-3 gap-6">`: Activa el sistema **CSS Grid**.
  - `grid-cols-1`: En pantallas pequeñas (móviles), las columnas se apilan verticalmente.
  - `md:grid-cols-3`: A partir de pantallas medianas (`768px`), las muestra lado a lado en **3 columnas de igual ancho**.
  - `gap-6`: Establece una separación de `1.5rem` (`24px`) entre columnas.

2. Columna 1: Por Hacer (`todo`)

HTML
```
    <!-- Columna 1: Por Hacer (todo) -->
    <div class="flex flex-col space-y-4">
      <div class="flex items-center justify-between bg-slate-900/90 px-4 py-3 rounded-2xl border border-slate-800">
        <div class="flex items-center gap-2.5">
          <span class="w-3 h-3 rounded-full bg-amber-400 shadow-md shadow-amber-400/50"></span>
          <h2 class="text-sm font-bold text-white uppercase tracking-wider">Por Hacer</h2>
        </div>
        <span id="badge-count-todo" class="px-2.5 py-0.5 text-xs font-extrabold bg-slate-800 text-amber-400 rounded-full border border-slate-700">0</span>
      </div>
```

- **Línea 6:** `<div class="flex flex-col space-y-4">`: Contenedor vertical que agrupa el encabezado de la columna y la zona receptora de tarjetas, separándolos por `16px` (`space-y-4`).
- **Líneas 7-13: Encabezado de Estado**:
  - Contenedor con bordes redondeados (`rounded-2xl`) y fondo oscuro semitransparente (`bg-slate-900/90`).
  - `<span class="w-3 h-3 rounded-full bg-amber-400 ...">`: Indicador visual circular en tono amarillo/ámbar con resplandor (*glow*).
  - `<span id="badge-count-todo">`: Badge contador. Muestra la cantidad de instancias cuyo `status === 'todo'`. JavaScript actualiza su `textContent` dinámicamente.

HTML
```
        <div id="col-todo" data-status="todo" class="kanban-column glass-card rounded-2xl p-4 border border-slate-800/80 space-y-3 flex flex-col">
            <!-- Tarjetas inyectadas dinámicamente -->
        </div>
    </div>
```

- **Líneas 15-17: Contenedor Receptor Drag & Drop**:
    - `id="col-todo"`: Identificador del DOM que usa `ProjectManager.renderUI()` para inyectar las tarjetas correspondientes.
    - **`data-status="todo"`**: Atributo dataset fundamental. Cuando el usuario suelta (*drop*) una tarjeta en esta área, JavaScript lee `data-status` para actualizar el atributo `status` de la instancia.
    - `class="kanban-column ..."`: Aplica la altura mínima de `520px` y los eventos CSS de resaltado al arrastrar.

3. Columna 2: En Proceso (`in_progress`)

HTML
```
    <!-- Columna 2: En Proceso (in_progress) -->
    <div class="flex flex-col space-y-4">
        <div class="flex items-center justify-between bg-slate-900/90 px-4 py-3 rounded-2xl border border-slate-800">
            <div class="flex items-center gap-2.5">
                <span class="w-3 h-3 rounded-full bg-brand-blue shadow-md shadow-blue-400/50"></span>
                <h2 class="text-sm font-bold text-white uppercase tracking-wider">En Proceso</h2>
            </div>
            <span id="badge-count-progress" class="px-2.5 py-0.5 text-xs font-extrabold bg-slate-800 text-brand-blue rounded-full border border-slate-700">0</span>
        </div>

        <div id="col-in_progress" data-status="in_progress" class="kanban-column glass-card rounded-2xl p-4 border border-slate-800/80 space-y-3 flex flex-col">
            <!-- Tarjetas inyectadas dinámicamente -->
        </div>
    </div>
```

- **Líneas 20-33:** Sigue exactamente la misma arquitectura estructural que la primera columna, variando en:
    - **Identidad de Color**: Indicador y contador teñidos en azul (`bg-brand-blue`).
    - **Dataset e ID**: `id="col-in_progress"` y **`data-status="in_progress"`**, asociando los elementos que se estén ejecutando activamente.

- **Badge**: `<span id="badge-count-progress">` actualizado por JavaScript.

4. Columna 3: Completado (`done`)

HTML

```
      <!-- Columna 3: Completado (done) -->
       <div class="flex flex-col space-y-4">
          <div class="flex items-center justify-between bg-slate-900/90 px-4 py-3 rounded-2xl
border border-slate-800">
             <div class="flex items-center gap-2.5">
                <span class="w-3 h-3 rounded-full bg-emerald-400 shadow-md
shadow-emerald-400/50"></span>
                <h2 class="text-sm font-bold text-white uppercase
tracking-wider">Completado</h2>
             </div>
             <span id="badge-count-done" class="px-2.5 py-0.5 text-xs font-extrabold
bg-slate-800 text-emerald-400 rounded-full border border-slate-700">0</span>
          </div>

          <div id="col-done" data-status="done" class="kanban-column glass-card rounded-2xl
p-4 border border-slate-800/80 space-y-3 flex flex-col">
             <!-- Tarjetas inyectadas dinámicamente -->
          </div>
       </div>

    </div>
</main>
```

- **Líneas 36-52:** Tercera columna del flujo de trabajo:
  - **Identidad de Color**: Tono verde esmeralda (`bg-emerald-400`) para denotar éxito o tarea finalizada.
  - **Dataset e ID**: `id="col-done"` y **`data-status="done"`**.
  - **Cierre**: Cierra la grilla de 3 columnas (`</div>`) y la etiqueta principal `<main>`.

Este bloque implementa la interfaz modal superpuesta (*overlay*) para la recolección de atributos y la posterior instanciación de clases, junto con el contenedor flotante para el sistema de notificaciones (*Toasts*).

Explicación línea por línea

1. Contenedor Principal del Modal (Overlay & Blur)

HTML

```
<!-- MODAL DE CREACIÓN DE TAREAS -->
```

- **Línea 2:**
  - `id="modal-task"`: Identificador del DOM manipulado por JavaScript mediante `classList.remove('hidden')` o `classList.add('hidden')`.
  - `fixed inset-0`: Posiciona el contenedor de forma fija ocupando el 100% de la pantalla (`top: 0, right: 0, bottom: 0, left: 0`).
  - `z-50`: Asegura que se renderice por encima de cualquier otro elemento de la vista (incluyendo el header con `z-40`).
  - `hidden`: Mantiene la ventana oculta por defecto hasta que el usuario haga clic en *"Nueva Tarea"*.
  - `bg-slate-950/80 backdrop-blur-md`: Oscurece el fondo al 80% y aplica un desenfoque (*glassmorphism*) a los elementos del tablero que quedan detrás.
  - `flex items-center justify-center p-4`: Centra perfectamente la tarjeta modal en el punto medio de la pantalla con un margen de seguridad de `16px`.

HTML

- **Línea 3:** Tarjeta contenedor del formulario.
  - `glass-card`: Aplica los estilos CSS de desenfoque y fondo traslúcido.
  - `w-full max-w-lg`: Ajuste responsivo con un ancho máximo estricto de `512px` (`max-w-lg`).
  - `rounded-3xl p-6`: Bordes suavizados de `24px` y relleno interno de `24px`.
  - `space-y-5`: Establece una separación vertical de `1.25rem` (`20px`) entre la cabecera del modal y el formulario.

2. Cabecera del Modal (Título y Botón de Cierre)
HTML

<h3 class="text-lg font-bold text-white flex items-center gap-2">
 Nueva Instancia de Tarea
</h3>
<button id="btn-close-modal" class="text-slate-400 hover:text-white text-lg"></button>

- **Línea 5:** Layout flexbox horizontal que distribuye el título a la izquierda y el botón de cierre a la derecha con una línea divisoria inferior (`border-b border-slate-800 pb-3`).
- **Líneas 6-8:** Título con icono FontAwesome teñido en púrpura (`text-brand-purple`).
- **Línea 9:** Botón `#btn-close-modal` con la "X" (`fa-xmark`) que dispara el evento de ocultación en JavaScript.

3. Formulario e Input de Título

HTML

```
<form id="form-task" class="space-y-4">
    <div>
        <label class="block text-xs font-bold text-slate-300 uppercase tracking-wider mb-1.5">Título de la Tarea</label>
        <input type="text" id="task-title" class="w-full py-2.5 px-3.5 text-sm rounded-xl glass-input font-medium" placeholder="Ej: Corregir fuga de memoria en WebSocket" required>
    </div>
```

- **Línea 11:** `<form id="form-task" class="space-y-4">`: Captura el evento `submit` en JavaScript.
- **Líneas 12-15:** Campo obligatorio (`required`) para el nombre o resumen de la tarea. Su valor es leído posteriormente para asignarlo a `this.title`.

4. Campo del Encargado / Responsable (*Assignee*)

HTML

```
<!-- CAMPO ENCARGADO / RESPONSABLE DE LA TAREA -->
<div>
    <label class="block text-xs font-bold text-slate-300 uppercase tracking-wider mb-1.5">Encargado / Responsable</label>
    <div class="relative">
        <i class="fa-solid fa-user absolute left-3.5 top-3 text-slate-500 text-xs"></i>
        <input type="text" id="task-assignee" class="w-full pl-9 pr-3.5 py-2.5 text-sm rounded-xl glass-input font-medium" placeholder="Ej: Arístides Molina, Lisbeth..." required>
    </div>
</div>
```

- **Líneas 18-24:**
  - `<div class="relative">`: Permite superponer el icono del usuario en el extremo izquierdo.
  - `pl-9`: Agrega un padding izquierdo de `36px` en el input para no solapar el texto con el icono.
  - `#task-assignee`: Atributo clave capturado para asignarlo al parámetro `assignee` en la clase base `Task`.

5. Selectores de Tipo de Clase y Prioridad (Grid de 2 Columnas)

HTML

```
<div class="grid grid-cols-2 gap-3">
    <div>
        <label class="block text-xs font-bold text-slate-300 uppercase tracking-wider mb-1.5">Tipo de Clase</label>
        <select id="task-type" class="w-full py-2.5 px-3.5 text-sm rounded-xl glass-input bg-slate-900 font-medium">
            <option value="bug">🐛 BugTask (Error)</option>
            <option value="feature">🚀 FeatureTask (Función)</option>
        </select>
    </div>

    <div>
        <label class="block text-xs font-bold text-slate-300 uppercase tracking-wider mb-1.5">Prioridad</label>
        <select id="task-priority" class="w-full py-2.5 px-3.5 text-sm rounded-xl glass-input bg-slate-900 font-medium">
            <option value="Baja">Baja</option>
            <option value="Media" selected>Media</option>
            <option value="Alta">Alta</option>
        </select>
    </div>
</div>
```

- **Línea 26:** Dispone los dos selectores lado a lado en 2 columnas iguales (`grid grid-cols-2 gap-3`).
- **Líneas 27-33:** `#task-type`: Menú desplegable que define si la tarea instanciará una subclase `BugTask` (`value="bug"`) o `FeatureTask` (`value="feature"`).
- **Líneas 35-42:** `#task-priority`: Determina la prioridad (`Baja`, `Media`, `Alta`). `selected` fija la opción por defecto.

6. Contenedor Dinámico y Botones del Formulario

HTML

```
<!-- Campo Dinámico según la Subclase seleccionada -->
<div id="dynamic-fields" class="p-3.5 bg-slate-900/60 rounded-2xl border border-slate-800 space-y-3">
    <!-- Se inyectan campos específicos de BugTask o FeatureTask -->
</div>
```

- **Líneas 45-47:** `#dynamic-fields`: Área inyectada por la función `renderDynamicFields()`. Genera campos HTML específicos según el tipo de clase seleccionado (Severidad para Bugs, Impacto/Horas para Features).

HTML

```
            <div class="flex justify-end gap-3 pt-2">
                <button type="button" id="btn-cancel-task" class="px-4 py-2.5 text-xs
font-semibold text-slate-400 hover:text-white bg-slate-800 hover:bg-slate-700 rounded-xl
border border-slate-700 transition">
                    Cancelar
                </button>
                <button type="submit" class="px-5 py-2.5 bg-gradient-to-r from-purple-600
to-indigo-600 hover:from-purple-500 hover:to-indigo-500 text-white font-bold text-xs
rounded-xl shadow-lg shadow-purple-600/30 transition">
                    Instanciar Tarea
                </button>
            </div>
        </form>

    </div>
</div>
```

- **Líneas 50-52:** `#btn-cancel-task` con `type="button"` para cerrar el modal sin enviar el formulario.
- **Líneas 53-55:** Botón con `type="submit"` que dispara el proceso de instanciación con el operador `new`.

7. Contenedor Global de Notificaciones (Toasts)

HTML

```
<!-- Toast Notifications -->
<div id="toast-container" class="fixed bottom-5 right-5 z-50 flex flex-col gap-2
pointer-events-none"></div>
```

- **Línea 62:**
  - `id="toast-container"`: Nodo padre donde la función `mostrarToast()` inserta dinámicamente las alertas.
  - `fixed bottom-5 right-5`: Fija el contenedor en la esquina inferior derecha de la ventana.
  - `pointer-events-none`: Evita que el contenedor invisible bloquee los clics en los elementos subyacentes de la pantalla.

Esta es la **clase base `Task`**, el pilar fundamental de la arquitectura **Orientada a Objetos (POO)** de la aplicación. Define la estructura común, las propiedades compartidas y la interfaz base para los métodos que aplicarán **polimorfismo** en las subclases.

Explicación línea por línea

1. Apertura del Script y Definición de la Clase Base

JavaScript

```
<script>
   //
==========================================================================
   // 1. ARQUITECTURA DE CLASES (POO, HERENCIA Y POLIMORFISMO)
   //
==========================================================================

   /**
    * Clase Base: Task
    */
   class Task {
```

- **Línea 1:** Abre el bloque ejecutable de JavaScript dentro del documento HTML.
- **Líneas 2-8:** Comentarios organizadores que marcan el inicio del módulo de POO.
- **Línea 9:** Declaración de la clase `Task` utilizando la sintaxis moderna de ES6. Actúa como el molde base para todas las tareas del sistema.

2. Método Constructor y Parámetros por Defecto

JavaScript

```
      constructor(id, title, priority = 'Media', status = 'todo', assignee = 'Sin asignar') {
```

- **Línea 10:** Método especial `constructor()` que se ejecuta automáticamente cada vez que se crea un objeto con la palabra clave `new Task(...)`.
- **Parámetros por defecto (ES6):**
  - `id`: Identificador único de la tarea.
  - `title`: Título o descripción corta (obligatorio).
  - `priority = 'Media'`: Asigna `'Media'` si no se proporciona una prioridad.
  - `status = 'todo'`: Estado inicial en el tablero (`'todo'`, `'in_progress'`, `'done'`).
  - `assignee = 'Sin asignar'`: Nombre del encargado asignado por defecto.

3. Asignación de Propiedades de Instancia

JavaScript

```
         this.id = id || 'task-' + Date.now() + '-' + Math.floor(Math.random() * 1000);
          this.title = title;
          this.priority = priority;
          this.status = status; // 'todo', 'in_progress', 'done'
          this.assignee = assignee || 'Sin asignar';
          this.createdAt = new Date().toLocaleDateString('es-ES', { month: 'short', day:
'numeric' });
       }
```

- **Línea 11: Generador de ID Único**: Si `id` llega como `null` o `undefined` (al instanciar una tarea nueva), genera una cadena única uniendo la marca de tiempo exacta (`Date.now()`) con un entero aleatorio entre 0 y 999 (`Math.floor(Math.random() * 1000)`).
- **Líneas 12-15:** Asigna los argumentos recibidos a las propiedades de la instancia actual usando la palabra clave `this`.
- **Línea 16:** Genera la fecha de creación en formato corto en español (ej. `"24 oct"`), utilizando el método `toLocaleDateString()`.

4. Método Generador de Badge de Prioridad

JavaScript

```
    // Método genérico para el badge de prioridad
    getPriorityBadge() {
        const colors = {
            'Alta': 'bg-rose-500/10 text-rose-400 border-rose-500/20',
            'Media': 'bg-amber-500/10 text-amber-400 border-amber-500/20',
            'Baja': 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20'
        };
        return `<span class="px-2 py-0.5 text-[10px] font-bold rounded-md border ${colors[this.priority]}">${this.priority}</span>`;
    }
```

- **Línea 19:** Método de instancia que retorna el marcado HTML estilizado para la etiqueta de prioridad.
- **Líneas 20-24: Diccionario de Mapeo de Estilos**: Objeto literal `colors` que asocia cada nivel de prioridad con sus clases CSS correspondientes (rojo para Alta, amarillo para Media, verde para Baja).
- **Línea 25:** Retorna un Template Literal que inyecta dinámicamente la clase CSS seleccionada (`colors[this.priority]`) y el texto de la prioridad.

5. Interfaz Base para Polimorfismo

JavaScript

```
    // Método polimórfico (sobreescrito por subclases)
    getTypeBadgeHTML() {
        return `<span class="px-2 py-0.5 text-[10px] font-bold bg-slate-800 text-slate-300 rounded-md border border-slate-700">General</span>`;
    }

    getSpecificDetailHTML() {
        return '';
    }
}
```

- **Líneas 28-30:** Método `getTypeBadgeHTML()`. Retorna la etiqueta genérica *"General"*. Este método está diseñado para ser **sobreescrito (*override*) por las subclases `BugTask` y `FeatureTask`**, aplicando el concepto de **polimorfismo**.
- **Líneas 32-34:** Método `getSpecificDetailHTML()`. Retorna una cadena vacía en la clase base. Las subclases lo sobreescriben para retornar metadatos específicos (severidad del error o impacto/horas estimadas).
- **Línea 35:** Cierra la definición de la clase `Task`.

Este bloque demuestra **Herencia (`extends`)** y **Polimorfismo (Sobreescritura de Métodos)** en ES6. Aquí creamos las dos subclases especializadas a partir de la clase base `Task`.

Explicación línea por línea

1. Subclase 1: `BugTask` (Herencia y Constructor)

JavaScript

```
/**
 * Subclase 1: BugTask (Hereda de Task)
 */
class BugTask extends Task {
```

- **Líneas 1-3:** Comentario descriptivo del bloque.
- **Línea 4:** Declaración de la clase `BugTask`. La palabra clave `extends Task` indica que **hereda** automáticamente todas las propiedades y métodos de la clase padre `Task`.

JavaScript

```
  constructor(id, title, priority, status, assignee, severity = 'Alta', stepsToReproduce = 'Sin pasos especificadas') {
      super(id, title, priority, status, assignee); // Pasa assignee al constructor padre
      this.type = 'bug';
      this.severity = severity;
      this.stepsToReproduce = stepsToReproduce;
  }
```

- **Línea 5:** Método `constructor()`. Recibe tanto los parámetros comunes (`id`, `title`, `priority`, `status`, `assignee`) como los específicos de un fallo (`severity`, `stepsToReproduce`).
- **Línea 6: `super(...)`**: **REGLA FUNDAMENTAL DE ES6**. En una clase hija, es **obligatorio** llamar a `super()` antes de usar `this`. Esto ejecuta el constructor de `Task`, inicializando `id`, `title`, `priority`, `status`, `assignee` y `createdAt`.

- **Líneas 7-9:** Inicializa los atributos propios de la subclase:
    - `this.type = 'bug'`: Identificador de tipo (útil para filtrado y rehidratación).
    - `this.severity`: Nivel de impacto del fallo (ej. `'Crítica'`).
    - `this.stepsToReproduce`: Pasos en texto para replicar el error.

2. Polimorfismo en `BugTask` (Sobreescritura)

```
JavaScript
  // Sobreescritura Polimórfica
   getTypeBadgeHTML() {
      return `<span class="px-2 py-0.5 text-[10px] font-bold bg-rose-500/10 text-rose-400
border border-rose-500/20 rounded-md"><i class="fa-solid fa-bug"></i> Bug</span>`;
   }

   getSpecificDetailHTML() {
      return `<p class="text-[11px] text-slate-400 truncate"><strong
class="text-slate-300">Severidad:</strong> ${this.severity}</p>`;
   }
}
```

- **Líneas 12-14: Sobreescritura polimórfica 1**: Reemplaza el `getTypeBadgeHTML()` genérico del padre para retornar una etiqueta con icono de cucaracha (`fa-bug`) y tonos rojizos (`bg-rose-500/10`).
- **Líneas 16-18: Sobreescritura polimórfica 2**: Reemplaza el `getSpecificDetailHTML()` vacío del padre para renderizar la severidad específica del reporte.
- **Línea 19:** Cierra la clase `BugTask`.

3. Subclase 2: `FeatureTask` (Herencia y Constructor)

```
JavaScript
/**
 * Subclase 2: FeatureTask (Hereda de Task)
 */
class FeatureTask extends Task {
   constructor(id, title, priority, status, assignee, impact = 'Backend', estimatedHours = 4) {
      super(id, title, priority, status, assignee); // Pasa assignee al constructor padre
      this.type = 'feature';
      this.impact = impact;
      this.estimatedHours = estimatedHours;
   }
```

- **Línea 24:** Declaración de `FeatureTask` heredando también de `Task` (`extends Task`).
- **Línea 25:** Constructor que añade parámetros específicos de nuevas funcionalidades (`impact` y `estimatedHours`).

- **Línea 26:** Invocación a `super(...)` para delegar la creación de la estructura base a la clase `Task`.
- **Líneas 27-29:** Asignación de propiedades exclusivas de la funcionalidad:
  - `this.type = 'feature'`.
  - `this.impact`: Área afectada (`'Frontend'`, `'Backend'`, `'UI/UX'`).
  - `this.estimatedHours`: Carga horaria proyectada.

4. Polimorfismo en `FeatureTask` (Sobreescritura)

```
JavaScript
  // Sobreescritura Polimórfica
   getTypeBadgeHTML() {
      return `<span class="px-2 py-0.5 text-[10px] font-bold bg-purple-500/10 text-purple-400
border border-purple-500/20 rounded-md"><i class="fa-solid fa-rocket"></i>
Feature</span>`;
   }

   getSpecificDetailHTML() {
      return `<p class="text-[11px] text-slate-400 truncate"><strong
class="text-slate-300">Impacto:</strong> ${this.impact} (${this.estimatedHours}h)</p>`;
   }
}
```

- **Líneas 32-34: Sobreescritura polimórfica 1**: Retorna un badge visual con icono de cohete (`fa-rocket`) y tono púrpura (`bg-purple-500/10`).
- **Líneas 36-38: Sobreescritura polimórfica 2**: Muestra el área de impacto junto a la estimación en horas.
- **Línea 39:** Cierra la clase `FeatureTask`.

Este bloque implementa el **Patrón Factory (Fábrica)** combinado con la técnica de **Hidratación de Objetos**. Resuelve uno de los problemas conceptuales más comunes en JavaScript Nivel 2: la **pérdida de prototipos y métodos** cuando un objeto se recupera desde `localStorage` tras haber sido serializado con `JSON.stringify()`.

Explicación línea por línea

1. Encabezado de Sección y Definición de la Clase Factory

```
JavaScript
// ====================================================================
// 2. PATRÓN FACTORY / HYDRATION (RECONSTRUCCIÓN DESDE LOCALSTORAGE)
// ====================================================================

class TaskFactory {
```

- **Líneas 1-3:** Comentario estructurado que indica la segunda sección del script JavaScript.
- **Línea 5:** Declaración de la clase `TaskFactory`. A diferencia de `Task`, esta clase no se utiliza para guardar datos directamente, sino como una **fábrica especializada en crear e instanciar objetos**.

2. Definición del Método Estático (`static`)

```
JavaScript
  /**
   * Convierte un objeto plano recuperado de JSON.parse en una Instancia real con
Métodos.
   */
  static createFromObject(obj) {
```

- **Líneas 6-8:** Comentario de documentación (JSDoc) que explica el propósito pedagógico del método.
- **Línea 9: Método Estático (`static`)**:
  - La palabra clave `static` permite invocar el método directamente desde la clase sin necesidad de crear una instancia previa con `new` (es decir, se ejecuta como `TaskFactory.createFromObject(obj)` en lugar de `const factory = new TaskFactory(); factory.createFromObject(...)`).
  - Recibe como parámetro `obj`, que representa un objeto literal plano devuelto por `JSON.parse()`.

3. Evaluación de Tipo y Reconstrucción de Subclase `BugTask`

```
JavaScript
    if (obj.type === 'bug') {
        return new BugTask(obj.id, obj.title, obj.priority, obj.status, obj.assignee, obj.severity,
obj.stepsToReproduce);
    }
```

- **Línea 10:** Evalúa la propiedad discriminante `obj.type === 'bug'`.
- **Línea 11: Rehidratación / Instanciación de `BugTask`**:
  - Ejecuta `new BugTask(...)` pasando individualmente todas las propiedades del objeto plano como argumentos del constructor.
  - **Efecto clave:** Devuelve una instancia real de `BugTask` que vuelve a conectar la cadena de prototipos, recuperando el acceso a métodos como `getTypeBadgeHTML()`, `getPriorityBadge()` y `getSpecificDetailHTML()`.

4. Evaluación de Tipo y Reconstrucción de Subclase `FeatureTask`

```
JavaScript
```

```
        else if (obj.type === 'feature') {
            return new FeatureTask(obj.id, obj.title, obj.priority, obj.status, obj.assignee,
obj.impact, obj.estimatedHours);
        }
```

- **Línea 12:** Verifica si el objeto guardado corresponde a una funcionalidad (`obj.type === 'feature'`).
- **Línea 13:** Instancia una clase real `FeatureTask` pasando sus atributos específicos (`impact` y `estimatedHours`), junto con las propiedades comunes y el nuevo campo `assignee`.

5. Fallback a Clase Base y Cierre de la Fábrica

```
JavaScript
        return new Task(obj.id, obj.title, obj.priority, obj.status, obj.assignee);
    }
}
```

- **Línea 15: Fallback (Cláusula de resguardo)**: Si el objeto no posee la propiedad `type` o su valor no coincide con `'bug'` ni `'feature'`, reconstruye el objeto como una instancia directa de la clase base `Task`.
- **Línea 16:** Cierra la función estática `createFromObject`.
- **Línea 17:** Cierra la definición de la clase `TaskFactory`.

Este bloque constituye el **corazón del control de estado y renderizado** de la aplicación. Aplica el **Patrón Singleton** para asegurar una única fuente de verdad en memoria, coordina la rehidratación desde `localStorage` e inyecta dinámicamente las tarjetas Kanban habilitando el sistema **Drag & Drop**.

Explicación línea por línea

1. Implementación del Patrón Singleton (`constructor`)

```
JavaScript
// ====================================================================
// 3. GESTOR DE ESTADO SINGLETON (ProjectManager)
// ====================================================================

class ProjectManager {
    constructor() {
        if (ProjectManager.instance) {
            return ProjectManager.instance;
        }
        this.tasks = [];
```

```
        this.init();
        ProjectManager.instance = this;
    }
```

- **Líneas 1-5:** Definición de la clase `ProjectManager`.
- **Líneas 7-9: Lógica del Patrón Singleton**: Verifica si la propiedad estática `ProjectManager.instance` ya existe. Si se intenta instanciar la clase por segunda vez (`new ProjectManager()`), detiene la creación y **retorna la instancia única previamente guardada**.
- **Línea 10:** `this.tasks = []`: Inicializa el arreglo principal que almacenará todas las instancias vivas de las tareas (`Task`, `BugTask`, `FeatureTask`).
- **Líneas 11-12:** Invoca la inicialización y asigna `this` a la propiedad estática `ProjectManager.instance` para congelar el Singleton.

2. Operaciones de Modificación de Estado y Persistencia

JavaScript

```
    init() {
        this.loadFromLocalStorage();
    }

    addTask(taskInstance) {
        this.tasks.push(taskInstance);
        this.saveToLocalStorage();
        this.renderUI();
    }

    removeTask(id) {
        this.tasks = this.tasks.filter(t => t.id !== id);
        this.saveToLocalStorage();
        this.renderUI();
    }

    updateTaskStatus(id, newStatus) {
        const task = this.tasks.find(t => t.id === id);
        if (task) {
            task.status = newStatus;
            this.saveToLocalStorage();
            this.renderUI();
        }
    }
```

- **`addTask(taskInstance)`**: Añade la nueva instancia recibida al arreglo `this.tasks`, guarda los cambios en disco e invoca el redibujado de la interfaz.
- **`removeTask(id)`**: Filtra el arreglo descartando la tarea cuyo `id` coincida con el parámetro recibido.

- **`updateTaskStatus(id, newStatus)`**: Localiza la tarea por su ID con `.find()` y modifica directamente la propiedad `status` (cambiándola a `'todo'`, `'in_progress'` o `'done'`) al arrastrar y soltar.

3. Lectura, Rehidratación y Sembrado de Datos (`LocalStorage`)

```
JavaScript
  saveToLocalStorage() {
      localStorage.setItem('taskflow_kanban_data', JSON.stringify(this.tasks));
  }

  loadFromLocalStorage() {
      const rawData = localStorage.getItem('taskflow_kanban_data');
      if (rawData) {
          try {
              const parsedArray = JSON.parse(rawData);
              // Hidratación de objetos planos a Clases reales
              this.tasks = parsedArray.map(obj => TaskFactory.createFromObject(obj));
          } catch (e) {
              console.error("Error al rehidratar datos:", e);
              this.tasks = [];
          }
      } else {
          this.seedDefaultData();
      }
  }
```

- **`saveToLocalStorage()`**: Serializa el arreglo de instancias a formato texto con `JSON.stringify()` bajo la clave `'taskflow_kanban_data'`.
- **`loadFromLocalStorage()`**:
  - Obtiene la cadena guardada en el navegador.
  - `JSON.parse(rawData)`: Convierte el texto JSON en un arreglo de objetos planos.
  - **`.map(obj => TaskFactory.createFromObject(obj))`**: **Punto Clave de Rehidratación**. Pasa cada objeto plano por la fábrica para reconstruirlo como una instancia real con prototipos y métodos de clase.
  - Si la clave no existe en el navegador (`else`), dispara `seedDefaultData()` para poblar el tablero por primera vez.

4. Limpieza y Sembrado Inicial (`clearAll` y `seedDefaultData`)

```
JavaScript
  clearAll() {
      this.tasks = [];
      this.saveToLocalStorage();
      this.renderUI();
  }
```

```
  seedDefaultData() {
      this.tasks = [
          new BugTask(null, "Error 500 al autenticar con OAuth2", "Alta", "todo", "Arístides
Molina", "Crítica", "1. Ir a /login 2. Clic en Google"),
          new FeatureTask(null, "Integración de pasarela de pago Stripe", "Media",
"in_progress", "Lisbeth Pérez", "Backend", 12),
          new FeatureTask(null, "Diseñar nuevo Landing Page en Tailwind", "Baja", "done",
"Valentina R.", "Frontend", 6)
      ];
      this.saveToLocalStorage();
      this.renderUI();
  }
```

- **`clearAll()`**: Vacía por completo el arreglo de tareas y sincroniza la interfaz.
- **`seedDefaultData()`**: Instancia tres tareas de demostración asignando encargados y propiedades reales a través de `new BugTask(...)` y `new FeatureTask(...)`.

5. Renderizado de Columnas en el DOM (`renderUI`)

JavaScript

```
  renderUI() {
      const columns = {
          todo: document.getElementById('col-todo'),
          in_progress: document.getElementById('col-in_progress'),
          done: document.getElementById('col-done')
      };

      // Vaciar columnas
      Object.values(columns).forEach(col => col.innerHTML = '');

      // Agrupar e inyectar tarjetas
      this.tasks.forEach(task => {
          const cardHTML = this.createCardElement(task);
          if (columns[task.status]) {
              columns[task.status].appendChild(cardHTML);
          }
      });

      // Actualizar contadores
      document.getElementById('badge-count-todo').textContent = this.tasks.filter(t =>
t.status === 'todo').length;
      document.getElementById('badge-count-progress').textContent = this.tasks.filter(t =>
t.status === 'in_progress').length;
      document.getElementById('badge-count-done').textContent = this.tasks.filter(t =>
t.status === 'done').length;
```

```
}
```

- **Líneas 67-72:** Mapea las tres columnas del DOM (`col-todo`, `col-in_progress`, `col-done`) en un objeto literal.
- **Línea 75:** `Object.values(columns).forEach(...)`: Limpia el contenido HTML de todas las columnas antes de volver a dibujar.
- **Líneas 78-83:** Recorre `this.tasks`. Crea el nodo elemento con `this.createCardElement(task)` y lo inserta en el contenedor cuya clave coincida con `task.status`.
- **Líneas 86-88:** Filtra las tareas según su estado y actualiza los badges de conteo numérico en la cabecera de cada columna.

6. Creación del Nodo e Invocación Polimórfica (`createCardElement`)

JavaScript

```
  createCardElement(task) {
      const card = document.createElement('div');
      card.id = task.id;
      card.draggable = true;
      card.className = 'task-card bg-slate-900/90 p-4 rounded-2xl border
border-slate-800/80 shadow-md hover:border-purple-500/40 flex flex-col justify-between
space-y-3';
```

- **`document.createElement('div')`**: Crea dinámicamente la tarjeta HTML en memoria.
- **`card.draggable = true`**: Habilita la **API nativa de Drag & Drop** en la tarjeta.

JavaScript

```
      card.innerHTML = `
          <div class="space-y-2">
              <div class="flex items-center justify-between">
                  <div class="flex items-center gap-1.5">
                      ${task.getTypeBadgeHTML()}
                      ${task.getPriorityBadge()}
                  </div>
                  <button class="btn-delete-task text-slate-500 hover:text-rose-400 text-xs
transition" data-id="${task.id}">
                      <i class="fa-solid fa-trash pointer-events-none"></i>
                  </button>
              </div>
              <h4 class="font-semibold text-xs text-white leading-snug">${task.title}</h4>
          </div>

          <div class="pt-2 border-t border-slate-800/60 flex flex-col gap-1.5">
              ${task.getSpecificDetailHTML()}
```

```
            <!-- Muestra del Encargado y Fecha de Creación -->
            <div class="flex items-center justify-between text-[10px] text-slate-400 pt-1">
                <span class="flex items-center gap-1 text-slate-300 font-medium truncate
max-w-[140px]" title="Encargado: ${task.assignee}">
                    <i class="fa-solid fa-user-circle text-purple-400"></i> ${task.assignee}
                </span>
                <span class="text-slate-500 font-medium"><i class="fa-regular fa-clock"></i>
${task.createdAt}</span>
            </div>
        </div>
    `;
```

- **Puntos de Polimorfismo**:
    - `${task.getTypeBadgeHTML()}`: Ejecuta el método sobreescrito de la subclase para renderizar el badge de Bug o Feature.
    - `${task.getSpecificDetailHTML()}`: Inyecta los metadatos específicos según la clase (Severidad o Impacto/Horas).
- Muestra la etiqueta del encargado (`task.assignee`) junto con el icono de usuario en color púrpura y la fecha formateada de creación (`task.createdAt`).

7. Eventos de Arrastre e Instanciación del Singleton

JavaScript

```
    // Eventos Drag & Drop individuales
    card.addEventListener('dragstart', (e) => {
        e.dataTransfer.setData('text/plain', task.id);
        card.classList.add('dragging');
    });

    card.addEventListener('dragend', () => {
        card.classList.remove('dragging');
    });

    return card;
  }
}

// Instancia Global Singleton
const manager = new ProjectManager();
```

- **`dragstart`**: Guarda el `id` de la tarjeta en el almacén de datos de transferencia del navegador (`e.dataTransfer.setData('text/plain', task.id)`) y añade la clase CSS `dragging` para reducir la opacidad durante el arrastre.
- **`dragend`**: Remueve la clase `dragging` cuando se suelta la tarjeta.
- **`const manager = new ProjectManager()`**: Congela e inicializa el **Singleton global**, disparando el ciclo de carga inicial.

Este bloque final conecta la interfaz de usuario con la arquitectura de clases, gestionando los eventos del formulario, las interacciones con el modal, la API nativa de **Drag & Drop** en las columnas y las notificaciones visuales (*Toasts*).

Explicación línea por línea

1. Referencias al DOM y Renderizado Dinámico de Formulario

JavaScript

```
// ========================================================================
// 4. INTERACCIONES DEL DOM Y FORMULARIOS
// ========================================================================

const modalTask = document.getElementById('modal-task');
const selectType = document.getElementById('task-type');
const dynamicFields = document.getElementById('dynamic-fields');
```

- **Líneas 1-3:** Encabezado de sección de interacciones.
- **Líneas 5-7:** Captura las referencias permanentes del DOM para el contenedor modal (#modal-task), el selector de tipo de tarea (#task-type) y el área de campos condicionales (#dynamic-fields).

JavaScript

```
// Renderizar campos dinámicos del modal según el tipo de Clase
function renderDynamicFields(type) {
    if (type === 'bug') {
        dynamicFields.innerHTML = `
            <div>
                <label class="block text-[11px] font-bold text-slate-300 uppercase
mb-1">Severidad del Error</label>
                <select id="bug-severity" class="w-full py-2 px-3 text-xs rounded-xl glass-input
bg-slate-900">
                    <option value="Menor">Menor</option>
                    <option value="Alta" selected>Alta</option>
                    <option value="Crítica">Crítica</option>
                </select>
            </div>
        `;
    } else {
        dynamicFields.innerHTML = `
            <div class="grid grid-cols-2 gap-2">
                <div>
                    <label class="block text-[11px] font-bold text-slate-300 uppercase mb-1">Área /
Impacto</label>
                    <select id="feature-impact" class="w-full py-2 px-3 text-xs rounded-xl
glass-input bg-slate-900">
                        <option value="Frontend">Frontend</option>
                        <option value="Backend" selected>Backend</option>
```

```
                    <option value="UI/UX">UI/UX</option>
                </select>
            </div>
            <div>
                <label class="block text-[11px] font-bold text-slate-300 uppercase mb-1">Horas
Estimadas</label>
                <input type="number" id="feature-hours" value="4" min="1" max="100"
class="w-full py-2 px-3 text-xs rounded-xl glass-input">
            </div>
        </div>
    `;
  }
}
```

- **Línea 10:** Función renderDynamicFields(type). Inyecta contenido HTML en el contenedor #dynamic-fields dependiendo de la subclase a instanciar.
- **Líneas 11-20:** Si type === 'bug', genera el menú desplegable de **Severidad del Error** (#bug-severity).
- **Líneas 21-38:** Si es una funcionalidad, inyecta los selectores de **Área/Impacto** (#feature-impact) y las **Horas Estimadas** (#feature-hours).

2. Controladores de Apertura y Cierre del Modal

JavaScript

```
// Listeners del Modal
document.getElementById('btn-open-modal').addEventListener('click', () => {
  renderDynamicFields(selectType.value);
  modalTask.classList.remove('hidden');
});

document.getElementById('btn-close-modal').addEventListener('click', () =>
modalTask.classList.add('hidden'));
document.getElementById('btn-cancel-task').addEventListener('click', () =>
modalTask.classList.add('hidden'));

selectType.addEventListener('change', (e) => renderDynamicFields(e.target.value));
```

- **Líneas 41-44:** Al hacer clic en el botón *"Nueva Tarea"*, renderiza primero los campos dinámicos acordes al valor actual del selector e invoca classList.remove('hidden') para revelar la ventana emergente.
- **Líneas 46-47:** Ocultan el modal agregando nuevamente la clase hidden al hacer clic en la "X" o en *"Cancelar"*.
- **Línea 49:** Escucha el evento change en #task-type. Si el usuario alterna entre "Bug" y "Feature", redibuja los campos dinámicos en tiempo real.

3. Procesamiento del Formulario e Instanciación POO

JavaScript

```
// Submit del Formulario: Instanciación Orientada a Objetos
document.getElementById('form-task').addEventListener('submit', (e) => {
```

```
    e.preventDefault();
    const title = document.getElementById('task-title').value.trim();
    const assignee = document.getElementById('task-assignee').value.trim();
    const type = selectType.value;
    const priority = document.getElementById('task-priority').value;

    let newTaskInstance = null;

    if (type === 'bug') {
        const severity = document.getElementById('bug-severity').value;
        // Instanciación directa con subclase BugTask
        newTaskInstance = new BugTask(null, title, priority, 'todo', assignee, severity);
    } else {
        const impact = document.getElementById('feature-impact').value;
        const hours = parseInt(document.getElementById('feature-hours').value, 10);
        // Instanciación directa con subclase FeatureTask
        newTaskInstance = new FeatureTask(null, title, priority, 'todo', assignee, impact, hours);
    }

    manager.addTask(newTaskInstance);
    modalTask.classList.add('hidden');
    document.getElementById('form-task').reset();
    mostrarToast("Nueva instancia creada exitosamente", "exito");
});
```

- **Línea 53:** e.preventDefault(): Detiene la recarga por defecto de la página.
- **Líneas 54-57:** Lee los datos del formulario limpiando espacios en blanco (.trim()).
- **Líneas 61-70: Punto Clave de Instanciación**:
    - Si el tipo es 'bug', lee #bug-severity y ejecuta new BugTask(null, title, priority, 'todo', assignee, severity). Passing null en el primer argumento indica a la clase base que genere un ID único.
    - De lo contrario, extrae #feature-impact junto con #feature-hours e instancia new FeatureTask(...).
- **Línea 72:** Agrega la instancia al Singleton con manager.addTask(newTaskInstance).
- **Líneas 73-75:** Cierra el modal, limpia los campos (.reset()) y emite una notificación de éxito.

4. Configuración de Zonas de Soltado Nativas (Drag & Drop)

JavaScript

```
// Configuración de Zonas de Soltado Drag & Drop (Columnas)
document.querySelectorAll('.kanban-column').forEach(column => {
    column.addEventListener('dragover', (e) => {
        e.preventDefault();
        column.classList.add('drag-over');
    });

    column.addEventListener('dragleave', () => {
        column.classList.remove('drag-over');
```

```
    });

    column.addEventListener('drop', (e) => {
        e.preventDefault();
        column.classList.remove('drag-over');
        const taskId = e.dataTransfer.getData('text/plain');
        const newStatus = column.getAttribute('data-status');

        manager.updateTaskStatus(taskId, newStatus);
        mostrarToast(`Tarea movida a ${newStatus.replace('_', ' ')}`, "info");
    });
});
```

- **Línea 79:** Selecciona las 3 columnas del tablero mediante document.querySelectorAll('.kanban-column').
- **Líneas 80-83 (dragover): Obligatorio para habilitar zonas de soltado**. Ejecutar e.preventDefault() le indica al navegador que la columna es una zona válida para soltar elementos. Agrega la clase drag-over para encender el borde punteado visual.
- **Líneas 85-87 (dragleave):** Remueve el efecto visual cuando el cursor arrastrado sale de la columna.
- **Líneas 89-97 (drop):** Ocurre al soltar la tarjeta sobre la columna:
    - e.dataTransfer.getData('text/plain'): Recupera el taskId adjuntado previamente durante el evento dragstart.
    - column.getAttribute('data-status'): Extrae el nuevo estado ('todo', 'in_progress' o 'done') leyendo el atributo dataset de la columna receptora.
    - manager.updateTaskStatus(taskId, newStatus): Actualiza el estado en el Singleton, guarda en localStorage y redibuja la interfaz.

5. Delegación de Eventos para Eliminación y Botones Globales

JavaScript

```
// Delegación de Eventos para Eliminación
document.addEventListener('click', (e) => {
    if (e.target.classList.contains('btn-delete-task')) {
        const id = e.target.getAttribute('data-id');
        manager.removeTask(id);
        mostrarToast("Instancia eliminada del estado", "info");
    }
});
```

- **Líneas 100-106: Delegación de Eventos en el Documento**: Captura el clic global y verifica si el objetivo (e.target) posee la clase .btn-delete-task. Si coincide, extrae el data-id y remueve la instancia mediante manager.removeTask(id).

JavaScript

```
// Botones de Utilidad en Header
document.getElementById('btn-seed-data').addEventListener('click', () => {
    manager.seedDefaultData();
    mostrarToast("Datos de prueba cargados", "exito");
```

```
});

document.getElementById('btn-clear-all').addEventListener('click', () => {
    if (confirm("¿Deseas vaciar todo el tablero?")) {
        manager.clearAll();
        mostrarToast("Tablero vaciado", "info");
    }
});
```

- **Líneas 109-112:** Carga datos de muestra invocando manager.seedDefaultData().
- **Líneas 114-119:** Solicita confirmación nativa y limpia el tablero si el usuario confirma.

6. Sistema de Notificaciones Toast e Inicialización Global

```
JavaScript
function mostrarToast(mensaje, tipo = 'exito') {
    const container = document.getElementById('toast-container');
    const toast = document.createElement('div');
    const colorBg = tipo === 'exito' ? 'bg-purple-600' : tipo === 'error' ? 'bg-rose-600' :
'bg-indigo-600';

    toast.className = `${colorBg} text-white text-xs font-semibold px-4 py-3 rounded-xl
shadow-xl flex items-center gap-2 transform transition duration-300 pointer-events-auto`;
    toast.innerHTML = `<i class="fa-solid fa-circle-info"></i> <span>${mensaje}</span>`;

    container.appendChild(toast);

    setTimeout(() => {
        toast.classList.add('opacity-0', 'translate-y-2');
        setTimeout(() => container.removeChild(toast), 300);
    }, 3000);
}

// Inicialización
window.onload = () => manager.renderUI();
</script>
</body>
</html>
```

- **Líneas 121-135:** Genera una alerta flotante animada que se destruye automáticamente del DOM a los 3.3 segundos para prevenir fugas de memoria.
- **Línea 138:** window.onload = () => manager.renderUI(): Punto de entrada principal. Garantiza que el DOM esté completamente cargado antes de renderizar la interfaz inicial a partir de las instancias rehidratadas desde localStorage.